Reviewer-Revised Research Report: Transformation Vectors and Composition Diagnostics

Date

2026-06-12

Purpose

Fix the reviewer-revised research state for external verification. This version incorporates AC-level review concerns: endpoint leakage, syntax-holdout overclaiming, McNemar tests, semantic-control statistics, and the terminology change to a third-order signed permutation coherence test.

Track 1 fixed result

Delta vectors add reproducible information beyond target-only endpoints in the multiseed full-semantic setting. The syntax=1.0 result has now been directly ablated: y_only also reaches 1.0, so the syntax split is interpreted as endpoint/surface leakage rather than as geometry. BERT-large and DeBERTa-v3-base spot-checks both rank delta best under Linear SVC and logistic regression. UPAT is now reported as a hard-holdout boundary where delta is not reliably better than y_only.

Track 2 fixed result

Composition order matters. Semantic equivalence shifts noncommutativity distributions with strong statistical tests. The third-order result is now framed as signed permutation coherence, not a formal Jacobi identity. Decoder pairwise composition is now included. After multiple-testing correction, QMT is the only below-null triple that is stable across all five tested models.

Main conclusion

The evidence supports a local QMT signed-permutation cancellation effect across tested encoder models, while negation-heavy triples expose the boundary of the phenomenon. RISE (Freenor and Alvarez, ICLR 2026) is now treated as the closest prior work for spherical/geodesic semantic-syntactic transformations across languages and embedding models. Our Track 3 result is therefore framed as a stress test: UPAT-large cross-model Procrustes transfer survives N=1000 random-label, random-pairing, random-orthogonal, and held-out-anchor controls. The first RISE-aware comparison shows strong within-model prototype target prediction, harder cross-model target prediction, and higher aligned delta-classifier transfer on its own class-discrimination metric.

Delta vs endpoint ablation with McNemar evidence

model	delta_mean	delta_std	y_only_mean	y_only_std	concat_mean	concat_std	x_only_mean	diff_delta_y	diff_delta_concat	max_mcnemar_p	all_mcnemar_sig_0_001
bert-base-uncased	0.9075	0.0117	0.8632	0.0167	0.8969	0.0185	0.1667	0.0443	0.0106	0.0000	1.0000
distilgpt2	0.8572	0.0125	0.7265	0.0094	0.7604	0.0107	0.1667	0.1307	0.0968	0.0000	1.0000
distilroberta-base	0.8810	0.0053	0.8306	0.0056	0.8417	0.0060	0.1667	0.0504	0.0394	0.0000	1.0000
gpt2	0.8332	0.0076	0.7496	0.0152	0.7765	0.0093	0.1667	0.0836	0.0567	0.0000	1.0000
roberta-base	0.8817	0.0090	0.8391	0.0072	0.8666	0.0089	0.1667	0.0426	0.0151	0.0000	1.0000

Syntax representation ablation: y_only solves the split

model	classifier	concat	delta	x_only	y_only
bert-base-uncased	linear_svc	1.0000	1.0000	0.1667	1.0000
bert-base-uncased	logreg	0.9904	0.9910	0.1667	0.9861
distilgpt2	linear_svc	1.0000	1.0000	0.1667	1.0000
distilgpt2	logreg	0.9976	1.0000	0.1667	0.9957
distilroberta-base	linear_svc	1.0000	1.0000	0.1667	1.0000
distilroberta-base	logreg	1.0000	1.0000	0.1667	1.0000
gpt2	linear_svc	1.0000	1.0000	0.1667	1.0000
gpt2	logreg	0.9979	0.9924	0.1667	0.9876
roberta-base	linear_svc	1.0000	1.0000	0.1667	1.0000
roberta-base	logreg	1.0000	1.0000	0.1667	1.0000

Layerwise/pooling syntax sanity check: top rows

classifier	accuracy	model	layer	pooling	representation	n_base	n_train	n_test
linear_svc	1.0000	bert-base-uncased	0.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	0.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	1.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	1.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	1.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	4.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	4.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	3.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	3.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	3.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	cls	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	12.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	7.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	6.0000	last_token	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	6.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	11.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	11.0000	mean	delta	200.0000	2400.0000	4800.0000

DeBERTa-v3-small modern spot-check

model	classifier	concat	delta	x_only	y_only
microsoft/deberta-v3-small	linear_svc	0.8225	0.8708	0.1667	0.8042
microsoft/deberta-v3-small	logreg	0.8283	0.7958	0.1667	0.7700

BERT-large and DeBERTa-v3-base spot-checks

model	classifier	concat	delta	x_only	y_only
bert-large-uncased	linear_svc	0.8508	0.9025	0.1667	0.8375
bert-large-uncased	logreg	0.7600	0.8542	0.1667	0.7500
microsoft/deberta-v3-base	linear_svc	0.7758	0.8117	0.1667	0.7467
microsoft/deberta-v3-base	logreg	0.6942	0.7517	0.1667	0.7258

UPAT hard-holdout representation ablation

model	feature	acc	f1	baseline
bert-base-uncased	x_only	0.2000	0.0667	0.2000
bert-base-uncased	y_only	0.8250	0.8266	0.2000
bert-base-uncased	delta	0.7250	0.7253	0.2000
bert-base-uncased	concat_xy	0.8000	0.7974	0.2000
distilroberta-base	x_only	0.2000	0.0667	0.2000
distilroberta-base	y_only	0.6500	0.6105	0.2000
distilroberta-base	delta	0.7250	0.6530	0.2000
distilroberta-base	concat_xy	0.6250	0.5703	0.2000
roberta-base	x_only	0.2000	0.0667	0.2000
roberta-base	y_only	0.7250	0.7210	0.2000
roberta-base	delta	0.6750	0.6333	0.2000
roberta-base	concat_xy	0.7250	0.7185	0.2000
gpt2	x_only	0.2000	0.0667	0.2000
gpt2	y_only	0.4750	0.4358	0.2000
gpt2	delta	0.4250	0.3481	0.2000
gpt2	concat_xy	0.4750	0.4338	0.2000
distilgpt2	x_only	0.2000	0.0959	0.2000
distilgpt2	y_only	0.6000	0.5441	0.2000
distilgpt2	delta	0.6500	0.5953	0.2000
distilgpt2	concat_xy	0.5750	0.5169	0.2000

UPAT delta vs y_only McNemar tests

model	compare	delta_acc	y_only_acc	diff	b_delta_correct_y_wrong	c_delta_wrong_y_correct	p_value
bert-base-uncased	delta_vs_y_only	0.7250	0.8250	-0.1000	2.0000	6.0000	0.2891
distilroberta-base	delta_vs_y_only	0.7250	0.6500	0.0750	4.0000	1.0000	0.3750
roberta-base	delta_vs_y_only	0.6750	0.7250	-0.0500	7.0000	9.0000	0.8036
gpt2	delta_vs_y_only	0.4250	0.4750	-0.0500	7.0000	9.0000	0.8036
distilgpt2	delta_vs_y_only	0.6500	0.6000	0.0500	5.0000	3.0000	0.7266

UPAT-large Procrustes N=1000 null controls

train_model	test_model	null_type	n_null	observed_raw_f1	observed_aligned_f1	observed_gain_f1	null_mean_f1	null_std_f1	null_max_f1	null_mean_acc	p_empirical_null_ge_obs	observed_minus_null_mean
bert-base-uncased	roberta-base	random_pairing	1000.0000	0.2462	0.6498	0.4036	0.1384	0.0659	0.4351	0.1995	0.0010	0.5115
bert-base-uncased	roberta-base	random_labels	1000.0000	0.2462	0.6498	0.4036	0.1717	0.0802	0.4751	0.2009	0.0010	0.4781
bert-base-uncased	roberta-base	random_orthogonal	1000.0000	0.2462	0.6498	0.4036	0.1172	0.0484	0.3105	0.2041	0.0010	0.5326
bert-base-uncased	distilroberta-base	random_pairing	1000.0000	0.2317	0.6133	0.3816	0.1352	0.0664	0.4506	0.2009	0.0010	0.4781
bert-base-uncased	distilroberta-base	random_labels	1000.0000	0.2317	0.6133	0.3816	0.1655	0.0800	0.5105	0.2005	0.0010	0.4478
bert-base-uncased	distilroberta-base	random_orthogonal	1000.0000	0.2317	0.6133	0.3816	0.1141	0.0474	0.2913	0.2041	0.0010	0.4991
bert-base-uncased	sentence-transformers/all-Mi	random_pairing	1000.0000	0.1657	0.7538	0.5881	0.1513	0.0718	0.4153	0.1990	0.0010	0.6025
bert-base-uncased	sentence-transformers/all-Mi	random_labels	1000.0000	0.1657	0.7538	0.5881	0.1746	0.0762	0.4602	0.1989	0.0010	0.5792
bert-base-uncased	sentence-transformers/all-Mi	random_orthogonal	1000.0000	0.1657	0.7538	0.5881	0.1288	0.0554	0.3847	0.2012	0.0010	0.6250
bert-base-uncased	sentence-transformers/all-mp	random_pairing	1000.0000	0.1517	0.8570	0.7053	0.1569	0.0805	0.5049	0.2026	0.0010	0.7001
bert-base-uncased	sentence-transformers/all-mp	random_labels	1000.0000	0.1517	0.8570	0.7053	0.1780	0.0831	0.5112	0.1986	0.0010	0.6790
bert-base-uncased	sentence-transformers/all-mp	random_orthogonal	1000.0000	0.1517	0.8570	0.7053	0.1243	0.0519	0.3036	0.2006	0.0010	0.7327
bert-base-uncased	microsoft/deberta-v3-b	random_pairing	1000.0000	0.1264	0.6845	0.5581	0.1543	0.0607	0.4212	0.1992	0.0010	0.5303
bert-base-uncased	microsoft/deberta-v3-b	random_labels	1000.0000	0.1264	0.6845	0.5581	0.1746	0.0691	0.4399	0.1967	0.0010	0.5099
bert-base-uncased	microsoft/deberta-v3-b	random_orthogonal	1000.0000	0.1264	0.6845	0.5581	0.1168	0.0434	0.2979	0.1995	0.0010	0.5677
roberta-base	bert-base-uncased	random_pairing	1000.0000	0.1995	0.6790	0.4795	0.1477	0.0618	0.4227	0.1996	0.0010	0.5313
roberta-base	bert-base-uncased	random_labels	1000.0000	0.1995	0.6790	0.4795	0.1792	0.0714	0.5296	0.1965	0.0010	0.4998
roberta-base	bert-base-uncased	random_orthogonal	1000.0000	0.1995	0.6790	0.4795	0.0969	0.0293	0.2374	0.2007	0.0010	0.5821
roberta-base	distilroberta-base	random_pairing	1000.0000	0.4374	0.6891	0.2517	0.1324	0.0646	0.3632	0.2006	0.0010	0.5566
roberta-base	distilroberta-base	random_labels	1000.0000	0.4374	0.6891	0.2517	0.1793	0.0879	0.5944	0.1954	0.0010	0.5097
roberta-base	distilroberta-base	random_orthogonal	1000.0000	0.4374	0.6891	0.2517	0.0848	0.0296	0.2680	0.1956	0.0010	0.6042
roberta-base	sentence-transformers/all-Mi	random_pairing	1000.0000	0.1512	0.7685	0.6174	0.1387	0.0698	0.4063	0.1987	0.0010	0.6298
roberta-base	sentence-transformers/all-Mi	random_labels	1000.0000	0.1512	0.7685	0.6174	0.1710	0.0835	0.5507	0.1944	0.0010	0.5975
roberta-base	sentence-transformers/all-Mi	random_orthogonal	1000.0000	0.1512	0.7685	0.6174	0.1026	0.0453	0.2628	0.1994	0.0010	0.6659
roberta-base	sentence-transformers/all-mp	random_pairing	1000.0000	0.0931	0.7804	0.6873	0.1438	0.0773	0.4447	0.1968	0.0010	0.6367
roberta-base	sentence-transformers/all-mp	random_labels	1000.0000	0.0931	0.7804	0.6873	0.1726	0.0870	0.5321	0.1938	0.0010	0.6078
roberta-base	sentence-transformers/all-mp	random_orthogonal	1000.0000	0.0931	0.7804	0.6873	0.0952	0.0429	0.3000	0.1994	0.0010	0.6852
roberta-base	microsoft/deberta-v3-b	random_pairing	1000.0000	0.4772	0.6277	0.1505	0.1535	0.0609	0.4175	0.2009	0.0010	0.4742
roberta-base	microsoft/deberta-v3-b	random_labels	1000.0000	0.4772	0.6277	0.1505	0.1836	0.0747	0.5324	0.1977	0.0010	0.4440
roberta-base	microsoft/deberta-v3-b	random_orthogonal	1000.0000	0.4772	0.6277	0.1505	0.0933	0.0305	0.2532	0.2005	0.0010	0.5343

UPAT-large held-out alignment-size curve

alignment_size	mean_f1	std_f1	mean_acc	mean_full_anchor_f1	mean_raw_f1	mean_gap_to_full_anchor	mean_gain_over_raw	n
25.0000	0.4520	0.1040	0.4986	0.6847	0.2415	-0.2326	0.2105	300.0000
50.0000	0.5732	0.0943	0.6072	0.6847	0.2415	-0.1114	0.3317	300.0000
100.0000	0.6294	0.0744	0.6602	0.6847	0.2415	-0.0552	0.3879	300.0000
250.0000	0.6537	0.0733	0.6841	0.6847	0.2415	-0.0310	0.4122	300.0000
500.0000	0.6591	0.0720	0.6908	0.6847	0.2415	-0.0256	0.4176	300.0000
1000.0000	0.6619	0.0753	0.6934	0.6847	0.2415	-0.0227	0.4204	300.0000

UPAT-large held-out alignment by direction: largest anchor size

train_model	test_model	alignment_size	mean_f1	std_f1	mean_acc	observed_full_anchor_f	observed_raw_f1	mean_gap_to_full_ancho	mean_gain_over_raw	n
microsoft/deberta-v3-ba	sentence-transformers/all-mpne	1000.0000	0.7444	0.0189	0.7522	0.8316	0.2290	-0.0872	0.5154	10.0000
microsoft/deberta-v3-ba	bert-base-uncased	1000.0000	0.6922	0.0037	0.7334	0.7614	0.1028	-0.0692	0.5894	10.0000
distilroberta-base	bert-base-uncased	1000.0000	0.5468	0.0096	0.6262	0.6137	0.3369	-0.0669	0.2100	10.0000
microsoft/deberta-v3-ba	roberta-base	1000.0000	0.6143	0.0083	0.6674	0.6762	0.4724	-0.0619	0.1419	10.0000
nce-transformers/all-Mini	microsoft/deberta-v3-ba	1000.0000	0.6589	0.0055	0.6594	0.7127	0.1444	-0.0538	0.5145	10.0000
ce-transformers/all-mpne	bert-base-uncased	1000.0000	0.5566	0.0112	0.5744	0.6090	0.1959	-0.0524	0.3606	10.0000
distilroberta-base	sentence-transformers/all-Mini	1000.0000	0.6919	0.0091	0.7456	0.7384	0.0995	-0.0465	0.5924	10.0000
microsoft/deberta-v3-ba	distilroberta-base	1000.0000	0.5391	0.0037	0.5890	0.5849	0.3539	-0.0458	0.1852	10.0000
roberta-base	sentence-transformers/all-Mini	1000.0000	0.7312	0.0088	0.7508	0.7685	0.1512	-0.0373	0.5800	10.0000
distilroberta-base	sentence-transformers/all-mpne	1000.0000	0.7212	0.0109	0.7370	0.7550	0.1983	-0.0338	0.5229	10.0000
nce-transformers/all-Mini	sentence-transformers/all-mpne	1000.0000	0.7279	0.0043	0.7474	0.7583	0.2265	-0.0304	0.5014	10.0000
roberta-base	bert-base-uncased	1000.0000	0.6508	0.0100	0.6760	0.6790	0.1995	-0.0282	0.4513	10.0000
nce-transformers/all-Mini	distilroberta-base	1000.0000	0.5601	0.0108	0.6252	0.5877	0.1320	-0.0276	0.4281	10.0000
ce-transformers/all-mpne	microsoft/deberta-v3-ba	1000.0000	0.6619	0.0110	0.6648	0.6894	0.1960	-0.0275	0.4659	10.0000
microsoft/deberta-v3-ba	sentence-transformers/all-Mini	1000.0000	0.7465	0.0088	0.7558	0.7730	0.2228	-0.0266	0.5237	10.0000
bert-base-uncased	microsoft/deberta-v3-ba	1000.0000	0.6610	0.0064	0.6780	0.6845	0.1264	-0.0236	0.5345	10.0000
roberta-base	sentence-transformers/all-mpne	1000.0000	0.7575	0.0084	0.7592	0.7804	0.0931	-0.0230	0.6643	10.0000
roberta-base	distilroberta-base	1000.0000	0.6702	0.0050	0.7172	0.6891	0.4374	-0.0189	0.2328	10.0000
roberta-base	microsoft/deberta-v3-ba	1000.0000	0.6110	0.0072	0.6128	0.6277	0.4772	-0.0167	0.1338	10.0000
distilroberta-base	roberta-base	1000.0000	0.6178	0.0084	0.6872	0.6339	0.4776	-0.0161	0.1402	10.0000
bert-base-uncased	distilroberta-base	1000.0000	0.5979	0.0088	0.6578	0.6133	0.2317	-0.0154	0.3662	10.0000
bert-base-uncased	sentence-transformers/all-mpne	1000.0000	0.8421	0.0114	0.8430	0.8570	0.1517	-0.0149	0.6904	10.0000
nce-transformers/all-Mini	bert-base-uncased	1000.0000	0.6688	0.0092	0.6758	0.6807	0.3491	-0.0119	0.3197	10.0000
distilroberta-base	microsoft/deberta-v3-ba	1000.0000	0.6004	0.0091	0.6152	0.6094	0.1740	-0.0090	0.4264	10.0000
nce-transformers/all-Mini	roberta-base	1000.0000	0.6285	0.0076	0.6758	0.6086	0.1254	0.0199	0.5031	10.0000
bert-base-uncased	roberta-base	1000.0000	0.6699	0.0072	0.7318	0.6498	0.2462	0.0201	0.4237	10.0000
ce-transformers/all-mpne	distilroberta-base	1000.0000	0.5536	0.0072	0.6002	0.5322	0.2739	0.0214	0.2797	10.0000
ce-transformers/all-mpne	sentence-transformers/all-Mini	1000.0000	0.6953	0.0068	0.7314	0.6681	0.3469	0.0272	0.3484	10.0000
ce-transformers/all-mpne	roberta-base	1000.0000	0.6437	0.0083	0.7064	0.6122	0.3084	0.0315	0.3353	10.0000
bert-base-uncased	sentence-transformers/all-Mini	1000.0000	0.7965	0.0087	0.8048	0.7538	0.1657	0.0427	0.6308	10.0000

UPAT RISE-aware prototype comparison

comparison	method	mean_target_cosine	std_target_cosine	mean_retrieval_top1_acc	mean_retrieval_label_acc	mean_retrieval_label_f1	mean_delta_classifier_f1	n
cross_model_full_anchor	mdv_raw	0.5769	0.1822	0.3157	0.4667	0.4114	0.6847	30.0000
cross_model_full_anchor	mdv_unit	0.6136	0.1583	0.3485	0.4632	0.4077	0.6847	30.0000
cross_model_full_anchor	rise_style	0.5783	0.1614	0.3778	0.5037	0.4455	0.6847	30.0000
within_model	mdv_raw	0.9224	0.0733	0.3603	0.4953	0.4567	0.7156	6.0000
within_model	mdv_unit	0.9218	0.0741	0.3677	0.5027	0.4635	0.7156	6.0000
within_model	rise_style	0.9230	0.0722	0.3740	0.5173	0.4773	0.7156	6.0000

Full-semantic pooling ablation

model	pooling	classifier	concat	delta	x_only	y_only
bert-base-uncased	cls	linear_svc	0.8269	0.8775	0.1667	0.7397
bert-base-uncased	cls	logreg	0.7031	0.7200	0.1667	0.6781
bert-base-uncased	last_token	linear_svc	0.7564	0.8514	0.1667	0.7508
bert-base-uncased	last_token	logreg	0.6792	0.7103	0.1667	0.6847
bert-base-uncased	mean	linear_svc	0.8850	0.8958	0.1667	0.8806
bert-base-uncased	mean	logreg	0.8725	0.8844	0.1667	0.8603
gpt2	cls	linear_svc	0.3683	0.3867	0.1667	0.3683
gpt2	cls	logreg	0.3422	0.3203	0.1667	0.3683
gpt2	last_token	linear_svc	0.7469	0.7564	0.1667	0.7200
gpt2	last_token	logreg	0.6453	0.6683	0.1667	0.6253
gpt2	mean	linear_svc	0.7822	0.8450	0.1667	0.7539
gpt2	mean	logreg	0.7803	0.7492	0.1667	0.7633
roberta-base	cls	linear_svc	0.8236	0.8736	0.1667	0.7911
roberta-base	cls	logreg	0.7944	0.8514	0.1667	0.7669
roberta-base	last_token	linear_svc	0.7656	0.7650	0.1667	0.7042
roberta-base	last_token	logreg	0.7056	0.6756	0.1667	0.6739
roberta-base	mean	linear_svc	0.8631	0.8786	0.1667	0.8431
roberta-base	mean	logreg	0.8150	0.8092	0.1667	0.8017

Confusion analysis: negation vs non-negation recall

source	model	classifier	group	mean_recall	mean_error_rate	n_classes
full_semantic	bert-base-uncased	linear_svc	negation	1.0000	0.0000	1.0000
full_semantic	bert-base-uncased	linear_svc	non_negation	0.8725	0.1275	5.0000
full_semantic	bert-base-uncased	logreg	negation	0.9862	0.0138	1.0000
full_semantic	bert-base-uncased	logreg	non_negation	0.8817	0.1182	5.0000
full_semantic	distilgpt2	linear_svc	negation	1.0000	0.0000	1.0000
full_semantic	distilgpt2	linear_svc	non_negation	0.8020	0.1980	5.0000
full_semantic	distilgpt2	logreg	negation	0.9725	0.0275	1.0000
full_semantic	distilgpt2	logreg	non_negation	0.7295	0.2705	5.0000
full_semantic	distilroberta-base	linear_svc	negation	1.0000	0.0000	1.0000
full_semantic	distilroberta-base	linear_svc	non_negation	0.8470	0.1530	5.0000
full_semantic	distilroberta-base	logreg	negation	1.0000	0.0000	1.0000
full_semantic	distilroberta-base	logreg	non_negation	0.8252	0.1747	5.0000
full_semantic	gpt2	linear_svc	negation	0.9600	0.0400	1.0000
full_semantic	gpt2	linear_svc	non_negation	0.7973	0.2027	5.0000
full_semantic	gpt2	logreg	negation	0.7312	0.2687	1.0000
full_semantic	gpt2	logreg	non_negation	0.7410	0.2590	5.0000
full_semantic	roberta-base	linear_svc	negation	0.9988	0.0012	1.0000
full_semantic	roberta-base	linear_svc	non_negation	0.8570	0.1430	5.0000
full_semantic	roberta-base	logreg	negation	0.9300	0.0700	1.0000
full_semantic	roberta-base	logreg	non_negation	0.7760	0.2240	5.0000

Third-order signed permutation summary with bootstrap CI

model	triple	mean_relative_jacobi_me	relative_jacobi_norm_ci95_	relative_jacobi_norm_ci95_	mean_jacobi_to_null_mean	jacobi_to_null_mean_ratio_ci9	jacobi_to_null_mean_ratio_ci9	mean_null_percentile_smaller_is	n
bert-base-uncased	NMT	0.3048	0.3005	0.3090	0.7750	0.7669	0.7829	0.3522	100.0000
bert-base-uncased	NQM	0.2695	0.2660	0.2727	0.8628	0.8533	0.8720	0.3850	100.0000
bert-base-uncased	NQT	0.3719	0.3674	0.3763	1.1170	1.1082	1.1259	0.7005	100.0000
bert-base-uncased	QMT	0.2760	0.2712	0.2807	0.6833	0.6773	0.6892	0.1018	100.0000
distilroberta-base	NMT	0.3522	0.3485	0.3558	1.1338	1.1276	1.1404	0.7066	100.0000
distilroberta-base	NQM	0.2847	0.2809	0.2882	1.0821	1.0735	1.0914	0.7777	100.0000
distilroberta-base	NQT	0.2777	0.2752	0.2802	1.0803	1.0765	1.0841	0.6274	100.0000
distilroberta-base	QMT	0.1849	0.1832	0.1866	0.6311	0.6267	0.6356	0.0997	100.0000
roberta-base	NMT	0.4315	0.4255	0.4371	1.2532	1.2423	1.2637	0.8057	100.0000
roberta-base	NQM	0.3122	0.3086	0.3158	0.9799	0.9743	0.9853	0.6475	100.0000
roberta-base	NQT	0.3452	0.3413	0.3490	1.0805	1.0739	1.0871	0.7059	100.0000
roberta-base	QMT	0.2190	0.2160	0.2222	0.6229	0.6168	0.6287	0.1013	100.0000

Decoder signed permutation replication

model	triple	mean_jacobi_norm	mean_relative_jacobi_norm	mean_null_relative_jacobi_norm	jacobi_to_null_mean_ratio	null_percentile_smaller	n	relative_jacobi_norm_ci95_lo	relative_jacobi_norm_ci95_hi	jacobi_to_null_mean_ratio_ci95_lo	jacobi_to_null_mean_ratio_ci95_hi
distilgpt2	NMT	10.3687	0.2276	0.2387	0.9507	0.5781	100.0000	0.2171	0.2388	0.9284	0.9722
distilgpt2	NQM	9.5684	0.2144	0.2540	0.8464	0.4061	100.0000	0.2056	0.2239	0.8249	0.8692
distilgpt2	NQT	15.8990	0.3385	0.2403	1.4202	0.8621	100.0000	0.3241	0.3537	1.3820	1.4555
distilgpt2	QMT	9.8263	0.2492	0.3179	0.7708	0.3207	100.0000	0.2320	0.2668	0.7446	0.7972
gpt2	NMT	31.1167	0.2894	0.2137	1.3348	0.7670	100.0000	0.2736	0.3065	1.2981	1.3723
gpt2	NQM	24.9467	0.2799	0.2315	1.1947	0.7302	100.0000	0.2642	0.2955	1.1675	1.2221
gpt2	NQT	16.4593	0.1562	0.1777	0.8666	0.4331	100.0000	0.1466	0.1658	0.8356	0.9004
gpt2	QMT	14.3721	0.1705	0.3167	0.5393	0.1852	100.0000	0.1621	0.1797	0.5192	0.5611

Signed permutation multiple-testing correction

model	triple	n	mean_ratio_to_null	bootstrap_p_ratio_less_than	direction	bonferroni_p	bh_fdr_p	passes_bonferroni_0_05	passes_bh_fdr_0_05
bert-base-uncased	NMT	100.0000	0.7750	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
bert-base-uncased	NQM	100.0000	0.8628	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
bert-base-uncased	QMT	100.0000	0.6833	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
distilgpt2	NQM	100.0000	0.8464	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
distilgpt2	QMT	100.0000	0.7708	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
distilroberta-base	QMT	100.0000	0.6311	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
gpt2	NQT	100.0000	0.8666	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
gpt2	QMT	100.0000	0.5393	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
roberta-base	NQM	100.0000	0.9799	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
roberta-base	QMT	100.0000	0.6229	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
distilgpt2	NMT	100.0000	0.9507	0.0001	below_null	0.0020	0.0002	1.0000	1.0000
bert-base-uncased	NQT	100.0000	1.1170	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
distilgpt2	NQT	100.0000	1.4202	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
distilroberta-base	NMT	100.0000	1.1338	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
distilroberta-base	NQM	100.0000	1.0821	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
distilroberta-base	NQT	100.0000	1.0803	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
gpt2	NMT	100.0000	1.3348	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
gpt2	NQM	100.0000	1.1947	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
roberta-base	NMT	100.0000	1.2532	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
roberta-base	NQT	100.0000	1.0805	1.0000	above_null	1.0000	1.0000	0.0000	0.0000

Decoder pairwise composition summary

model	pair	mean_commutator_nor	std_commutator_nor	mean_relative_commutato	std_relative_commutator_r	mean_cosine	std_cosine	mean_noncommutativit	std_noncommutativity	n
distilgpt2	MT_vs_TM	10.5809	1.8042	0.8545	0.2156	0.7111	0.1725	0.2889	0.1725	80.0000
distilgpt2	NM_vs_MN	3.7641	0.9414	0.3098	0.0893	0.9583	0.0283	0.0417	0.0283	80.0000
distilgpt2	NQ_vs_QN	5.7897	0.6043	0.4841	0.1058	0.8844	0.0472	0.1156	0.0472	80.0000
distilgpt2	NT_vs_TN	11.1703	1.8386	0.8664	0.1749	0.7046	0.1705	0.2954	0.1705	80.0000
distilgpt2	QM_vs_MQ	7.7162	0.9341	0.6474	0.1543	0.8039	0.0973	0.1961	0.0973	80.0000
distilgpt2	QT_vs_TQ	2.7566	0.6003	0.3249	0.1427	0.9571	0.0470	0.0429	0.0470	80.0000
gpt2	MT_vs_TM	11.3260	1.9708	0.3072	0.0694	0.9707	0.0080	0.0293	0.0080	80.0000
gpt2	NM_vs_MN	5.2897	1.1579	0.1894	0.0387	0.9901	0.0035	0.0099	0.0035	80.0000
gpt2	NQ_vs_QN	11.1700	1.5640	0.3855	0.0863	0.9337	0.0283	0.0663	0.0283	80.0000
gpt2	NT_vs_TN	23.9958	3.0734	0.7100	0.1186	0.9667	0.0305	0.0333	0.0305	80.0000
gpt2	QM_vs_MQ	13.3556	2.9359	0.5125	0.1313	0.8888	0.0563	0.1112	0.0563	80.0000
gpt2	QT_vs_TQ	3.9191	0.8893	0.2190	0.0846	0.9761	0.0196	0.0239	0.0196	80.0000

Semantic equivalence control

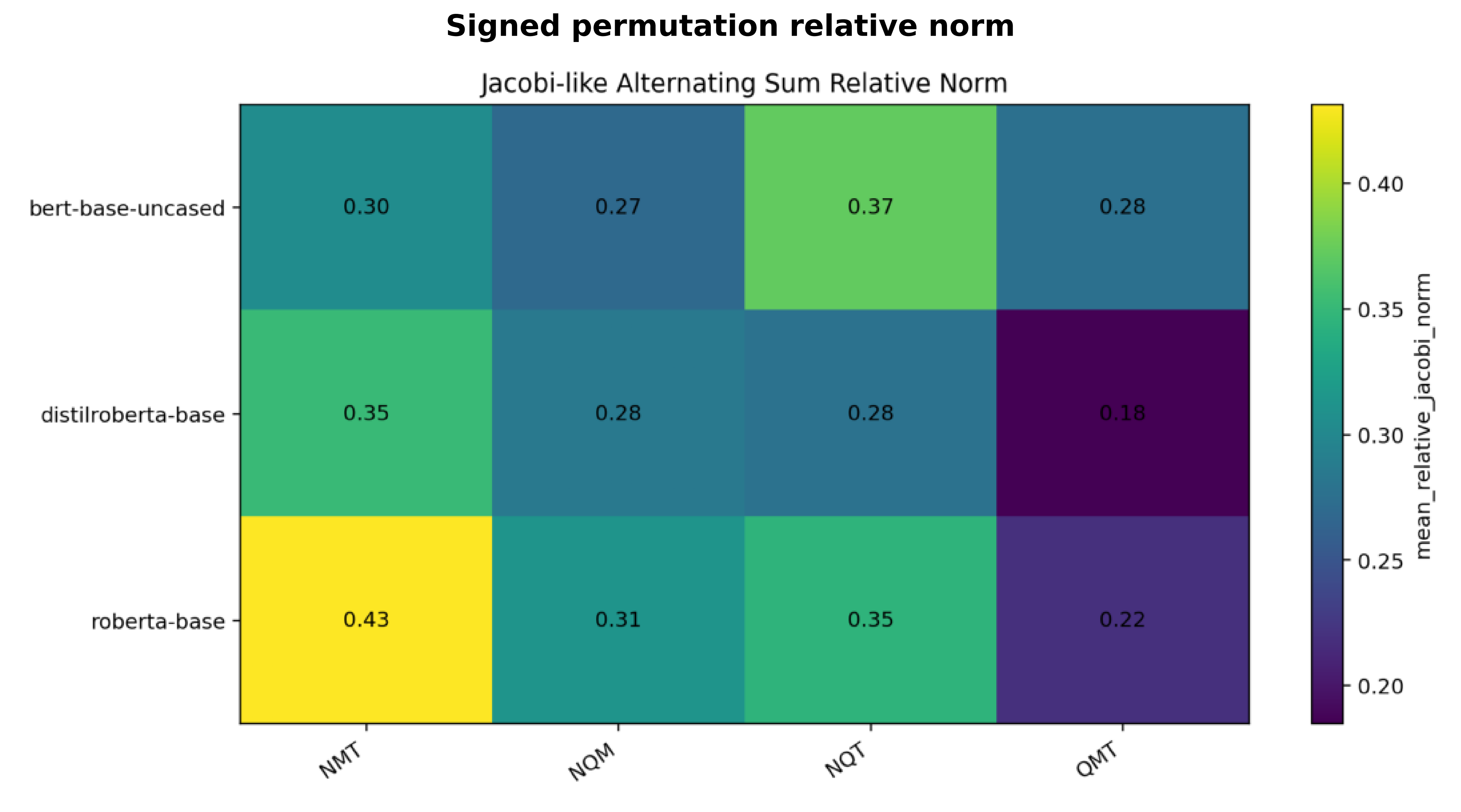
model	label	mean_noncommutativity	std_noncommutativity	mean_relative_commutator_st	std_relative_commutator_nor	mean_cosine	std_cosine	n
bert-base-uncased	equivalent	0.1130	0.0409	0.4818	0.0809	0.8870	0.0409	400.0000
bert-base-uncased	non_equivalent	0.3291	0.1134	0.8065	0.1326	0.6709	0.1134	400.0000
distilroberta-base	equivalent	0.1295	0.0483	0.5406	0.1277	0.8705	0.0483	400.0000
distilroberta-base	non_equivalent	0.2319	0.0381	0.6868	0.0560	0.7681	0.0381	400.0000
roberta-base	equivalent	0.1543	0.0630	0.5831	0.1304	0.8457	0.0630	400.0000
roberta-base	non_equivalent	0.2784	0.0439	0.7465	0.0597	0.7216	0.0439	400.0000

Semantic equivalence effect sizes

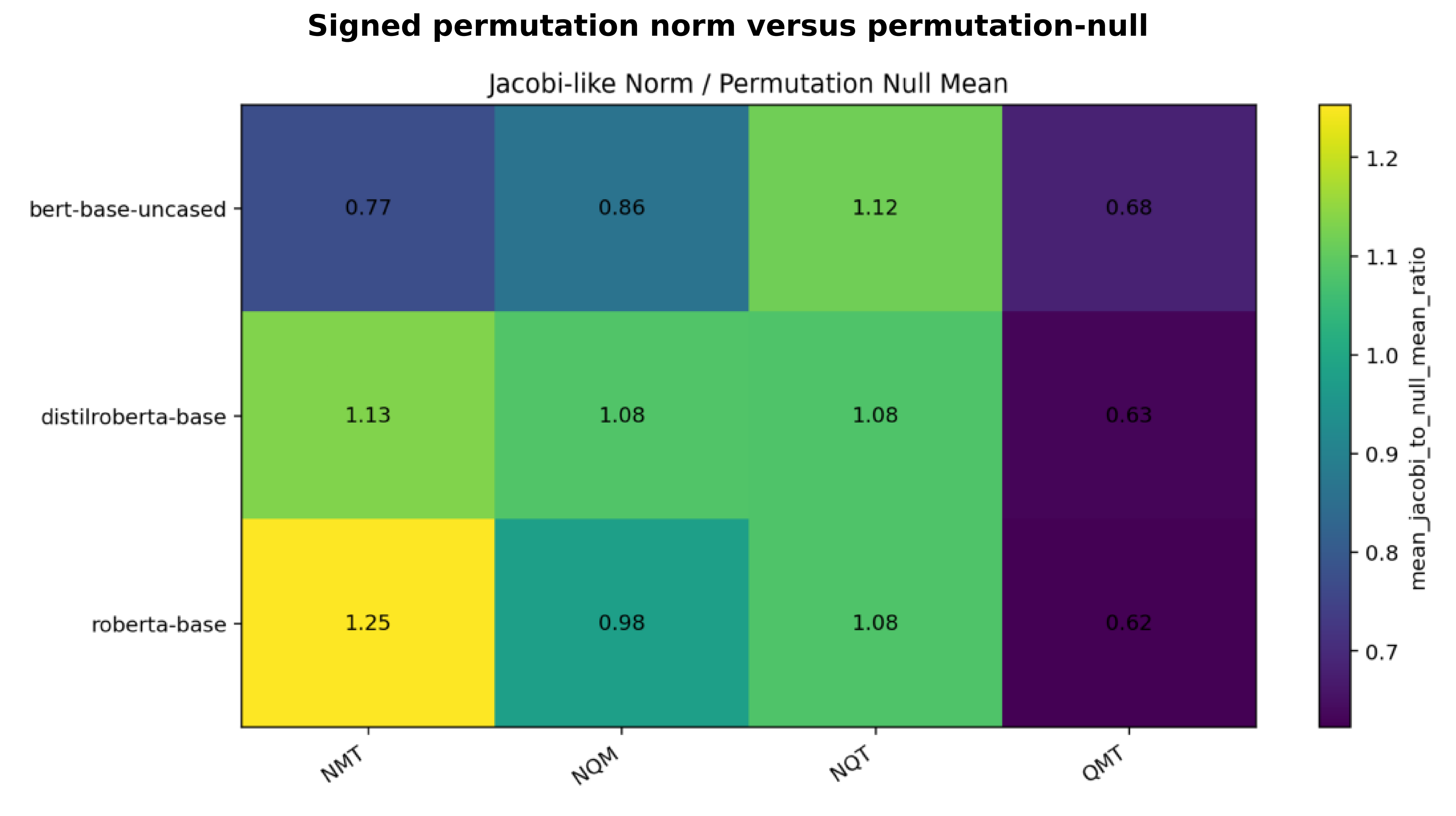
model	equiv_mean	non_equiv_mean	diff	cohens_d	equiv_std	non_equiv_std
bert-base-uncased	0.1130	0.3291	0.2161	2.5344	0.0409	0.1134
distilroberta-base	0.1295	0.2319	0.1024	2.3536	0.0483	0.0381
roberta-base	0.1543	0.2784	0.1241	2.2854	0.0630	0.0439

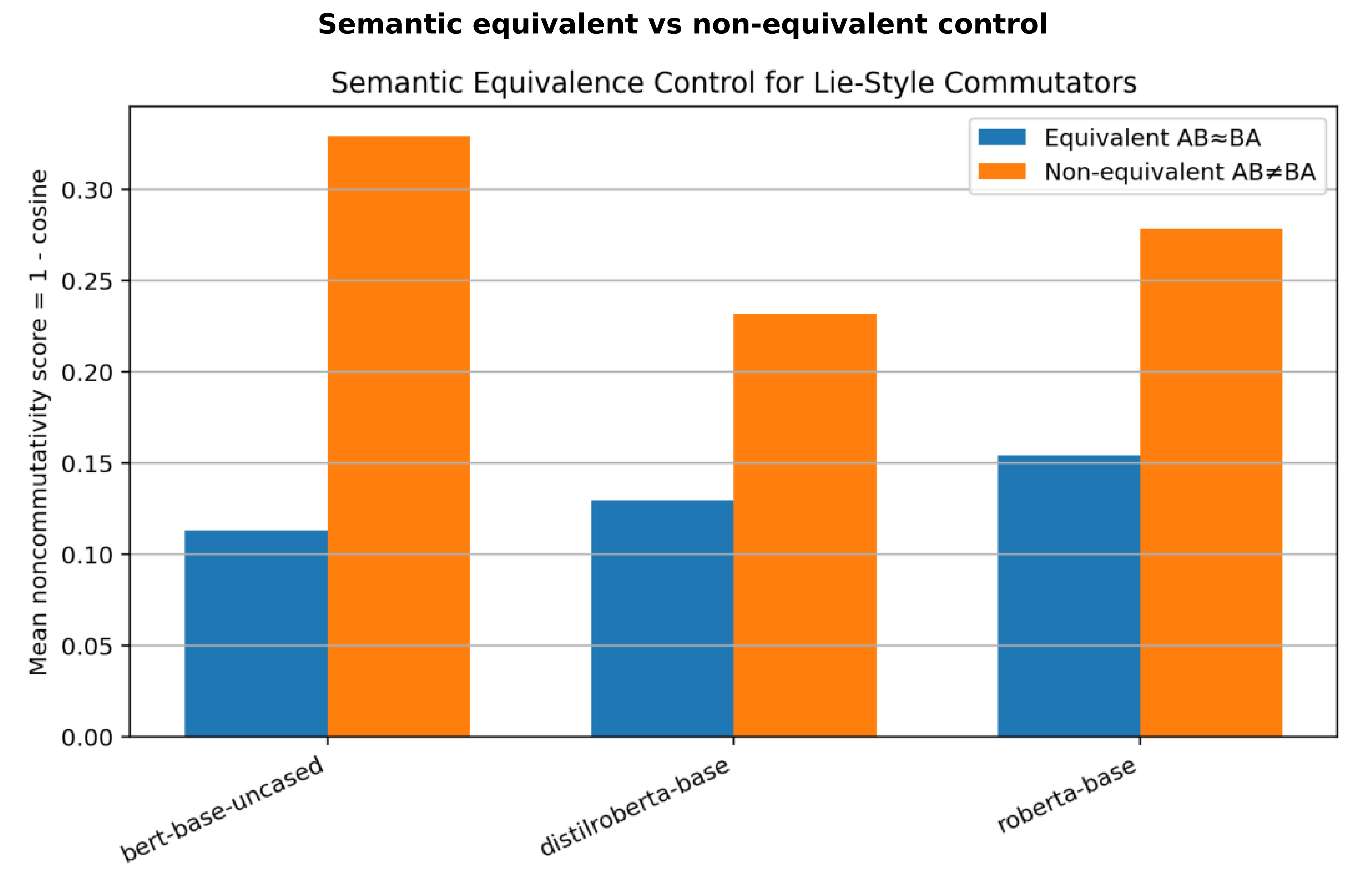
Composition summary: strongest noncommutativity rows

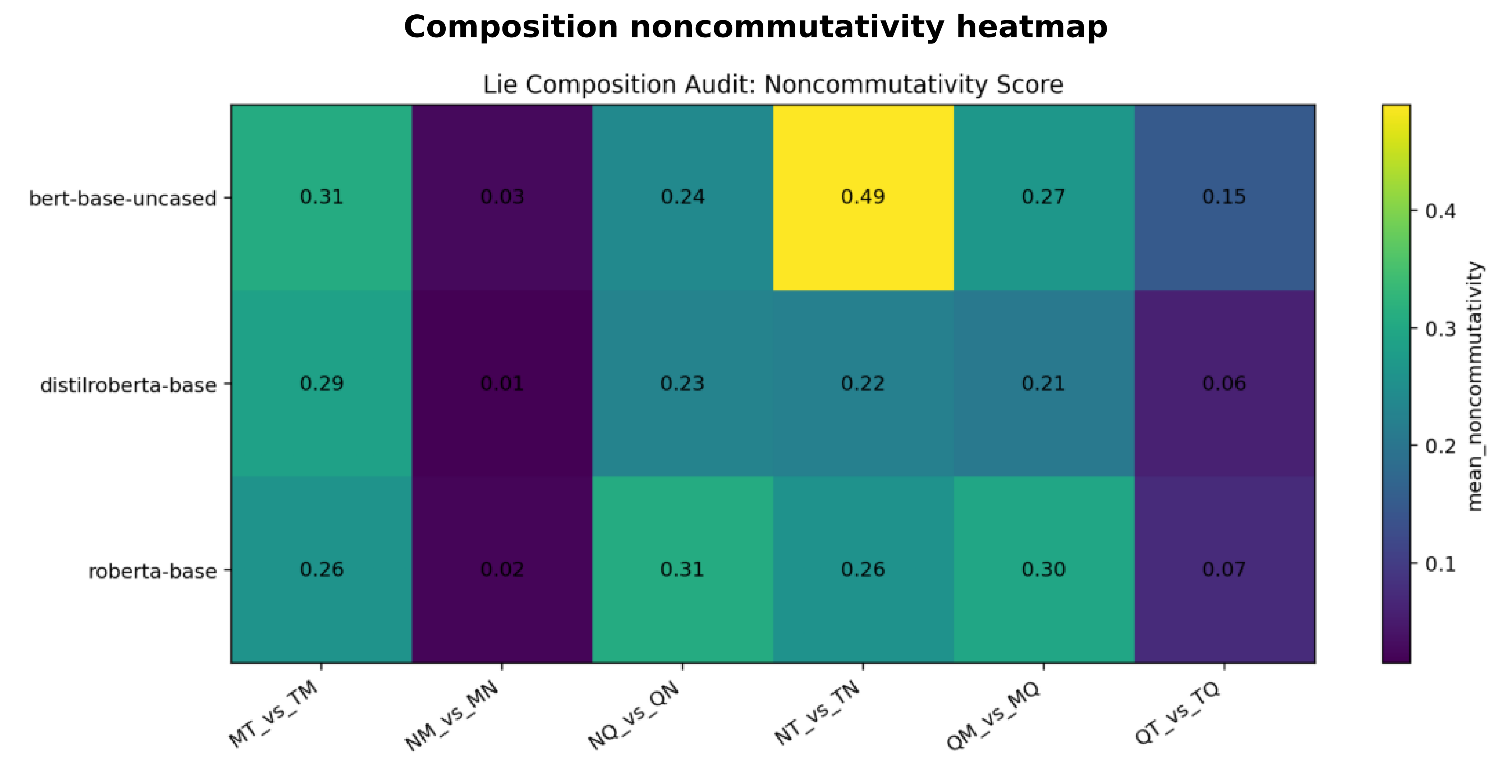
model	pair	mean_commutator_nor	std_commutator_	mean_relative_commutato	std_relative_commutator_r	mean_cosine	std_cosine	mean_noncommutativit	std_noncommutativity	n
bert-base-uncased	NT_vs_TN	5.1565	0.5832	0.9908	0.1066	0.5102	0.1009	0.4898	0.1009	80.0000
roberta-base	NQ_vs_QN	1.6422	0.0726	0.7846	0.0364	0.6928	0.0279	0.3072	0.0279	80.0000
bert-base-uncased	MT_vs_TM	4.2726	0.3075	0.7817	0.0954	0.6938	0.0727	0.3062	0.0727	80.0000
roberta-base	QM_vs_MQ	1.5649	0.0863	0.7714	0.0541	0.7032	0.0410	0.2968	0.0410	80.0000
distilroberta-base	MT_vs_TM	1.3504	0.0606	0.7683	0.0387	0.7124	0.0302	0.2876	0.0302	80.0000
bert-base-uncased	QM_vs_MQ	4.4793	0.2156	0.7341	0.0487	0.7340	0.0347	0.2660	0.0347	80.0000
roberta-base	MT_vs_TM	1.3956	0.0770	0.7215	0.0560	0.7411	0.0394	0.2589	0.0394	80.0000
roberta-base	NT_vs_TN	1.4185	0.1071	0.7172	0.0599	0.7429	0.0428	0.2571	0.0428	80.0000
bert-base-uncased	NQ_vs_QN	4.0903	0.2939	0.6970	0.0493	0.7598	0.0347	0.2402	0.0347	80.0000
distilroberta-base	NQ_vs_QN	1.4541	0.0644	0.6731	0.0384	0.7735	0.0257	0.2265	0.0257	80.0000
distilroberta-base	NT_vs_TN	1.3975	0.0725	0.6675	0.0342	0.7783	0.0232	0.2217	0.0232	80.0000
distilroberta-base	QM_vs_MQ	1.3672	0.0842	0.6613	0.0460	0.7926	0.0287	0.2074	0.0287	80.0000
bert-base-uncased	QT_vs_TQ	3.1001	0.1729	0.5463	0.0440	0.8513	0.0231	0.1487	0.0231	80.0000
roberta-base	QT_vs_TQ	0.7784	0.0701	0.3860	0.0342	0.9269	0.0128	0.0731	0.0128	80.0000
distilroberta-base	QT_vs_TQ	0.6580	0.0281	0.3422	0.0173	0.9415	0.0059	0.0585	0.0059	80.0000
bert-base-uncased	NM_vs_MN	1.2527	0.2368	0.2305	0.0538	0.9728	0.0125	0.0272	0.0125	80.0000
roberta-base	NM_vs_MN	0.3865	0.0403	0.2032	0.0248	0.9794	0.0050	0.0206	0.0050	80.0000
distilroberta-base	NM_vs_MN	0.3128	0.0212	0.1721	0.0201	0.9852	0.0036	0.0148	0.0036	80.0000



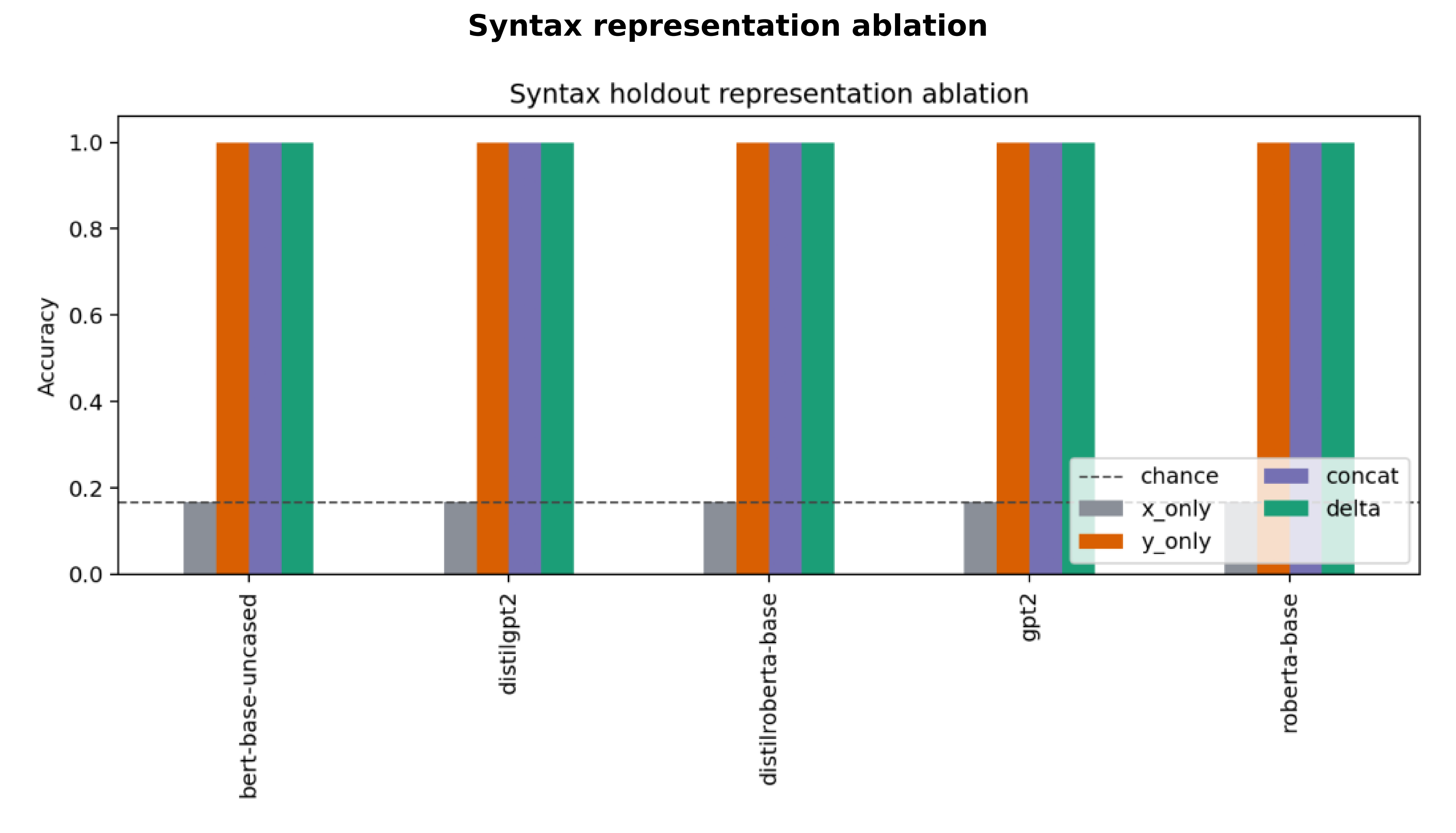
Signed permutation relative norm
Jacobi-like Alternating Sum Relative Norm
bert-base-uncased
distilroberta-base
roberta-base
NMT
NQM
NQT
QMT
0.30
0.27
0.37
0.28
0.35
0.28
0.28
0.18
0.43
0.31
0.35
0.22
mean_relative_jacobi_norm
0.20
0.25
0.30
0.35
0.40


Signed permutation norm versus permutation-null
Jacobi-like Norm / Permutation Null Mean
bert-base-uncased
distilroberta-base
roberta-base
NMT
NQM
NQT
QMT
0.77
0.86
1.12
0.68
1.13
1.08
1.08
0.63
1.25
0.98
1.08
0.62
mean_jacobi_to_null_mean_ratio
0.7
0.8
0.9
1.0
1.1
1.2

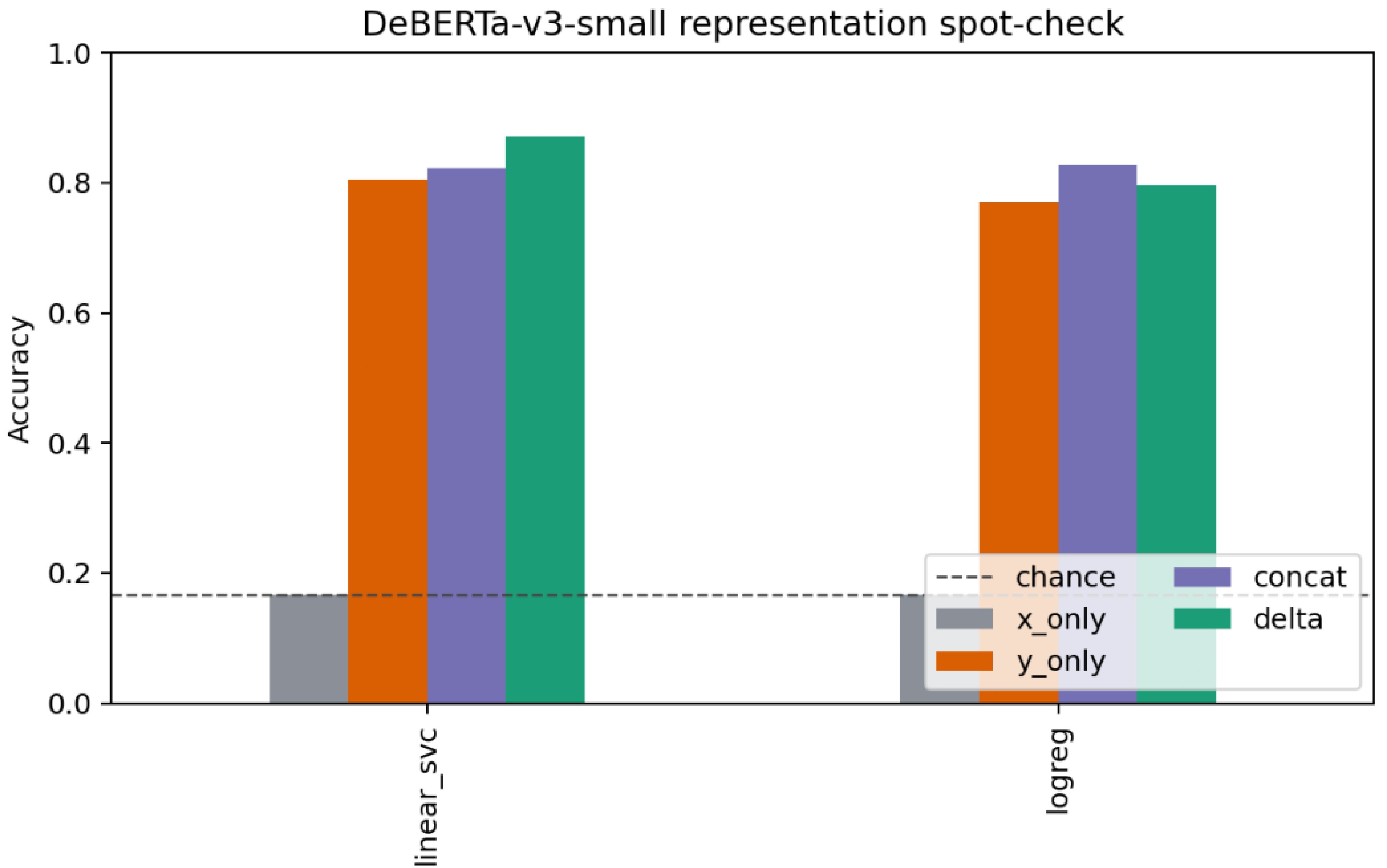

Semantic equivalent vs non-equivalent control
Semantic Equivalence Control for Lie-Style Commutators
Equivalent AB≈BA
Non-equivalent AB≠BA
Mean noncommutativity score = 1 - cosine
0.00
0.05
0.10
0.15
0.20
0.25
0.30
bert-base-uncased
distilroberta-base
roberta-base


Composition noncommutativity heatmap
Lie Composition Audit: Noncommutativity Score
bert-base-uncased
distilroberta-base
roberta-base
0.31
0.03
0.24
0.49
0.27
0.15
0.29
0.01
0.23
0.22
0.21
0.06
0.26
0.02
0.31
0.26
0.30
0.07
MT_vs_TM
NM_vs_MN
NQ_vs_QN
NT_vs_TN
QM_vs_MQ
QT_vs_TQ
mean_noncommutativity
0.1
0.2
0.3
0.4

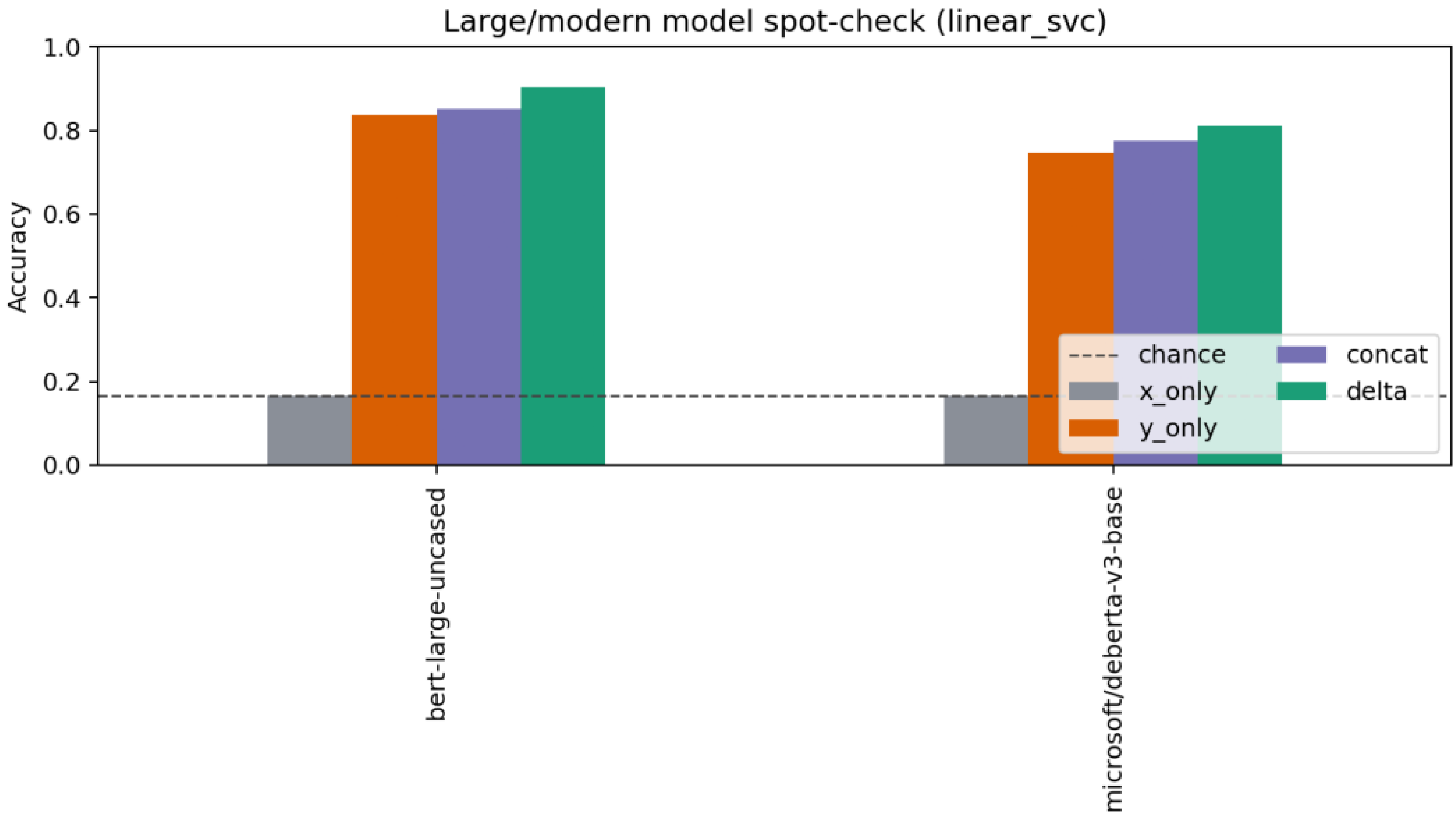
Original PCA class geometry


Syntax representation ablation
Syntax holdout representation ablation
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
chance
x_only
y_only
concat
delta
bert-base-uncased
distilgpt2
distilroberta-base
gpt2
roberta-base

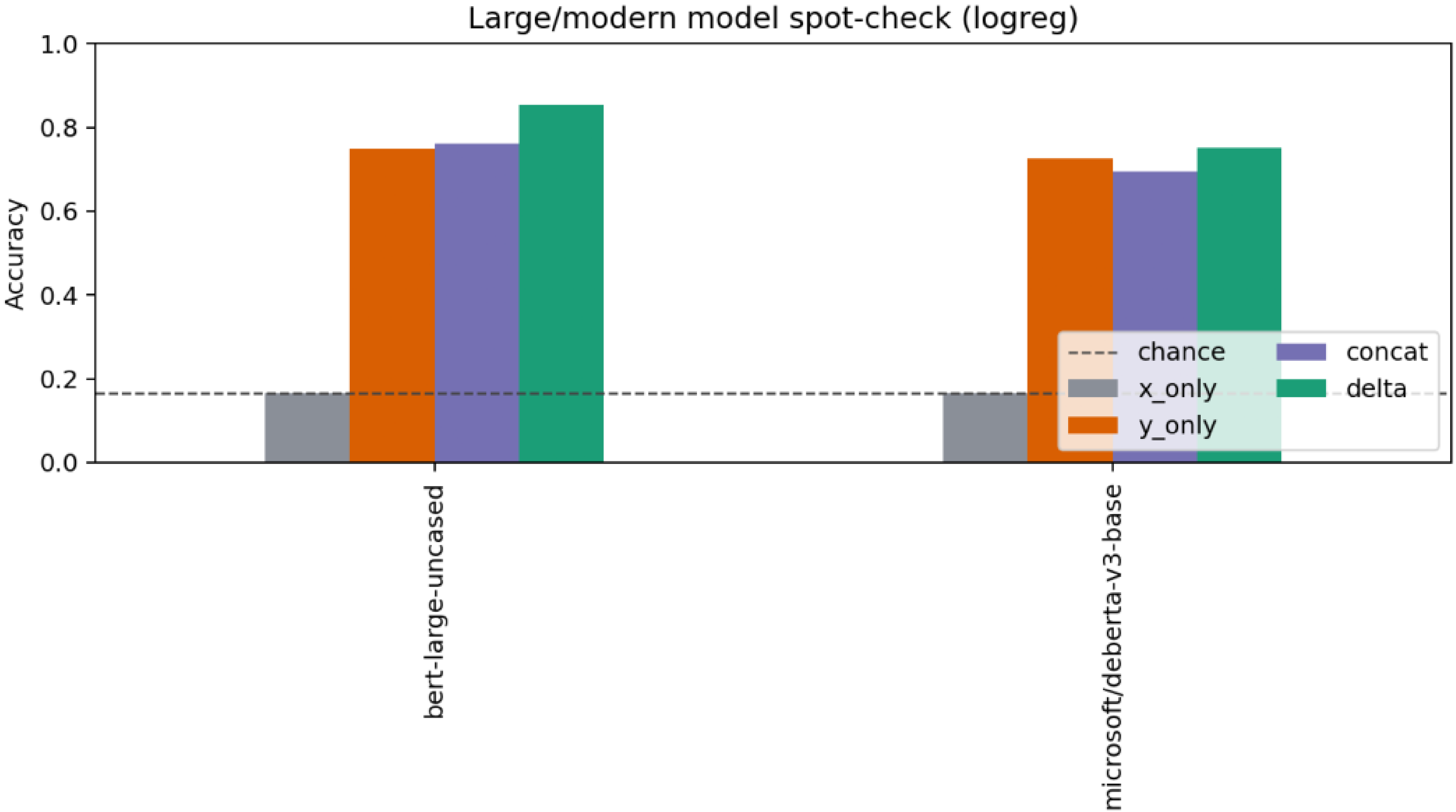
DeBERTa-v3-small spot-check

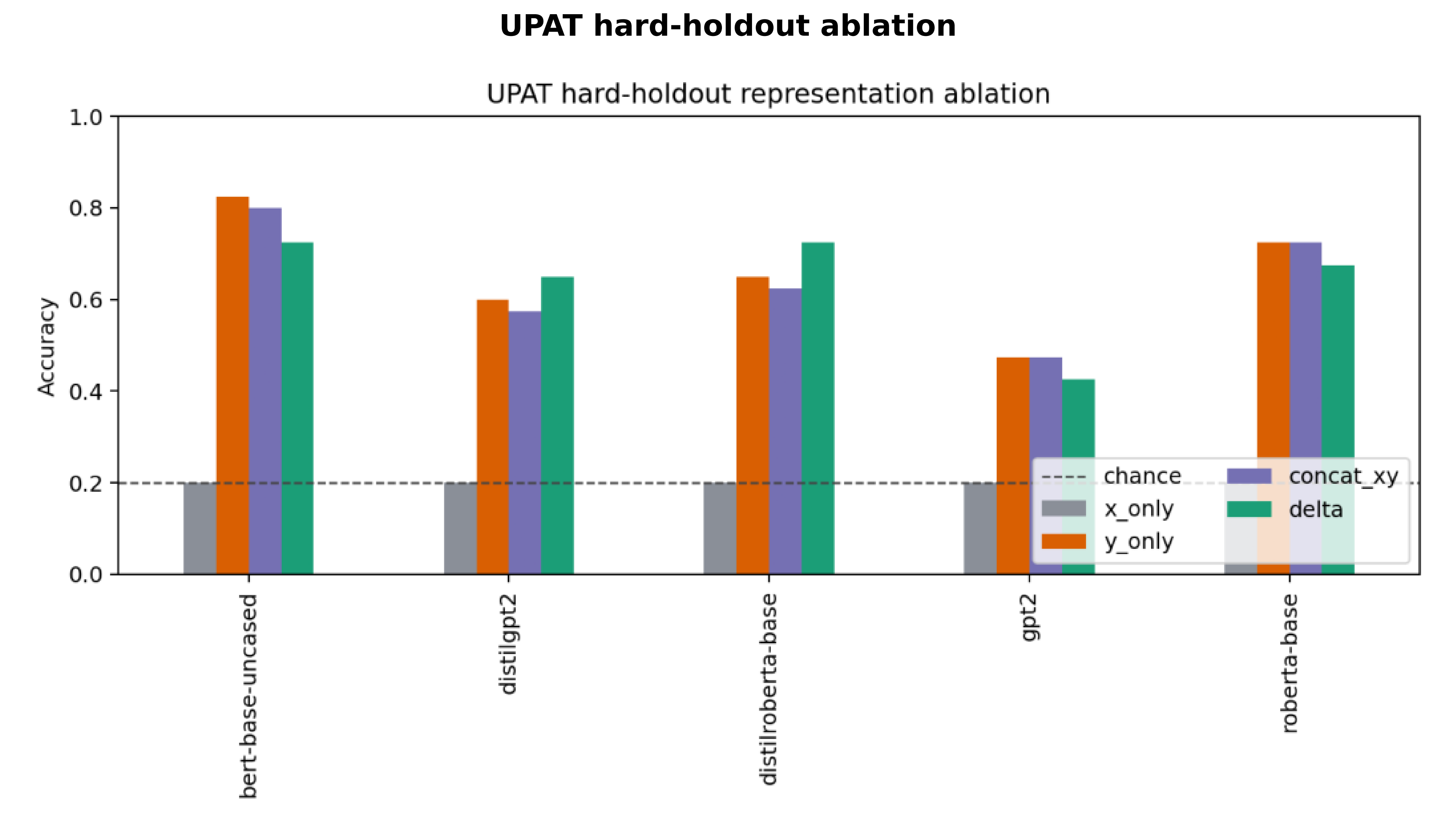


Large/modern spot-check: Linear SVC

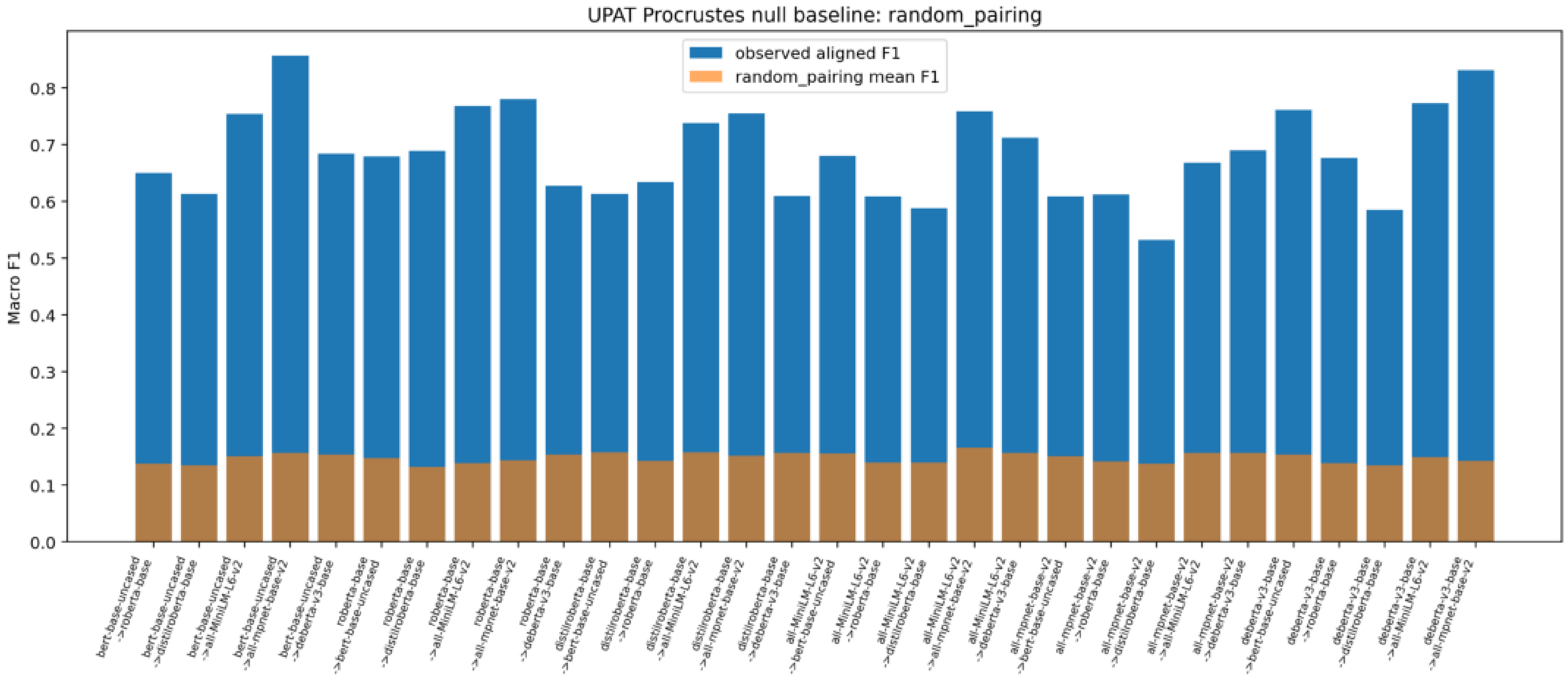


Large/modern spot-check: logistic regression

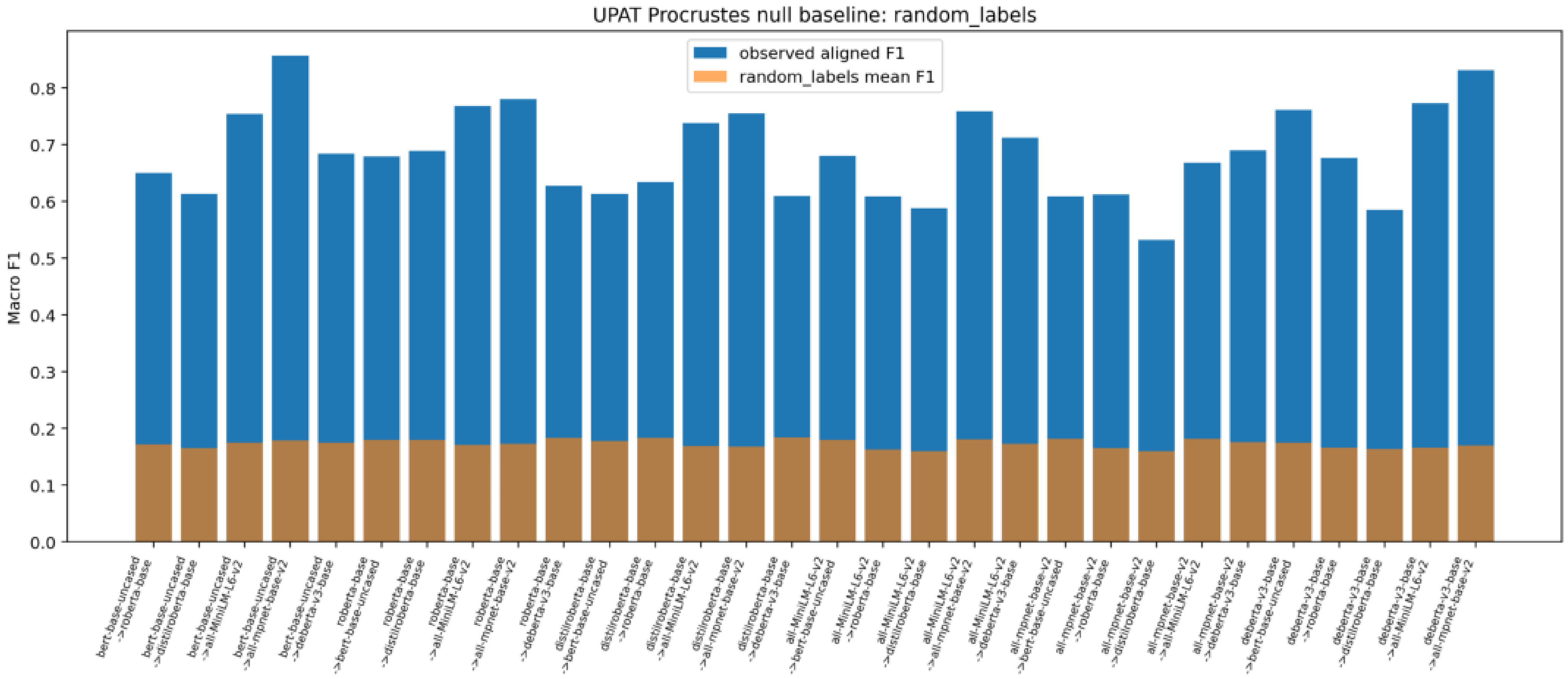



UPAT hard-holdout ablation
UPAT hard-holdout representation ablation
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
chance
x_only
y_only
concat_xy
delta
bert-base-uncased
distilgpt2
distilroberta-base
gpt2
roberta-base

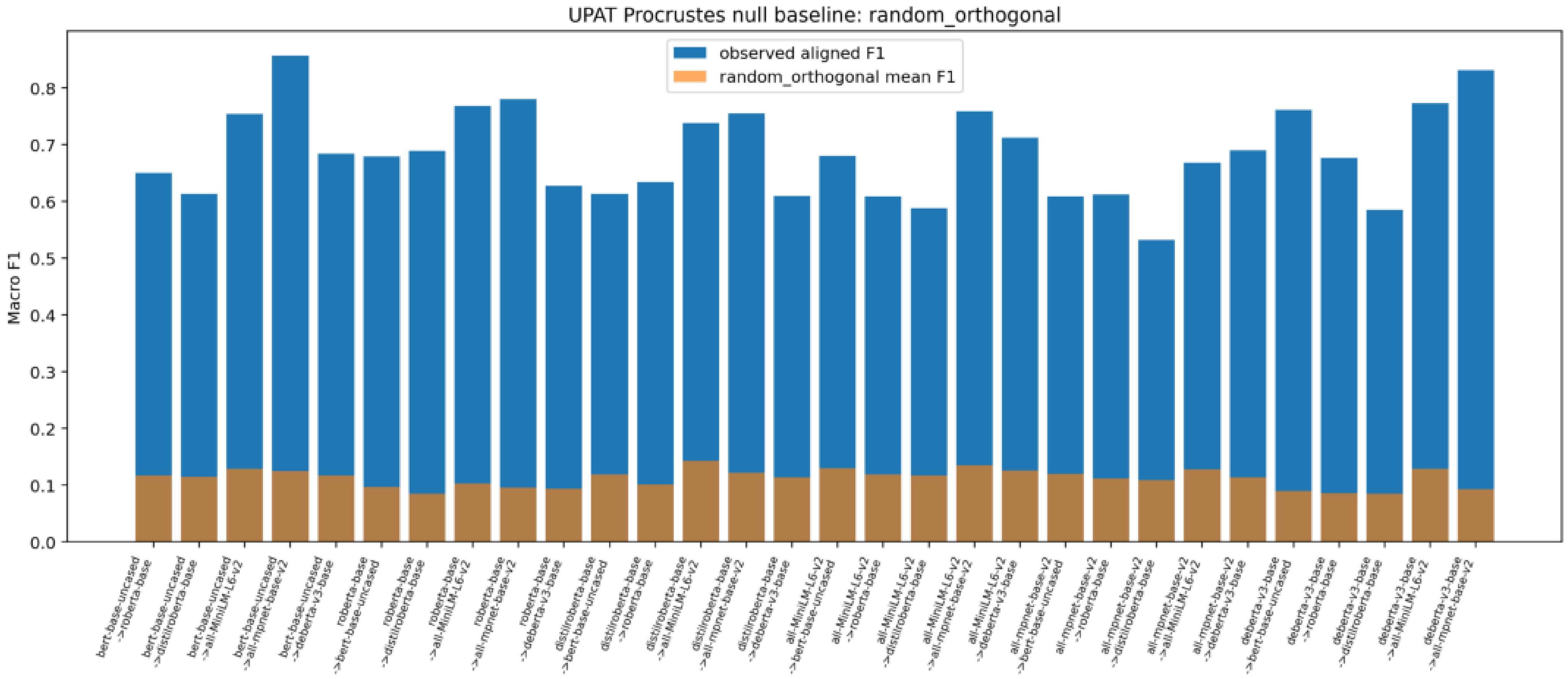
UPAT-large Procrustes random-pairing null



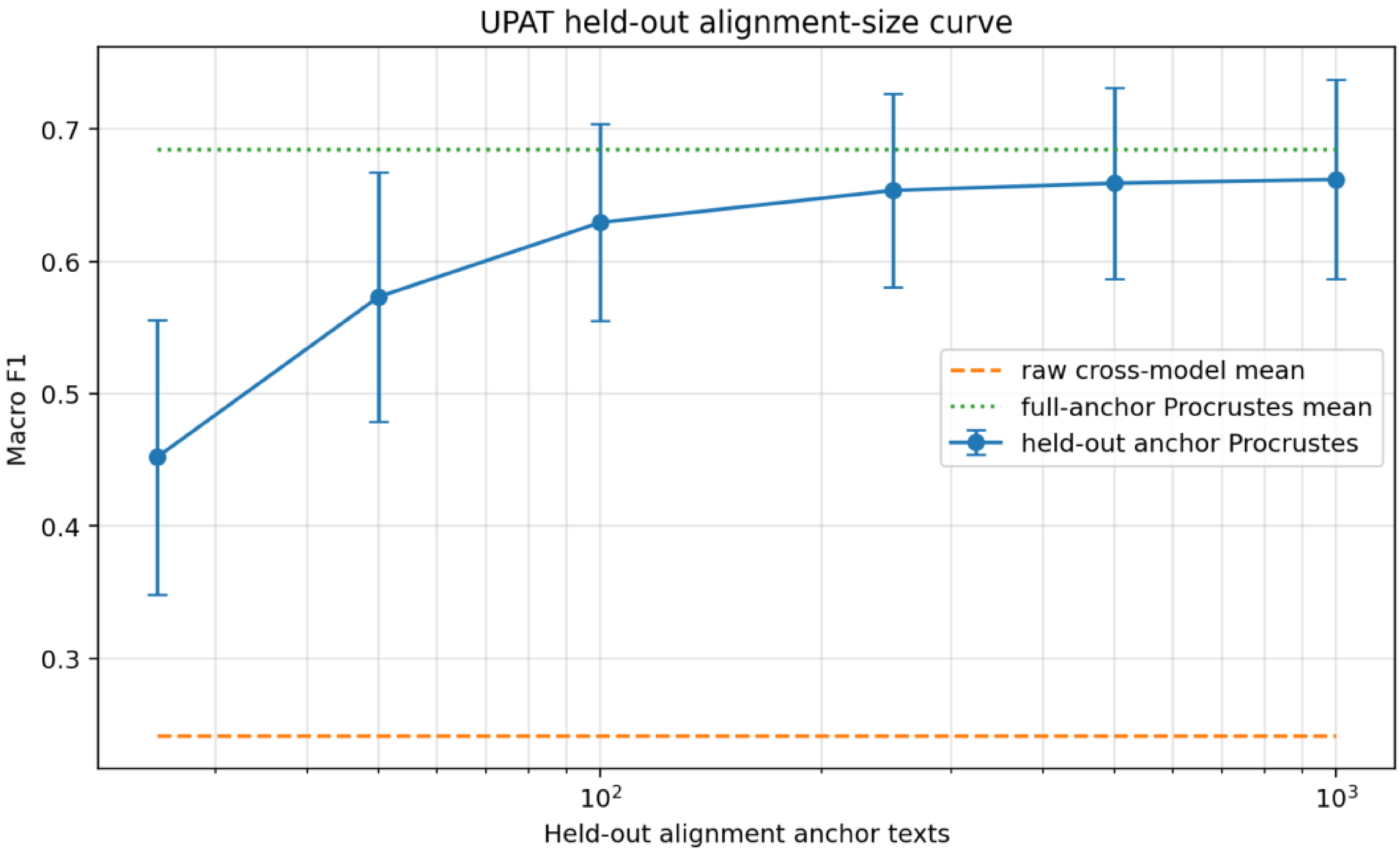
UPAT-large Procrustes random-label null



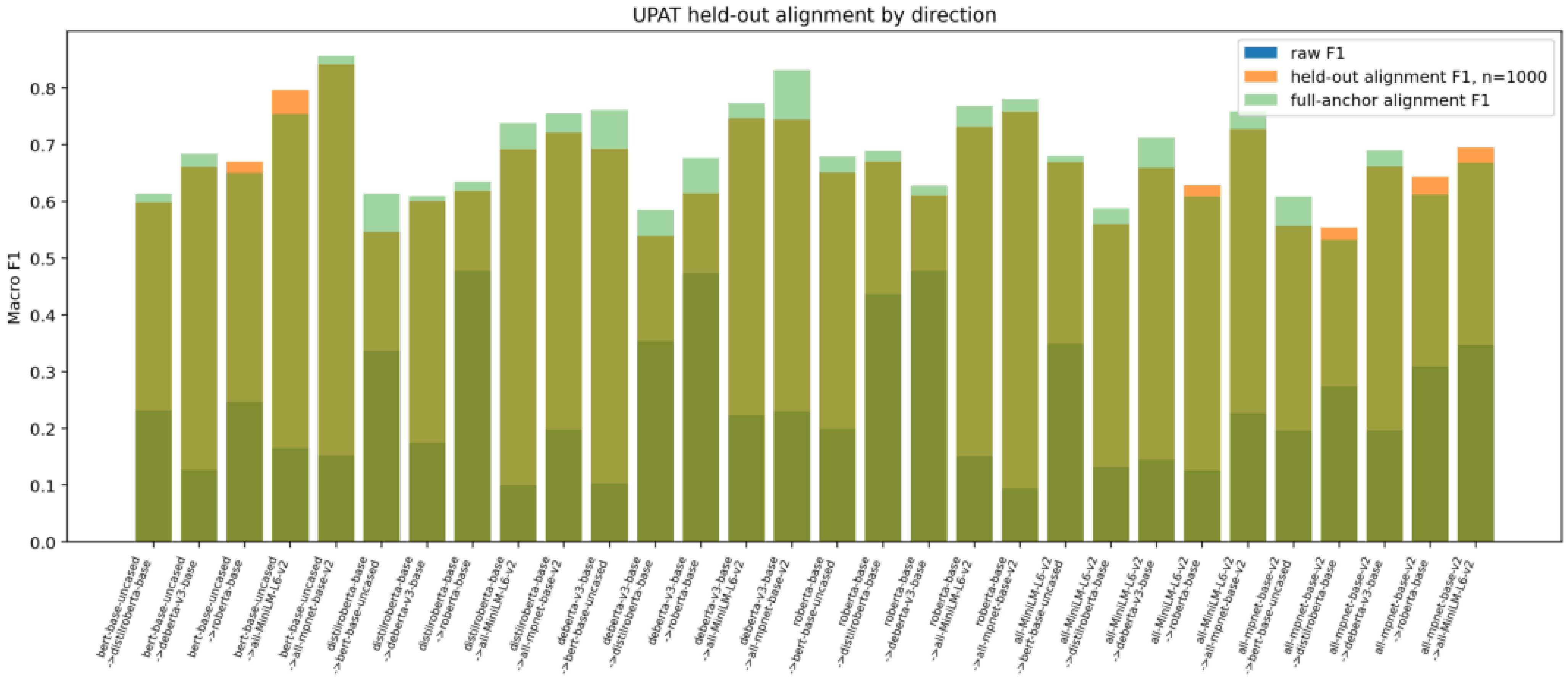
UPAT-large Procrustes random-orthogonal null

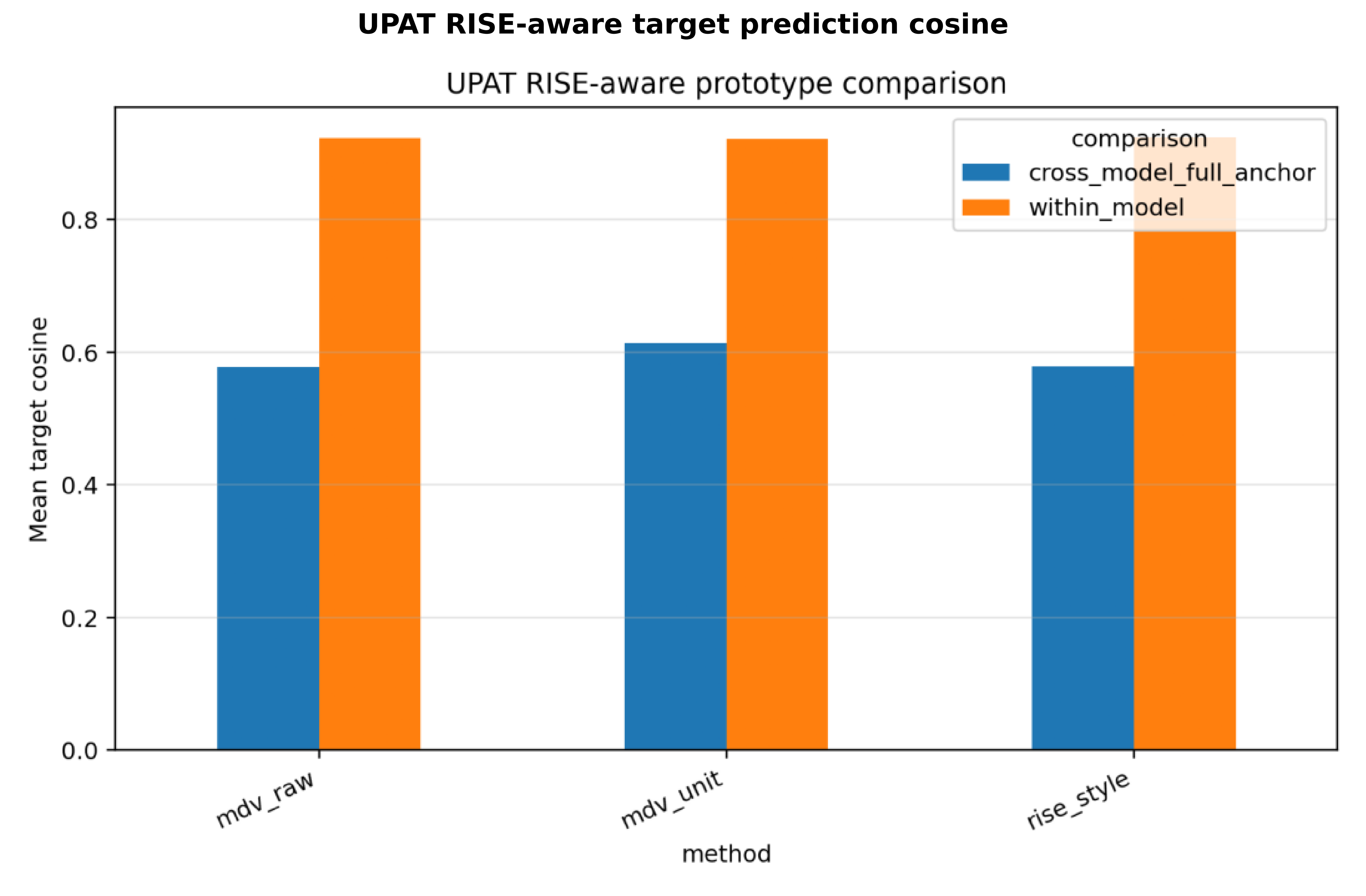


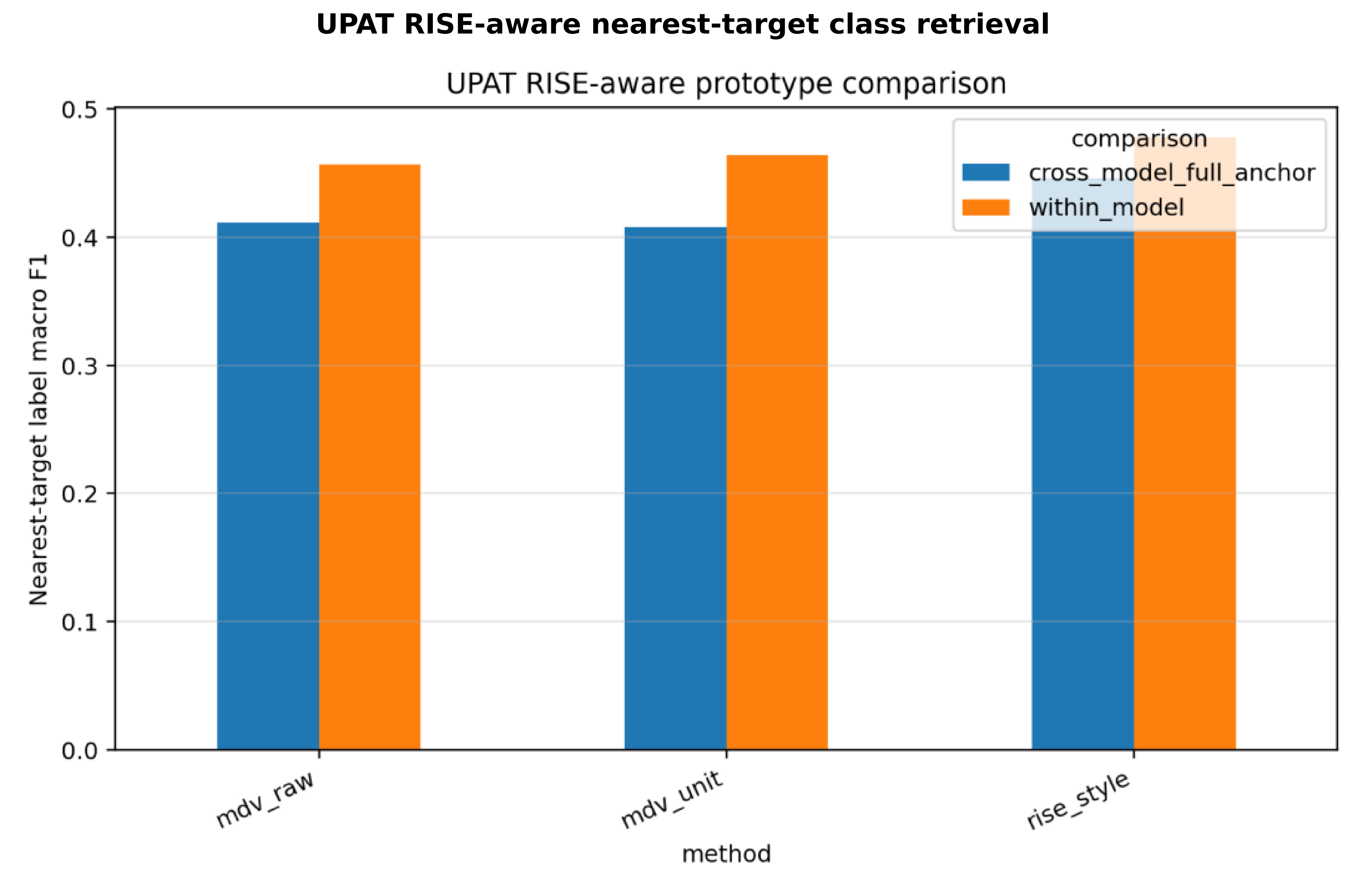
UPAT-large held-out alignment-size curve



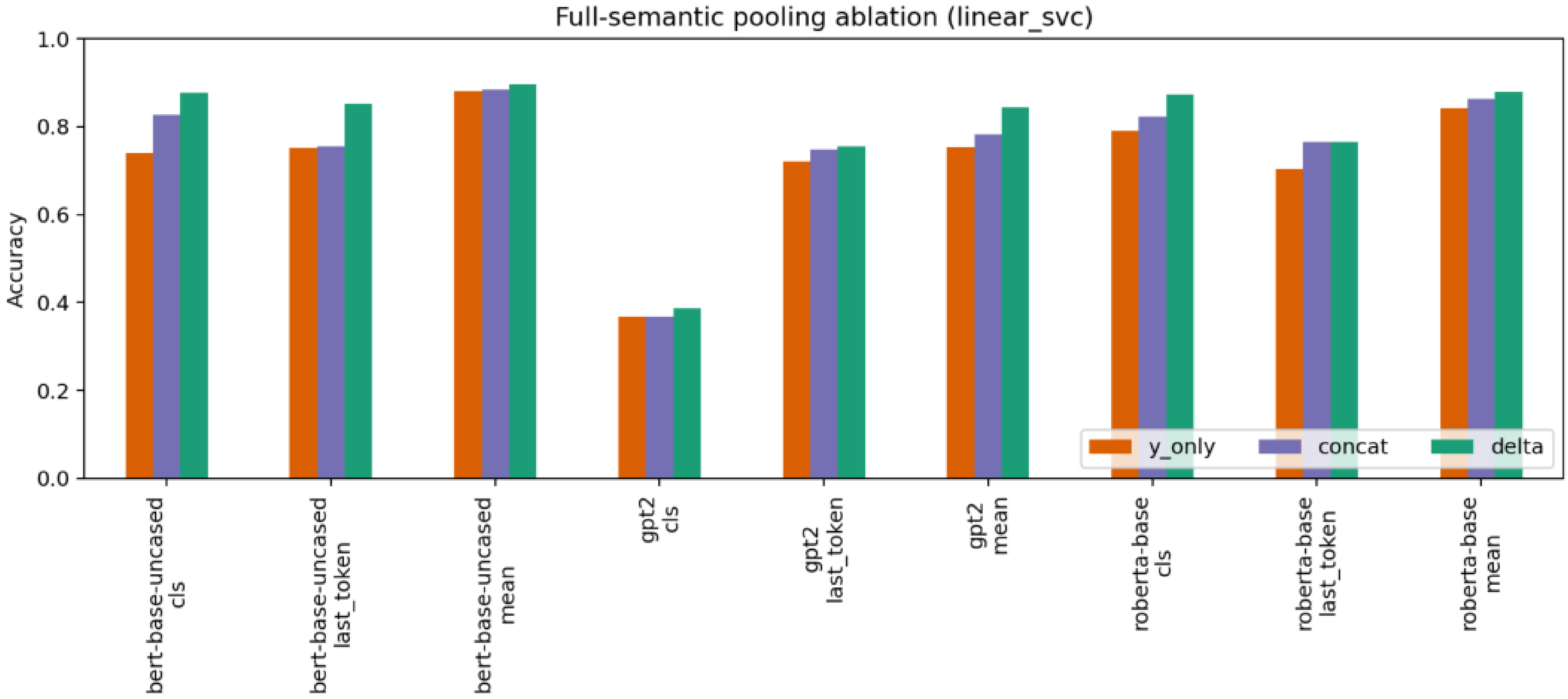
UPAT-large held-out alignment by direction



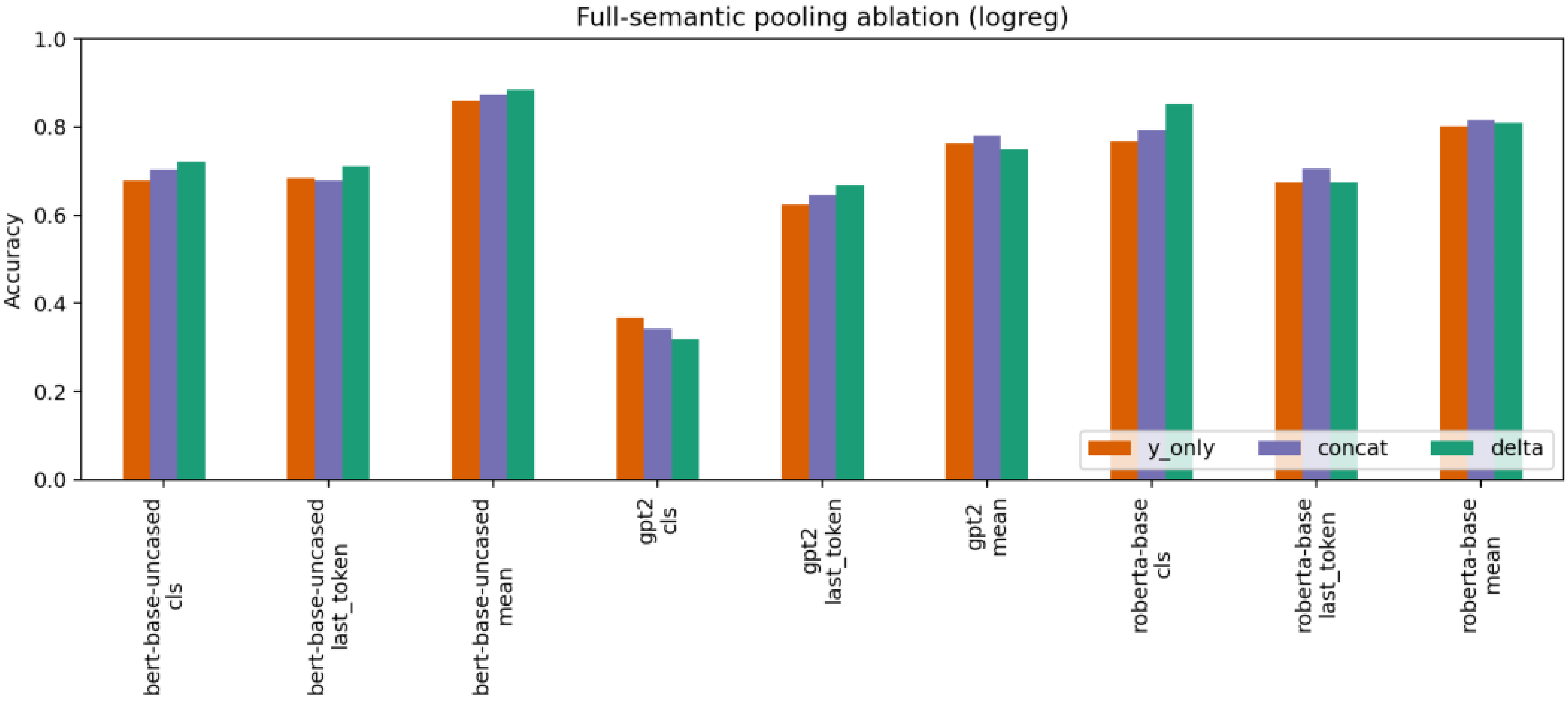

UPAT RISE-aware target prediction cosine
UPAT RISE-aware prototype comparison
comparison
cross_model_full_anchor
within_model
Mean target cosine
0.0
0.2
0.4
0.6
0.8
mdv_raw
mdv_unit
rise_style
method

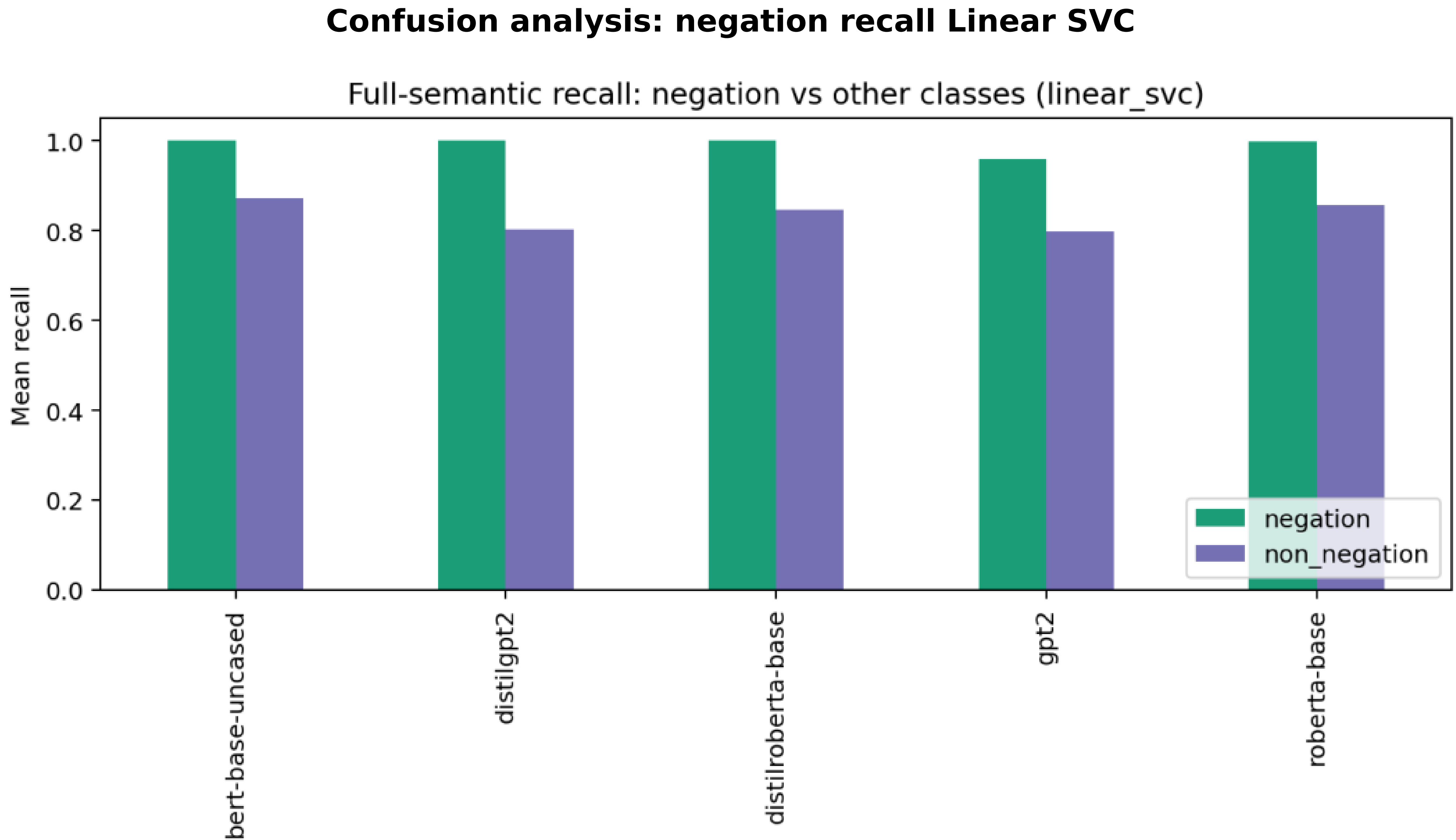

UPAT RISE-aware nearest-target class retrieval
UPAT RISE-aware prototype comparison
Nearest-target label macro F1
0.0
0.1
0.2
0.3
0.4
0.5
comparison
cross_model_full_anchor
within_model
mdv_raw
mdv_unit
rise_style
method

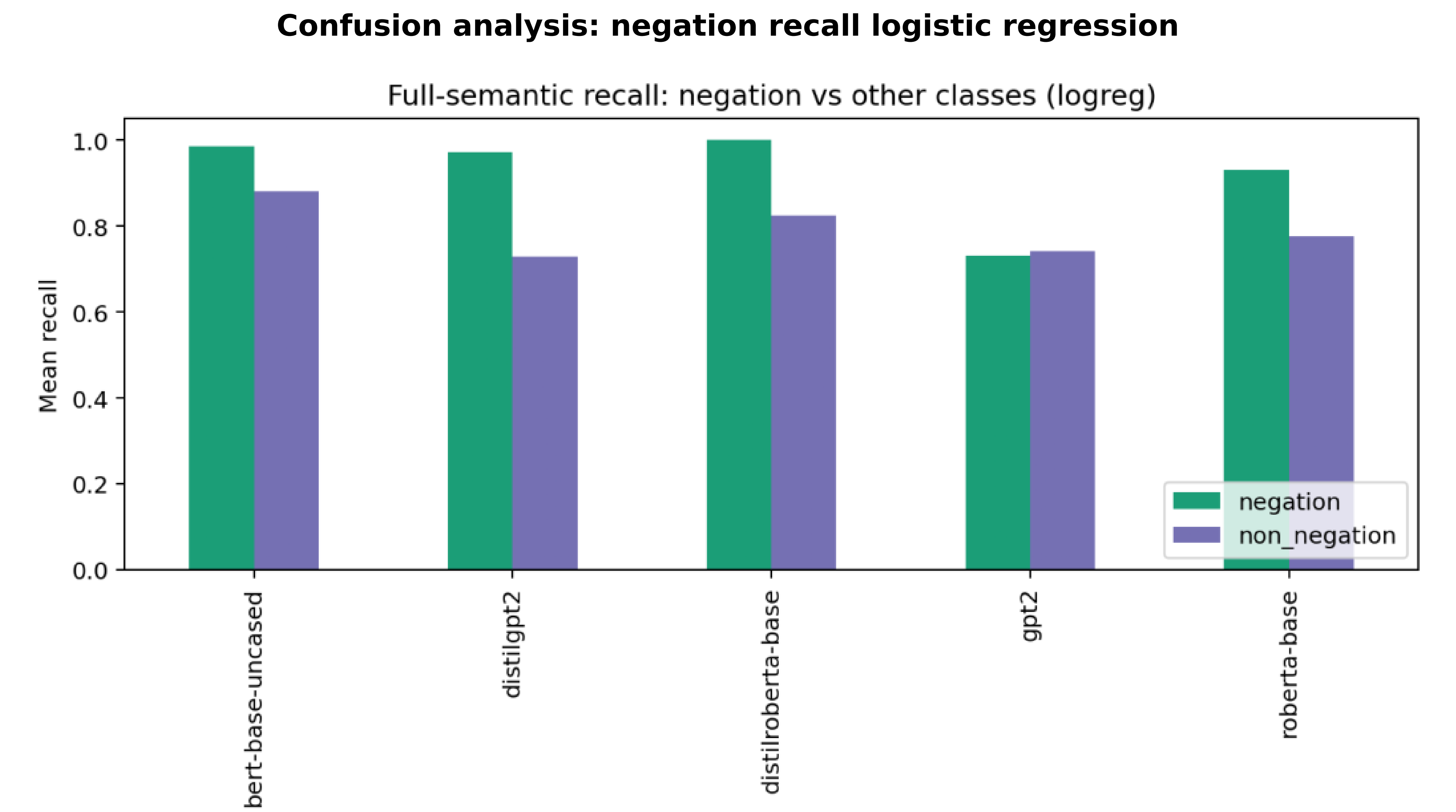
Full-semantic pooling ablation: Linear SVC



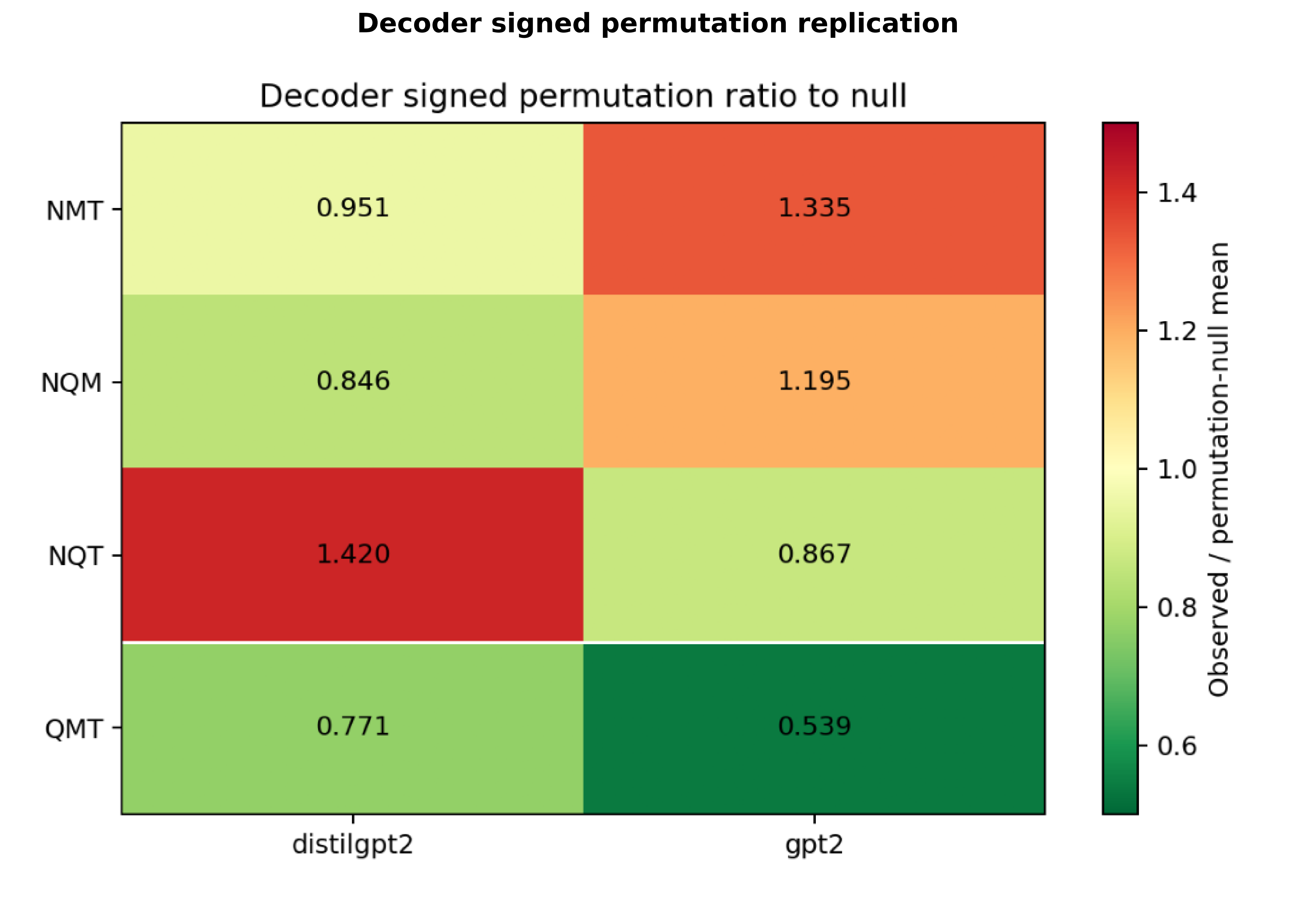
Full-semantic pooling ablation: logistic regression



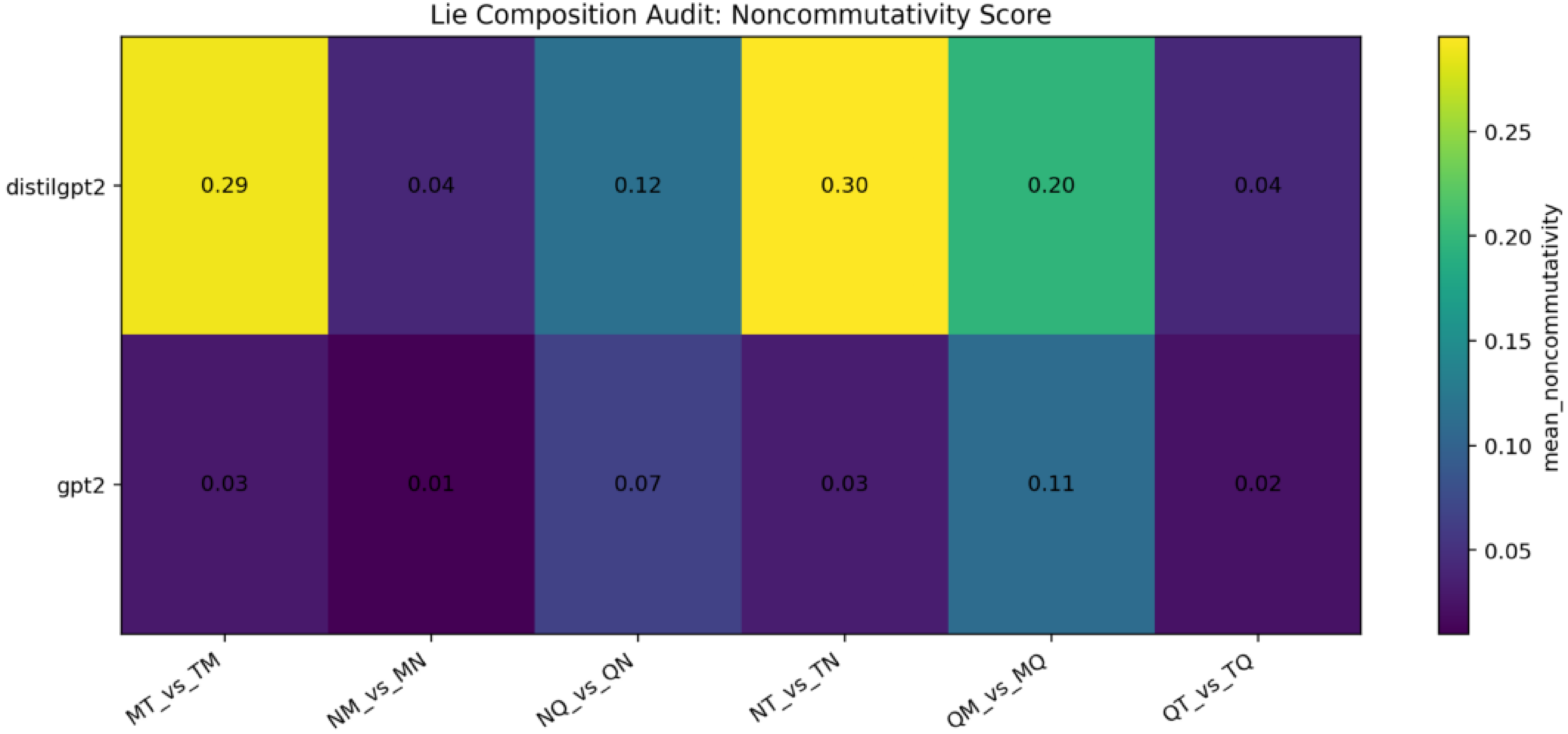

Confusion analysis: negation recall Linear SVC
Full-semantic recall: negation vs other classes (linear_svc)
Mean recall
1.0
0.8
0.6
0.4
0.2
0.0
bert-base-uncased
distilgpt2
distilroberta-base
gpt2
roberta-base
negation
non_negation



Confusion analysis: negation recall logistic regression
Full-semantic recall: negation vs other classes (logreg)
Mean recall
1.0
0.8
0.6
0.4
0.2
0.0
negation
non_negation
bert-base-uncased
distilgpt2
distilroberta-base
gpt2
roberta-base


Decoder signed permutation replication
Decoder signed permutation ratio to null
NMT
NQM
NQT
QMT
0.951
1.335
0.846
1.195
1.420
0.867
0.771
0.539
distilgpt2
gpt2
1.4
1.2
1.0
0.8
0.6
Observed / permutation-null mean

Decoder pairwise composition



Audit notes for external verifier

Antisymmetry is not reported as evidence because the implementation defines [B,A] as the negative order of [A,B]. It is a tautological implementation check only.

A literal nested-commutator Jacobi expression over the same six endpoint vectors cancels algebraically. The reported third-order diagnostic is the non-tautological signed permutation composition sum ABC+BCA+CAB-ACB-CBA-BAC.

Duplicate endpoint templates were detected and removed before finalizing today's result. The final dataset has 400 Jacobi rows and zero duplicate endpoint sets.

New Track 1 result: syntax y_only=1.0 confirms target/surface leakage for the syntax split. New Track 1 spot-check: DeBERTa-v3-small supports delta superiority for Linear SVC but not for logistic regression. New Track 2 result: GPT-2 and DistilGPT-2 both keep QMT below permutation null, while negation triples remain mixed.

2026-06-13 update: after cache cleanup, BERT-large and DeBERTa-v3-base were both run successfully. Delta is best for both Linear SVC and logistic regression on both models.

Second 2026-06-13 update: UPAT is now explicitly reported as a hard-holdout boundary condition. Full-semantic pooling ablation, confusion/negation analysis, decoder pairwise composition, and signed-permutation multiple-testing correction are included.

2026-06-14 update: UPAT-large Procrustes null controls are scaled to N=1000 and now include random-label, random-pairing, and random-orthogonal baselines. The observed aligned F1 exceeds every null repeat across all non-identity directions.

Second 2026-06-14 update: held-out alignment-size controls are now added. Procrustes maps fitted on auxiliary anchor texts disjoint from classifier train/test texts recover most of the full-anchor transfer effect.

Third 2026-06-14 update: RISE is now the central related-work anchor. Track 3 is reframed as RISE-aware stress testing rather than first-discovery cross-model geometry. A RISE/MDV-style UPAT comparison is now a required gate.

Fourth 2026-06-14 update: first-pass RISE-aware UPAT comparison is now added. MDV and simplified RISE-style prototype methods predict targets well within-model, but cross-model target prediction remains harder than aligned delta-classifier transfer on the class-discrimination metric.

Remaining blockers: confidence intervals and anchor-domain diversity for Track 3, stronger/faithful RISE comparison if feasible, resolve or expand UPAT hard-holdout, representation-ablation tables for every non-syntax holdout split, grammar-generated templates, endpoint-only controls for Track 2, composition layer/pooling ablations, and final bibliography formatting.

Round-3 Reviewer Response

Reviewer Response Round 3

Date: 2026-06-13

This note records the current response to the latest review of the research package.

Fixed in the Current Drafts

Antisymmetry

The Track 2 draft no longer presents antisymmetry as a discovery. It is now described as a tautological implementation check:

```text
[A,B] = delta_AB - delta_BA
[B,A] = delta_BA - delta_AB
```

Therefore `[A,B] = -[B,A]` follows for arbitrary vectors and does not provide evidence about transformer representations.

UPAT and y_only Confounding

The Track 1 draft now explicitly reports UPAT as a hard-holdout boundary result. UPAT sometimes favors `y_only` over `delta`, and no UPAT `delta` vs `y_only` McNemar comparison is significant. The claim is narrowed:

> Delta can add useful relational information, but endpoint-only features remain a serious confounder and can dominate under smaller, harder holdouts.

Layer-0 Syntax Result

The layer-0 syntax result is now framed as a red flag. Perfect separability at the embedding layer indicates token/form cues rather than deep semantic geometry.

Synthetic Lie Templates

The Track 2 draft now explicitly states that the current Lie-style composition dataset is synthetic and contains stable lexical markers. The result is a controlled diagnostic, not a general natural-language operator algebra.

Mixed Signed-Permutation Results

The Track 2 draft now explains that several triples are below null in some models, while others are above null. `QMT` is promoted only because it is the only tested triple that is below null across all five models after table-level correction.

Still Required Before Submission

1. Add a Procrustes alignment null baseline for UPAT:
   - random-label or random-pairing alignment
   - report null distribution for F1 gain
2. Increase shuffle controls from 100 permutations to at least 1000 before reporting precise p-values.
3. Add null baselines for pairwise commutator norms `||[A,B]||`.
4. Replace hand-written Lie templates with grammar-generated paraphrase templates.
5. Run endpoint-only controls for composition endpoints.
6. Convert large/modern Track 1 spot-checks into multiseed runs if Track 1 is promoted to submission.

Current Submission Readiness

The package is now stronger as a research record, but it should not be submitted as a main-track paper yet. The best current framing is:

- Track 1: a representation-geometry study with clear endpoint confounders and a hard-holdout boundary.
- Track 2: a diagnostic study of local signed-permutation coherence, with `QMT` as the only cross-model stable triple and negation as a composition instability.

Updated Research Roadmap

Research Program: From Linguistic Transformation Vectors to Composition Diagnostics

Core question

Can linguistic transformations be represented not only as separable classes in transformer embedding spaces, but as reusable geometric operations with meaningful composition structure?

The project currently has two paper tracks.

Strategic narrative

The strongest long-term story is broader than either current draft:

> Transformers may encode linguistic operators as geometric objects in a low-dimensional, partially universal transformation subspace, and this subspace may be transferable across architectures and languages.

Current results are not enough to claim this as a complete theory yet. The Procrustes transfer numbers now survive large random-label, random-pairing, random-orthogonal, and held-out-anchor alignment controls, but RISE already provides a stronger published neighboring result for spherical/geodesic semantic-syntactic transformations across languages and embedding models. The roadmap below therefore treats cross-model transfer as a stress-test and comparison track, not as an unqualified novelty claim.

Closest prior-work anchor:

- Freenor and Alvarez 2026, RISE, already demonstrates rotor-based discourse-level semantic-syntactic transformation geometry across languages and embedding models. Our work must be positioned as endpoint-controlled delta diagnostics, Procrustes/null stress testing, and ordered-composition diagnostics rather than as the first evidence for geometric linguistic transformations.

Track 1: Geometric transformation vectors

Working title:

Geometric Transformation Vectors in Transformer Embedding Spaces

Central claim:

Transformer embedding spaces contain displacement directions that encode linguistic transformations. These directions are recoverable across multiple models and holdout regimes.

Current evidence:

- Delta vectors classify transformations well across BERT, RoBERTa, DistilRoBERTa, GPT-2, and DistilGPT-2.
- Syntax holdouts reach `1.0` accuracy across tested models, but the new representation ablation shows `y_only=1.0` as well. This is now interpreted as endpoint/surface leakage, not as deep generalization.
- Full semantic holdouts remain above chance, with accuracies around `0.82-0.89`.
- Entity and variant holdouts remain strong, especially for BERT-family models.
- Modern/larger spot-checks now support the main delta advantage:
  - BERT-large Linear SVC: `delta=0.903`, `concat=0.851`, `y_only=0.838`
  - BERT-large logistic regression: `delta=0.854`, `concat=0.760`, `y_only=0.750`
  - DeBERTa-v3-base Linear SVC: `delta=0.812`, `concat=0.776`, `y_only=0.747`
  - DeBERTa-v3-base logistic regression: `delta=0.752`, `concat=0.694`, `y_only=0.726`
- UPAT hard-holdout is a boundary result: `delta` is worse than `y_only` for BERT, RoBERTa, and GPT-2, and no UPAT delta-vs-y McNemar test is significant.
- Full-semantic pooling ablation now supports mean pooling as a defensible main choice, while showing that the decoder effect is partly pooling-dependent.

Scientific status:

This is the more mature paper. It can be written as a representation-geometry result with careful controls against template memorization.

Main risk:

Some signal may come from target-sentence artifacts rather than pure transformation geometry. The concrete high-risk case is now confirmed: `syntax=1.0` reproduces under `y_only`, so it must be interpreted as a target/surface artifact unless future controls overturn that. UPAT also shows that `delta > y_only` is not universal under small hard-holdout regimes. The paper needs strong source-only, target-only, delta-only, and paraphrase controls.

Track 2: Signed permutation coherence for linguistic operators

Working title:

Third-Order Signed Permutation Coherence for Linguistic Transformations in Transformer Embedding Spaces

Central claim:

Some linguistic transformations behave like locally noncommuting operations in embedding space. Certain triples show third-order signed permutation cancellation stronger than permutation-null baselines, but the evidence supports local composition structure rather than a global Lie algebra.

Current evidence:

- Composition order matters: `AB` and `BA` often differ substantially.
- Semantic equivalence controls show lower noncommutativity for equivalent pairs than for non-equivalent pairs.
- Antisymmetry is not treated as evidence. It is a tautological implementation check because `[A,B] = delta_AB - delta_BA` implies `[B,A] = -[A,B]`.
- The non-tautological third-order signed permutation diagnostic is:

```text
S(A,B,C) = ABC + BCA + CAB - ACB - CBA - BAC
```

- The `QMT` triple shows robust cancellation across tested models:
  - BERT: ratio to permutation null `0.683`, CI `[0.677, 0.689]`
  - DistilRoBERTa: ratio `0.631`, CI `[0.627, 0.636]`
  - RoBERTa: ratio `0.623`, CI `[0.617, 0.629]`
- Decoder replication is now partially complete:
  - GPT-2 `QMT`: ratio `0.539`, CI `[0.519, 0.561]`
  - DistilGPT-2 `QMT`: ratio `0.771`, CI `[0.745, 0.797]`
  - Negation-containing triples remain mixed and model-dependent.
- Decoder pairwise composition summaries are now added, not only decoder third-order summaries.
- Multiple-testing correction over `4 triples x 5 models` supports the narrowed claim: `QMT` is the only below-null triple passing across all five tested models.

Scientific status:

This is promising but more fragile. It should be framed as a null-controlled diagnostic, not as proof of a Lie algebra or as a formal Jacobi identity.

Main risk:

Hand-written templates may induce or suppress cancellation. GPT-2/DistilGPT-2 now support `QMT` below-null cancellation, but negation behaves inconsistently across architectures. The current Lie-style templates still contain stable lexical markers, so the next experiments need grammar-driven templates, endpoint-only controls, commutator null baselines, and a focused negation analysis before expanding the operator set.

Track 3: Cross-model transformation transfer

Working title:

Stress-Testing Cross-Model Transformation Transfer in Transformer Embedding Spaces

Central hypothesis:

Transformation geometry may be partially model-independent after low-dimensional alignment. A classifier trained on transformation deltas in one model may transfer to another model after Procrustes alignment.

Promising exploratory evidence:

- BERT to `all-mpnet-base-v2`: raw F1 around `0.15`, aligned F1 around `0.857`.
- BERT to `all-MiniLM-L6-v2`: raw F1 around `0.17`, aligned F1 around `0.754`.
- UPAT-large cross-model results show large aligned-transfer gains across several architectures.
- A scaled null audit (`N=1000` repeats per direction and null type) found that every non-identity cross-model direction stayed above random-label, random-pairing, and random-orthogonal null baselines.
- Mean observed aligned F1 was `0.684651`.
- Mean null F1 was `0.173017` for random-label, `0.148036` for random-pairing, and `0.112103` for random-orthogonal.
- No null repeat reached observed aligned F1 in any of the `30 x 3` direction/null tests, so all empirical p-values are at the `N=1000` resolution floor: `1/(1000+1)=0.000999`.
- A held-out alignment-size curve now fits Procrustes maps on `1200` auxiliary anchor texts that are disjoint from the classifier train/test texts.
- Held-out anchor mean F1 rises from `0.452046` at `25` anchors to `0.661928` at `1000` anchors, compared with raw cross-model mean F1 `0.241524` and full-anchor Procrustes mean F1 `0.684651`.
- At `1000` held-out anchors, no direction falls below its raw cross-model baseline.
- A first RISE-aware UPAT comparison now evaluates `mdv_raw`, `mdv_unit`, and a simplified spherical `rise_style` prototype baseline. Within-model target prediction cosine is high (`rise_style=0.923008`), while cross-model target prediction is harder (`rise_style` mean cosine `0.578347`, nearest-target label F1 `0.445518`) than delta-classifier transfer in aligned spaces (`0.684651` mean F1).

Current status:

This is now a serious candidate for a complementary paper, but not as a "first universal geometry" claim. RISE is stronger on cross-lingual/cross-model geometric transformation modeling. Our distinct angle is stress testing: null-controlled Procrustes transfer, held-out anchors, endpoint controls, and comparison against simpler delta/MDV/RISE-style baselines.

Required gates before promotion:

1. Add bootstrap confidence intervals and direction-family summaries for the held-out alignment and RISE-aware comparison curves.
2. Stress-test anchor-domain diversity, e.g. anchors from a different template family or natural

paraphrase pool.

3. Improve the RISE-aware comparison:
   - verify the simplified `rise_style` baseline against the published RISE implementation if feasible
   - add confidence intervals
   - clarify target-prediction versus class-discrimination metrics
4. Reverse-direction transfer summary by model family, e.g. small model to large model and large model to small model.
5. Stronger architectures if feasible, such as Llama/Mistral-class embedding spaces or high-quality sentence encoders.
6. Package the result as a standalone Track 3 draft only after the RISE/MDV comparison and confidence-interval checks.

Track 4: Transformation vectors as editors

Working title:

From Transformation Geometry to Controllable Linguistic Editing

Central hypothesis:

If transformation directions are real, they should not only classify transformations; they should also steer generation or hidden states toward those transformations.

Minimal experiment:

1. Use GPT-2 as the first testbed.
2. Compute a centroid delta for a transformation such as negation or question formation.
3. Inject the vector into the residual stream during generation.
4. Measure whether outputs systematically acquire the target transformation.
5. Compare against random-vector, norm-matched, and wrong-class controls.

Scientific payoff:

This would move the project from descriptive geometry to causal intervention. A positive result would connect the work to representation steering and controllable generation.

Track 5: Cross-lingual transformation geometry

Working title:

Language-Invariant Geometry of Linguistic Transformations

Central hypothesis:

If transformation geometry is universal rather than English-template-specific, aligned multilingual models should show comparable transformation subspaces across languages.

Minimal experiment:

1. Use mBERT or XLM-R.
2. Build parallel transformation pairs in English and one or more non-English languages.
3. Compare delta geometry within each language.
4. Align language-specific transformation spaces with Procrustes.
5. Test whether classifiers or centroids transfer across languages.

Scientific payoff:

This is the cleanest answer to the criticism that the current effects may be English grammar or template artifacts.

Track 6: Dimensionality of transformation manifolds

Working title:

Effective Dimensionality of Linguistic Transformation Subspaces

Central hypothesis:

Each transformation may occupy a low-dimensional subspace rather than a single direction or the full embedding space.

Measurements:

- PCA spectra of per-class delta matrices.
- Participation ratio of singular values.
- Classification accuracy as a function of retained dimensions.
- Sample-complexity curves for learning each transformation.

Scientific payoff:

This would quantify the complexity of each linguistic operator and explain why capacity curves keep improving with more examples.

Iterative logic

The research direction is:

1. Show transformations are encoded as recoverable displacement vectors.
2. Test whether those directions compose nontrivially.
3. Separate semantic order effects from wording artifacts.
4. Test local third-order signed permutation structure with null baselines.
5. Expand from hand-written probes to systematic grammar-generated probes.
6. Decide which claims are strong enough for publication.

Common methodology standards

Every promoted claim should have at least one control:

- holdout split
- source-only and target-only baseline
- permutation/null baseline
- semantic equivalence control
- bootstrap confidence interval
- cross-model replication
- template de-duplication check

Negative results remain part of the research record.

Next research plan

Short term

1. Close methodology blockers required for any submission:
   - remove antisymmetry from the evidence narrative; already done in the draft, keep it that way
   - keep Procrustes null baselines in the evidence packet; `N=1000` random-label/random-pairing/random-orthogonal controls are now complete
   - increase shuffle/permutation controls to at least 1000, ideally 5000 for final numbers
   - add commutator norm null baselines
2. Resolve UPAT:
   - either expand UPAT and match train sizes against the main dataset
   - or keep it explicitly as a hard negative control and narrow Track 1 claims
3. Extend UPAT alignment controls beyond the completed `N=1000` null audit and held-out alignment curve:
   - bootstrap confidence intervals
   - direction-family summary
   - anchor-domain diversity check
4. Test cross-model transformation transfer as a RISE-aware stress-test paper:
   - reverse-direction transfer
   - alignment-size curve
   - random-label/null alignment controls
   - RISE/MDV-style prototype baseline

5. Treat the syntax holdout as resolved for the current draft: `y_only=1.0` and layer-0 `1.0` mean the `syntax=1.0` result is a target/surface artifact unless a future redesigned split proves otherwise.
6. Convert large/modern Track 1 spot-checks into multiseed runs if Track 1 is promoted to submission.
7. Build a grammar-driven template generator for `N,Q,M,T` that produces many paraphrases without duplicate endpoints.
8. Add automated dataset validation:
   - no duplicate endpoint strings within a composition tuple
   - balanced subjects/actions
   - controlled lexical overlap
9. Add commutator null baselines for `||[A,B]||` using random or label-shuffled operations with matched norms.
10. Re-run composition and signed-permutation diagnostics on generated templates.
11. Add target-only and endpoint-only baselines for the Lie-style paper.
12. Regenerate a dated PDF packet after every major run.

Medium term

1. Run the GPT-2 steering-vector experiment for one transformation, starting with negation or question formation.
2. Run the cross-lingual mBERT/XLM-R transformation-transfer experiment.
3. Measure effective dimensionality of transformation subspaces:
   - participation ratio
   - PCA retention curves
   - accuracy versus retained dimensions
4. Run layerwise and pooling ablations before expanding the operator set. If final-layer mean pooling is not the best representation, the expanded operator experiments should use the validated representation instead.
5. Add more linguistic operators:
   - passive voice
   - modality strength
   - evidentiality
   - aspect
   - conditionality
   - quantifier changes
6. Add paraphrase robustness with semantically equivalent endpoint variants.

Paper milestones

1. Track 1 is arXiv-ready only when this checklist is complete:
   - syntax holdout breakdown is in the draft and interpreted as endpoint leakage
   - McNemar tests are reported in the text
   - multiseed standard deviations are reported

   - at least one prior-work baseline or comparison is written up, such as task vectors or function vectors
   - at least one model outside the original five is included as a spot-check; currently satisfied by BERT-large and DeBERTa-v3-base
   - UPAT hard-holdout is either resolved experimentally or explicitly reported as a negative boundary condition

2. Keep Track 2 as a diagnostics paper until grammar-generated templates and endpoint-only controls succeed.
3. Promote Track 3 only if it becomes clearly complementary to RISE: null-controlled Procrustes transfer plus held-out anchors plus an explicit RISE/MDV comparison. The main narrative should be stress-testing cross-model transfer, not claiming first discovery of universal transformation geometry.
4. If steering works, write Track 4 as an intervention/controllable-generation paper.
5. If cross-lingual transfer works, it becomes the strongest version of the universality claim.
6. If Track 2 survives grammar-generated controls, write it as a separate diagnostics paper rather than merging it into Track 1.
7. If Track 2 weakens under controls, keep it as a negative/diagnostic section in a broader research note.

RISE Related-Work Positioning

Related Work Positioning: RISE and Our Tracks

This note records how the project should position itself after reading:

Freenor and Alvarez, **"Mapping Semantic & Syntactic Relationships with Geometric Rotation"**, ICLR 2026.

What RISE Already Covers

RISE is the closest and strongest neighboring work. It treats discourse-level semantic-syntactic transformations as geometric operations in sentence-embedding space.

Key points from the paper:

- Method: Rotor-Invariant Shift Estimation (RISE), a spherical/Riemannian rotor method that maps neutral-to-transformed sentence pairs through tangent-space prototypes.
- Transformations: negation, conditionality, and politeness.
- Scope: seven languages and three multilingual embedding models (`text-embedding-3-large`, `bge-m3`, `mBERT`).
- Baselines: mean difference vectors and Procrustes alignment.
- Evaluation: cosine similarity between predicted transformed embeddings and target transformed embeddings on held-out sets, plus cross-language and cross-model transfer.
- Main implication for us: we cannot claim novelty for the broad statement that discourse-level semantic-syntactic transformations have cross-lingual/cross-model geometric structure.

What Our Work Should Not Claim

We should not write any of the following as novelty claims:

- "First evidence that linguistic transformations are geometric operations in sentence embeddings."
- "First cross-model transformation geometry result."
- "Universal transformation subspaces across embedding models" without qualification.
- "Procrustes alignment establishes universality" without comparing to RISE-like spherical methods.

Remaining Distinct Contributions

Our current repository still has several distinct contributions if framed narrowly.

Track 1: Endpoint-Controlled Delta Diagnostics

RISE focuses on predicting transformed embeddings via rotor/MDV-style operations. Track 1 asks a different diagnostic question:

> Does `embedding(y) - embedding(x)` add class-discriminative transformation information beyond `x_only`, `y_only`, and `concat` baselines?

Distinct value:

- explicit endpoint leakage controls
- multiseed delta/y_only/concat/x_only ablations
- McNemar tests
- hard negative/boundary reporting via UPAT
- syntax `1.0` reinterpreted as endpoint/surface leakage rather than a success

This makes Track 1 a conservative control paper, not a stronger version of RISE.

Track 2: Ordered Composition Diagnostics

RISE reports that its transformations behave commutatively in the tangent-space framing. Track 2 asks almost the opposite question:

> Do ordered linguistic transformations show noncommutative or signed-permutation composition structure in embedding endpoints?

Distinct value:

- pairwise order diagnostics (`AB` versus `BA`)
- semantic-equivalence controls
- third-order signed-permutation diagnostic
- negation as a composition instability rather than simply a robust one-step transformation

This is the cleanest conceptual separation from RISE. It should be framed as a diagnostic of local composition/order effects, not as a steering method.

Track 3: Cross-Model Transfer Stress Tests

Track 3 is closest to RISE and therefore needs the most careful positioning.

Current distinct value:

- direct classifier transfer over transformation deltas
- `N=1000` random-label, random-pairing, and random-orthogonal Procrustes nulls
- held-out alignment-size curve using auxiliary anchor texts disjoint from classifier train/test texts
- explicit reporting of raw, full-anchor, and held-out-anchor transfer gaps

Now addressed as a first-pass comparison:

Track 3 now includes a first-pass RISE-inspired comparison on UPAT:

- MDV/prototype prediction baseline on UPAT
- spherical normalization / tangent-space variant if feasible
- comparison of classifier-transfer F1 versus target-embedding prediction similarity

Current interpretation:

- Prototype methods answer a target-embedding prediction question.
- Procrustes delta-classifier transfer answers a class-discriminative transfer question.
- Within-model prototype target cosine is high.
- Cross-model prototype prediction is harder and model-pair dependent.
- The simplified `rise_style` baseline improves nearest-target label retrieval over MDV in the current UPAT setup, but it is not a full reproduction of RISE.

Track 3 remains a useful stress-test package, but it still should not be promoted as a main novelty paper against RISE until confidence intervals, anchor-domain robustness, and a more faithful RISE implementation are added.

Recommended Framing

Use this framing in future drafts:

> RISE shows that some discourse-level semantic-syntactic transformations can be modeled as spherical/geodesic operations across languages and embedding models. Our work is complementary: we stress-test simpler endpoint and delta representations under endpoint-only controls, hard holdouts, Procrustes nulls, and held-out alignment anchors, and we separately test whether ordered linguistic transformations exhibit noncommutative composition diagnostics.

Immediate Backlog Changes

1. Add RISE to every related-work section as the closest prior work.
2. Stop using "universal transformation subspaces" as an unqualified Track 3 title.
3. Rename Track 3 working title toward "stress-testing cross-model transformation transfer".
4. Extend the first RISE/MDV comparison with confidence intervals and, if feasible, a more faithful RISE implementation before promoting Track 3.
5. Emphasize Track 2 composition/order diagnostics as the most distinct paper-level contribution.

Code listing: run_lie_algebraic_identities.py (1)

```
import gc
import json
import os
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import torch
import matplotlib.pyplot as plt

from transformers import AutoTokenizer, AutoModel
from sklearn.decomposition import PCA
from sklearn.metrics.pairwise import cosine_similarity

warnings.filterwarnings("ignore")


class LieAlgebraicIdentitiesAudit:
    def __init__(self):
        self.out_dir = Path(os.getenv("LIE_ALGEBRA_OUT_DIR", "results/experiments/lie_algebraic_identities_results"))
        self.csv_dir = self.out_dir / "csv"
        self.fig_dir = self.out_dir / "figures"
        self.csv_dir.mkdir(parents=True, exist_ok=True)
        self.fig_dir.mkdir(parents=True, exist_ok=True)

        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.hf_token = None
        self.pca_dim = 64
        self.n_jacobi_null = 1000
        self.n_bootstrap = 2000

        self.models = [
            model.strip()
            for model in os.getenv(
                "LIE_ALGEBRA_MODELS",
                "bert-base-uncased,distilroberta-base,roberta-base",
            ).split(",")
            if model.strip()
        ]

        self.ops = ["N", "Q", "M", "T"]

        self.subjects = [
            "scientist", "engineer", "teacher", "doctor", "programmer",
            "researcher", "analyst", "manager", "wizard", "dragon",
            "queen", "robot", "pirate", "oracle", "knight", "alien",
        ]

        self.actions = [
            ("accepted", "accept", "the explanation"),
            ("completed", "complete", "the repair"),
            ("confirmed", "confirm", "the answer"),
            ("approved", "approve", "the treatment"),
```

Code listing: run_lie_algebraic_identities.py (2)

```
            ("fixed", "fix", "the bug"),
            ("supported", "support", "the theory"),
            ("verified", "verify", "the report"),
            ("guarded", "guard", "the treasure"),
            ("opened", "open", "the portal"),
            ("signed", "sign", "the treaty"),
        ]

    def log(self, msg):
        print(msg, flush=True)

    def base_sentence(self, subject, past, obj):
        return f"The {subject} {past} {obj}."

    def compose(self, seq, subject, past, base, obj):
        s = "".join(seq)

        mapping = {
            "": f"The {subject} {past} {obj}.",

            "N": f"The {subject} failed to {base} {obj}.",
            "Q": f"Could the {subject} {base} {obj}?",
            "M": f"The {subject} allegedly {past} {obj}.",
            "T": f"The {subject} will {base} {obj} tomorrow.",

            "NQ": f"Could the {subject} fail to {base} {obj}?",
            "QN": f"Is it false that the {subject} {past} {obj}?",

            "NM": f"The {subject} allegedly failed to {base} {obj}.",
            "MN": f"Allegedly, the {subject} failed to {base} {obj}.",

            "NT": f"The {subject} will fail to {base} {obj} tomorrow.",
            "TN": f"The {subject} failed to have {past} {obj} earlier.",

            "QM": f"Could the {subject} allegedly {base} {obj}?",
            "MQ": f"Is it alleged that the {subject} {past} {obj}?",

            "QT": f"Could the {subject} {base} {obj} tomorrow?",
            "TQ": f"Will the {subject} {base} {obj} tomorrow?",

            "MT": f"The {subject} will allegedly {base} {obj} tomorrow.",
            "TM": f"The {subject} was allegedly going to {base} {obj}.",
        }

        if s in mapping:
            return mapping[s]

        # canonical triple templates
        if s == "NQM":
            return f"Could the {subject} allegedly fail to {base} {obj}?"
        if s == "QMN":
            return f"Is it false that the {subject} allegedly {past} {obj}?"
        if s == "MNQ":
            return f"Is it alleged that the {subject} failed to {base} {obj}?"
```

Code listing: run_lie_algebraic_identities.py (3)

```

        if s == "QNM":
            return f"Could it be false that the {subject} allegedly {past} {obj}?"
        if s == "NMQ":
            return f"Could it be alleged that the {subject} failed to {base} {obj}?"
        if s == "MQN":
            return f"Allegedly, is it false that the {subject} {past} {obj}?"

        if s == "NMT":
            return f"The {subject} will allegedly fail to {base} {obj} tomorrow."
        if s == "MTN":
            return f"The {subject} was allegedly going to fail to {base} {obj}."
        if s == "TNM":
            return f"Earlier, the {subject} allegedly failed to have {past} {obj}."

        if s == "NTM":
            return f"Allegedly, the {subject} will fail to {base} {obj} tomorrow."
        if s == "TMN":
            return f"Earlier, it was alleged that the {subject} failed to have {past} {obj}."
        if s == "MNT":
            return f"It is alleged that the {subject} will fail to {base} {obj} tomorrow."

        if s == "NQT":
            return f"Could the {subject} fail to {base} {obj} tomorrow?"
        if s == "QTN":
            return f"Is it false that the {subject} will {base} {obj} tomorrow?"
        if s == "TNQ":
            return f"Could the {subject} have failed to {base} {obj} earlier?"

        if s == "QNT":
            return f"Could it be false that the {subject} will {base} {obj} tomorrow?"
        if s == "NTQ":
            return f"Could it be that the {subject} will fail to {base} {obj} tomorrow?"
        if s == "TQN":
            return f"Will it be false that the {subject} {base} {obj} tomorrow?"

        if s == "QMT":
            return f"Could the {subject} allegedly {base} {obj} tomorrow?"
        if s == "MTQ":
            return f"Could it be alleged that the {subject} will {base} {obj} tomorrow?"
        if s == "TQM":
            return f"Allegedly, will the {subject} {base} {obj} tomorrow?"

        if s == "MQT":
            return f"Will it be alleged that the {subject} {base} {obj} tomorrow?"
        if s == "QTM":
            return f"Allegedly, could the {subject} {base} {obj} tomorrow?"
        if s == "TMQ":
            return f"Could the {subject} have allegedly planned to {base} {obj}?"

        raise ValueError(f"Unsupported composition: {s}")

    def build_dataset(self, n_templates=100, seed=42):
        rng = np.random.default_rng(seed)
```

Code listing: run_lie_algebraic_identities.py (4)

```
        combos = [(s, a) for s in self.subjects for a in self.actions]
        rng.shuffle(combos)
        combos = combos[:n_templates]

        rows = []

        pairs = [("N", "Q"), ("N", "M"), ("N", "T"), ("Q", "M"), ("Q", "T"), ("M", "T")]
        triples = [("N", "Q", "M"), ("N", "Q", "T"), ("N", "M", "T"), ("Q", "M", "T")]

        for i, (subject, action) in enumerate(combos):
            past, base, obj = action
            source = self.base_sentence(subject, past, obj)

            for a, b in pairs:
                rows.append({
                    "template_id": i,
                    "kind": "antisymmetry",
                    "source": source,
                    "a": a,
                    "b": b,
                    "c": "",
                    "ab": self.compose([a, b], subject, past, base, obj),
                    "ba": self.compose([b, a], subject, past, base, obj),
                })

            for a, b, c in triples:
                rows.append({
                    "template_id": i,
                    "kind": "jacobi",
                    "source": source,
                    "a": a,
                    "b": b,
                    "c": c,
                    "abc": self.compose([a, b, c], subject, past, base, obj),
                    "bca": self.compose([b, c, a], subject, past, base, obj),
                    "cab": self.compose([c, a, b], subject, past, base, obj),
                    "acb": self.compose([a, c, b], subject, past, base, obj),
                    "cba": self.compose([c, b, a], subject, past, base, obj),
                    "bac": self.compose([b, a, c], subject, past, base, obj),
                })

        df = pd.DataFrame(rows)
        df.to_csv(self.csv_dir / "lie_algebraic_identities_dataset.csv", index=False)
        return df

    @torch.no_grad()
    def get_embeddings(self, model_name, texts, batch_size=16):
        tokenizer = AutoTokenizer.from_pretrained(model_name, token=self.hf_token)

        if tokenizer.pad_token is None:
            tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"

        model = AutoModel.from_pretrained(model_name, token=self.hf_token).to(self.device)
        model.eval()
```

Code listing: run_lie_algebraic_identities.py (5)

```

        embs = []

        for i in range(0, len(texts), batch_size):
            batch = texts[i:i + batch_size]

            enc = tokenizer(
                batch,
                padding=True,
                truncation=True,
                max_length=128,
                return_tensors="pt",
            ).to(self.device)

            out = model(**enc)
            hidden = out.last_hidden_state
            mask = enc["attention_mask"].unsqueeze(-1)

            pooled = (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
            embs.append(pooled.detach().cpu().numpy())

        del model
        gc.collect()

        if self.device == "cuda":
            torch.cuda.empty_cache()

        return np.vstack(embs)

    def cos(self, x, y):
        return float(cosine_similarity(x.reshape(1, -1), y.reshape(1, -1))[0, 0])

    def alternating_null_stats(self, vectors, observed_norm, scale, rng):
        null_relative = []
        n = len(vectors)

        for _ in range(self.n_jacobi_null):
            positive = set(rng.choice(n, size=n // 2, replace=False).tolist())
            signed_sum = np.zeros_like(vectors[0])

            for i, vector in enumerate(vectors):
                signed_sum += vector if i in positive else -vector

            null_relative.append(np.linalg.norm(signed_sum) / scale)

        null_relative = np.asarray(null_relative, dtype=float)
        observed_relative = observed_norm / scale

        return {
            "null_mean_relative_jacobi_norm": float(null_relative.mean()),
            "null_std_relative_jacobi_norm": float(null_relative.std(ddof=1)),
            "jacobi_to_null_mean_ratio": float(observed_relative / (null_relative.mean() + 1e-12)),
            "null_percentile_smaller_is_better": float((null_relative <= observed_relative).mean()),
        }
```

Code listing: run_lie_algebraic_identities.py (6)

```

    def bootstrap_mean_ci(self, values, rng, alpha=0.05):
        values = np.asarray(values, dtype=float)
        if len(values) == 0:
            return np.nan, np.nan

        draws = rng.choice(values, size=(self.n_bootstrap, len(values)), replace=True).mean(axis=1)
        return (
            float(np.quantile(draws, alpha / 2)),
            float(np.quantile(draws, 1 - alpha / 2)),
        )

    def compute_for_model(self, model_name, df):
        self.log("\n" + "=" * 80)
        self.log(f"MODEL: {model_name}")
        self.log("=" * 80)

        text_cols = ["source", "ab", "ba", "abc", "bca", "cab", "acb", "cba", "bac"]
        all_texts = set()

        for col in text_cols:
            if col in df.columns:
                all_texts |= set(df[col].dropna().values)

        all_texts = sorted(all_texts)
        idx = {t: i for i, t in enumerate(all_texts)}

        raw = self.get_embeddings(model_name, all_texts)

        dim = min(self.pca_dim, raw.shape[0] - 1, raw.shape[1])
        pca = PCA(n_components=dim, random_state=42)
        vecs = pca.fit_transform(raw)

        anti_rows = []
        jacobi_rows = []

        anti_df = df[df["kind"] == "antisymmetry"]

        for row in anti_df.itertuples():
            x = vecs[idx[row.source]]
            ab = vecs[idx[row.ab]]
            ba = vecs[idx[row.ba]]

            delta_ab = ab - x
            delta_ba = ba - x

            comm_ab = delta_ab - delta_ba
            comm_ba = delta_ba - delta_ab

            antisym_error = np.linalg.norm(comm_ab + comm_ba)
            comm_norm = np.linalg.norm(comm_ab)

            anti_rows.append({
                "model": model_name,
```

Code listing: run_lie_algebraic_identities.py (7)

```
                "template_id": row.template_id,
                "pair": f"{row.a}{row.b}_vs_{row.b}{row.a}",
                "a": row.a,
                "b": row.b,
                "comm_norm": float(comm_norm),
                "antisym_error": float(antisym_error),
                "relative_antisym_error": float(antisym_error / (comm_norm + 1e-12)),
                "cos_comm_ab_neg_comm_ba": self.cos(comm_ab, -comm_ba),
            })

        jacobi_df = df[df["kind"] == "jacobi"]
        seed = sum((i + 1) * ord(ch) for i, ch in enumerate(model_name)) % (2 ** 32)
        rng = np.random.default_rng(seed)

        for row in jacobi_df.itertuples():
            x = vecs[idx[row.source]]

            abc = vecs[idx[row.abc]] - x
            bca = vecs[idx[row.bca]] - x
            cab = vecs[idx[row.cab]] - x

            acb = vecs[idx[row.acb]] - x
            cba = vecs[idx[row.cba]] - x
            bac = vecs[idx[row.bac]] - x

            # Jacobi-like alternating sum over permutations:
            # J = ABC + BCA + CAB - ACB - CBA - BAC
            jacobi = abc + bca + cab - acb - cba - bac

            positive_norm = np.linalg.norm(abc) + np.linalg.norm(bca) + np.linalg.norm(cab)
            negative_norm = np.linalg.norm(acb) + np.linalg.norm(cba) + np.linalg.norm(bac)
            scale = 0.5 * (positive_norm + negative_norm) + 1e-12

            jacobi_norm = np.linalg.norm(jacobi)
            null_stats = self.alternating_null_stats(
                [abc, bca, cab, acb, cba, bac],
                jacobi_norm,
                scale,
                rng,
            )

            jacobi_rows.append({
                "model": model_name,
                "template_id": row.template_id,
                "triple": f"{row.a}{row.b}{row.c}",
                "a": row.a,
                "b": row.b,
                "c": row.c,
                "jacobi_norm": float(jacobi_norm),
                "relative_jacobi_norm": float(jacobi_norm / scale),
                **null_stats,
            })

        anti_result = pd.DataFrame(anti_rows)
```

Code listing: run_lie_algebraic_identities.py (8)

```
        jacobi_result = pd.DataFrame(jacobi_rows)

        anti_result.to_csv(self.csv_dir / f"antisymmetry_raw_{self.safe_name(model_name)}.csv", index=False)
        jacobi_result.to_csv(self.csv_dir / f"jacobi_raw_{self.safe_name(model_name)}.csv", index=False)

        return anti_result, jacobi_result

    def safe_name(self, model_name):
        return model_name.replace("/", "_").replace("-", "_")

    def summarize(self, anti_all, jacobi_all):
        anti_summary = (
            anti_all.groupby(["model", "pair"])
            .agg(
                mean_comm_norm=("comm_norm", "mean"),
                mean_relative_antisym_error=("relative_antisym_error", "mean"),
                mean_cos_comm_ab_neg_comm_ba=("cos_comm_ab_neg_comm_ba", "mean"),
                n=("pair", "count"),
            )
            .reset_index()
        )

        jacobi_summary = (
            jacobi_all.groupby(["model", "triple"])
            .agg(
                mean_jacobi_norm=("jacobi_norm", "mean"),
                mean_relative_jacobi_norm=("relative_jacobi_norm", "mean"),
                mean_null_relative_jacobi_norm=("null_mean_relative_jacobi_norm", "mean"),
                mean_jacobi_to_null_mean_ratio=("jacobi_to_null_mean_ratio", "mean"),
                mean_null_percentile_smaller_is_better=("null_percentile_smaller_is_better", "mean"),
                n=("triple", "count"),
            )
            .reset_index()
        )

        rng = np.random.default_rng(20260611)
        ci_rows = []

        for row in jacobi_summary.itertuples():
            group = jacobi_all[
                (jacobi_all["model"] == row.model)
                & (jacobi_all["triple"] == row.triple)
            ]

            rel_low, rel_high = self.bootstrap_mean_ci(group["relative_jacobi_norm"].values, rng)
            ratio_low, ratio_high = self.bootstrap_mean_ci(group["jacobi_to_null_mean_ratio"].values, rng)

            ci_rows.append({
                "model": row.model,
                "triple": row.triple,
                "relative_jacobi_norm_ci95_low": rel_low,
                "relative_jacobi_norm_ci95_high": rel_high,
                "jacobi_to_null_mean_ratio_ci95_low": ratio_low,
                "jacobi_to_null_mean_ratio_ci95_high": ratio_high,
```

Code listing: run_lie_algebraic_identities.py (9)

```
            })

        jacobi_summary = jacobi_summary.merge(pd.DataFrame(ci_rows), on=["model", "triple"], how="left")

        anti_summary.to_csv(self.csv_dir / "antisymmetry_summary.csv", index=False)
        jacobi_summary.to_csv(self.csv_dir / "jacobi_summary.csv", index=False)

        return anti_summary, jacobi_summary

    def plot_heatmap(self, df, index_col, value_col, filename, title):
        rows = sorted(df["model"].unique())
        cols = sorted(df[index_col].unique())

        mat = np.zeros((len(rows), len(cols)))

        for i, model in enumerate(rows):
            for j, col in enumerate(cols):
                val = df[(df["model"] == model) & (df[index_col] == col)][value_col].values
                mat[i, j] = val[0] if len(val) else np.nan

        plt.figure(figsize=(10, 5))
        plt.imshow(mat, aspect="auto")
        plt.colorbar(label=value_col)

        plt.xticks(np.arange(len(cols)), cols, rotation=35, ha="right")
        plt.yticks(np.arange(len(rows)), rows)
        plt.title(title)

        for i in range(mat.shape[0]):
            for j in range(mat.shape[1]):
                plt.text(j, i, f"{mat[i, j]:.2f}", ha="center", va="center")

        path = self.fig_dir / filename
        plt.tight_layout()
        plt.savefig(path, dpi=220, bbox_inches="tight")
        plt.close()
        self.log(f"Saved figure: {path}")

    def run(self):
        self.log(f"DEVICE: {self.device}")
        self.log(f"OUT DIR: {self.out_dir}")

        df = self.build_dataset(n_templates=100, seed=42)

        anti_results = []
        jacobi_results = []

        for model in self.models:
            anti, jacobi = self.compute_for_model(model, df)
            anti_results.append(anti)
            jacobi_results.append(jacobi)

        anti_all = pd.concat(anti_results, ignore_index=True)
        jacobi_all = pd.concat(jacobi_results, ignore_index=True)
```

Code listing: run_lie_algebraic_identities.py (10)

```

        anti_all.to_csv(self.csv_dir / "antisymmetry_raw_all_models.csv", index=False)
        jacobi_all.to_csv(self.csv_dir / "jacobi_raw_all_models.csv", index=False)

        anti_summary, jacobi_summary = self.summarize(anti_all, jacobi_all)

        self.plot_heatmap(
            anti_summary,
            "pair",
            "mean_cos_comm_ab_neg_comm_ba",
            "01_antisymmetry_cosine_heatmap.png",
            "Antisymmetry Check: cos([A,B], -[B,A])",
        )

        self.plot_heatmap(
            anti_summary,
            "pair",
            "mean_relative_antisym_error",
            "02_antisymmetry_error_heatmap.png",
            "Antisymmetry Error",
        )

        self.plot_heatmap(
            jacobi_summary,
            "triple",
            "mean_relative_jacobi_norm",
            "03_jacobi_relative_norm_heatmap.png",
            "Jacobi-like Alternating Sum Relative Norm",
        )

        self.plot_heatmap(
            jacobi_summary,
            "triple",
            "mean_jacobi_to_null_mean_ratio",
            "04_jacobi_vs_permutation_null_heatmap.png",
            "Jacobi-like Norm / Permutation Null Mean",
        )

        metadata = {
            "antisymmetry": "checks whether [A,B] ≈ -[B,A]",
            "jacobi_like": "checks alternating sum ABC+BCA+CAB-ACB-CBA-BAC",
            "jacobi_null": (
                "compares the observed alternating sum with random 3-positive/3-negative "
                "sign assignments over the same six composition vectors; lower ratio and "
                "lower percentile mean stronger Jacobi-like cancellation"
            ),
            "caution": (
                "A literal nested-commutator Jacobi expansion over the same six embedded "
                "composition endpoints cancels by construction, so this script reports a "
                "non-tautological alternating third-order cancellation diagnostic instead."
            ),
            "note": "This is an empirical Lie-style diagnostic, not a formal proof of Lie algebra.",
            "models": self.models,
            "n_jacobi_null": self.n_jacobi_null,
```

Code listing: run_lie_algebraic_identities.py (11)

```
            "n_bootstrap": self.n_bootstrap,
            "jacobi_triples": ["NQM", "NQT", "NMT", "QMT"],
        }

        with open(self.out_dir / "metadata.json", "w", encoding="utf-8") as f:
            json.dump(metadata, f, indent=2)

        self.log("\nANTISYMMETRY SUMMARY")
        self.log(str(anti_summary))

        self.log("\nJACOBI SUMMARY")
        self.log(str(jacobi_summary))

        self.log("\nDONE")


if __name__ == "__main__":
    audit = LieAlgebraicIdentitiesAudit()
    audit.run()
```

Code listing: run_syntax_representation_ablation.py (1)

```
import gc
import os
from pathlib import Path

import numpy as np
import pandas as pd

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

import lie_llm_syntax_holdout_experiment as syntax


OUT_DIR = Path(os.getenv("SYNTAX_ABLATION_OUT_DIR", "results/experiments/syntax_representation_ablation_results"))
OUT_DIR.mkdir(parents=True, exist_ok=True)

MODELS = [m.strip() for m in os.getenv(
    "SYNTAX_ABLATION_MODELS",
    "bert-base-uncased,roberta-base,distilroberta-base,gpt2,distilgpt2",
).split(",") if m.strip()]

N_BASE = int(os.getenv("SYNTAX_ABLATION_N_BASE", str(syntax.N_BASE)))


def run_classifiers(X, labels, train_mask, test_mask):
    classifiers = {
        "logreg": make_pipeline(
            StandardScaler(),
            LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1),
        ),
        "linear_svc": make_pipeline(
            StandardScaler(),
            LinearSVC(class_weight="balanced", max_iter=20000),
        ),
    }

    rows = []
    for clf_name, clf in classifiers.items():
        clf.fit(X[train_mask], labels[train_mask])
        pred = clf.predict(X[test_mask])
        rows.append({
            "classifier": clf_name,
            "accuracy": float(accuracy_score(labels[test_mask], pred)),
            "n_train": int(train_mask.sum()),
            "n_test": int(test_mask.sum()),
        })

    return rows


def main():
```

Code listing: run_syntax_representation_ablation.py (2)

```
    syntax.OUT_DIR = str(OUT_DIR)
    os.makedirs(syntax.OUT_DIR, exist_ok=True)

    base_df = syntax.make_base_rows(N_BASE)
    pair_df = syntax.build_pairs(base_df)
    prompts, pair_df = syntax.index_prompts(pair_df)

    base_df.to_csv(OUT_DIR / "base_sentences.csv", index=False)
    pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
    pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(
        OUT_DIR / "prompts.csv",
        index=False,
    )

    labels = pair_df["class"].astype(str).to_numpy()
    syntax_family = pair_df["syntax_family"].astype(str).to_numpy()
    train_mask = np.isin(syntax_family, ["simple_svo", "with_context"])
    test_mask = np.isin(syntax_family, ["relative_clause", "temporal_clause", "reported_statement", "conditional"])
    src_idx = pair_df["source_idx"].to_numpy()
    tgt_idx = pair_df["target_idx"].to_numpy()

    all_rows = []

    for model_name in MODELS:
        print(f"\n=== MODEL: {model_name} ===")
        vectors = syntax.get_vectors(model_name, prompts)

        x_src = vectors[src_idx]
        x_tgt = vectors[tgt_idx]
        representations = {
            "x_only": x_src,
            "y_only": x_tgt,
            "delta": x_tgt - x_src,
            "concat": np.concatenate([x_src, x_tgt], axis=1),
        }

        for rep_name, X in representations.items():
            for row in run_classifiers(X, labels, train_mask, test_mask):
                row.update({
                    "model": model_name,
                    "representation": rep_name,
                    "random_baseline": 1 / len(set(labels)),
                    "n_base": N_BASE,
                })
                print(model_name, rep_name, row["classifier"], f"{row['accuracy']:.4f}")
                all_rows.append(row)

        del vectors, x_src, x_tgt, representations
        gc.collect()

    result = pd.DataFrame(all_rows)
    result.to_csv(OUT_DIR / "syntax_representation_ablation.csv", index=False)

    pivot = result.pivot_table(
```

Code listing: run_syntax_representation_ablation.py (3)

```
        index=["model", "classifier"],
        columns="representation",
        values="accuracy",
    ).reset_index()
    pivot.to_csv(OUT_DIR / "syntax_representation_ablation_pivot.csv", index=False)

    print("\nSUMMARY")
    print(pivot)
    print("Saved to", OUT_DIR)


if __name__ == "__main__":
    main()
```

Code listing: run_layerwise_pooling_ablation.py (1)

```
import gc
import os
import time
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC
from transformers import AutoModel, AutoTokenizer

import lie_llm_syntax_holdout_experiment as syntax


OUT_DIR = Path(os.getenv("LAYERWISE_OUT_DIR", "results/experiments/layerwise_pooling_ablation_results"))
OUT_DIR.mkdir(parents=True, exist_ok=True)

MODEL_NAME = os.getenv("LAYERWISE_MODEL", "bert-base-uncased")
N_BASE = int(os.getenv("LAYERWISE_N_BASE", "300"))
BATCH_SIZE = int(os.getenv("LAYERWISE_BATCH_SIZE", "16"))


def pool(hidden, mask, mode):
    if mode == "mean":
        m = mask.unsqueeze(-1).float()
        return (hidden * m).sum(dim=1) / m.sum(dim=1).clamp_min(1e-9)
    if mode == "cls":
        return hidden[:, 0, :]
    if mode == "last_token":
        idx = mask.sum(dim=1).clamp_min(1) - 1
        return hidden[torch.arange(hidden.shape[0]), idx]
    raise ValueError(mode)


def extract_layer_vectors(prompts):
    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"

    model = AutoModel.from_pretrained(MODEL_NAME, output_hidden_states=True)
    model.eval()

    rows_by_key = {}
    started = time.time()

    for start in range(0, len(prompts), BATCH_SIZE):
        batch = prompts[start:start + BATCH_SIZE]
        enc = tokenizer(batch, padding=True, truncation=True, max_length=160, return_tensors="pt")

        with torch.no_grad():
```

Code listing: run_layerwise_pooling_ablation.py (2)

```
            out = model(**enc)

        for layer_idx, hidden in enumerate(out.hidden_states):
            for pooling in ["mean", "cls", "last_token"]:
                key = (layer_idx, pooling)
                rows_by_key.setdefault(key, []).append(pool(hidden, enc["attention_mask"], pooling).cpu().numpy())

        done = start + len(batch)
        print(f"{MODEL_NAME}: {done}/{len(prompts)}, {time.time() - started:.1f}s")

    result = {key: np.vstack(parts) for key, parts in rows_by_key.items()}

    del model
    gc.collect()
    return result


def fit_score(X, labels, train_mask, test_mask):
    classifiers = {
        "logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1)),
        "linear_svc": make_pipeline(StandardScaler(), LinearSVC(class_weight="balanced", max_iter=20000)),
    }

    rows = []
    for clf_name, clf in classifiers.items():
        clf.fit(X[train_mask], labels[train_mask])
        pred = clf.predict(X[test_mask])
        rows.append({
            "classifier": clf_name,
            "accuracy": float(accuracy_score(labels[test_mask], pred)),
        })
    return rows


def main():
    base_df = syntax.make_base_rows(N_BASE)
    pair_df = syntax.build_pairs(base_df)
    prompts, pair_df = syntax.index_prompts(pair_df)

    pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
    pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)

    labels = pair_df["class"].astype(str).to_numpy()
    syntax_family = pair_df["syntax_family"].astype(str).to_numpy()
    train_mask = np.isin(syntax_family, ["simple_svo", "with_context"])
    test_mask = np.isin(syntax_family, ["relative_clause", "temporal_clause", "reported_statement", "conditional"])
    src_idx = pair_df["source_idx"].to_numpy()
    tgt_idx = pair_df["target_idx"].to_numpy()

    vectors_by_key = extract_layer_vectors(prompts)

    all_rows = []
    for (layer_idx, pooling), vectors in vectors_by_key.items():
        x_src = vectors[src_idx]
```

Code listing: run_layerwise_pooling_ablation.py (3)

```
        x_tgt = vectors[tgt_idx]

        for rep_name, X in {
            "delta": x_tgt - x_src,
            "y_only": x_tgt,
        }.items():
            for row in fit_score(X, labels, train_mask, test_mask):
                row.update({
                    "model": MODEL_NAME,
                    "layer": layer_idx,
                    "pooling": pooling,
                    "representation": rep_name,
                    "n_base": N_BASE,
                    "n_train": int(train_mask.sum()),
                    "n_test": int(test_mask.sum()),
                })
                print(row)
                all_rows.append(row)

    result = pd.DataFrame(all_rows)
    result.to_csv(OUT_DIR / "layerwise_pooling_ablation.csv", index=False)

    best = (
        result[result["classifier"] == "linear_svc"]
        .sort_values("accuracy", ascending=False)
        .head(20)
    )
    best.to_csv(OUT_DIR / "layerwise_pooling_ablation_top20.csv", index=False)
    print(best)
    print("Saved to", OUT_DIR)


if __name__ == "__main__":
    main()
```

Code listing: run_track1_spotcheck.py (1)

```python
import gc
import os
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

import lie_llm_full_semantic_holdout_experiment as full


OUT_DIR = Path(os.getenv("SPOTCHECK_OUT_DIR", "results/experiments/track1_spotcheck_results"))
OUT_DIR.mkdir(parents=True, exist_ok=True)

MODELS = [m.strip() for m in os.getenv("SPOTCHECK_MODELS", "bert-large-uncased").split(",") if m.strip()]
N_BASE = int(os.getenv("SPOTCHECK_N_BASE", "100"))


def run_classifiers(X, labels, split):
    train_mask = split == "train"
    test_mask = split == "test"
    classifiers = {
        "logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1)),
        "linear_svc": make_pipeline(StandardScaler(), LinearSVC(class_weight="balanced", max_iter=20000)),
    }
    rows = []
    for clf_name, clf in classifiers.items():
        clf.fit(X[train_mask], labels[train_mask])
        pred = clf.predict(X[test_mask])
        rows.append({
            "classifier": clf_name,
            "accuracy": float(accuracy_score(labels[test_mask], pred)),
            "n_train": int(train_mask.sum()),
            "n_test": int(test_mask.sum()),
        })
    return rows


def main():
    full.OUT_DIR = str(OUT_DIR)
    full.N_BASE = N_BASE
    os.makedirs(full.OUT_DIR, exist_ok=True)

    pair_df = full.build_dataset()
    prompts, pair_df = full.index_prompts(pair_df)
    pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
    pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)

    labels = pair_df["class"].astype(str).to_numpy()
    split = pair_df["split"].astype(str).to_numpy()
```

Code listing: run_track1_spotcheck.py (2)

```
    src_idx = pair_df["source_idx"].to_numpy()
    tgt_idx = pair_df["target_idx"].to_numpy()

    rows = []
    for model_name in MODELS:
        print(f"\n=== MODEL: {model_name} ===")
        vectors = full.get_vectors(model_name, prompts)
        x_src = vectors[src_idx]
        x_tgt = vectors[tgt_idx]
        reps = {
            "x_only": x_src,
            "y_only": x_tgt,
            "delta": x_tgt - x_src,
            "concat": np.concatenate([x_src, x_tgt], axis=1),
        }
        for rep_name, X in reps.items():
            for row in run_classifiers(X, labels, split):
                row.update({
                    "model": model_name,
                    "representation": rep_name,
                    "random_baseline": 1 / len(set(labels)),
                    "n_base": N_BASE,
                })
                print(model_name, rep_name, row["classifier"], f"{row['accuracy']:.4f}")
                rows.append(row)
        del vectors, reps
        gc.collect()

    result = pd.DataFrame(rows)
    result.to_csv(OUT_DIR / "spotcheck_representation_ablation.csv", index=False)
    pivot = result.pivot_table(index=["model", "classifier"], columns="representation", values="accuracy").reset_index()
    pivot.to_csv(OUT_DIR / "spotcheck_representation_ablation_pivot.csv", index=False)
    print(pivot)


if __name__ == "__main__":
    main()
```

Code listing: run_full_semantic_pooling_ablation.py (1)

```
import gc
import os
import random
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC
from transformers import AutoModel, AutoTokenizer

import lie_llm_full_semantic_holdout_experiment as full


OUT_DIR = Path(os.getenv("POOLING_OUT_DIR", "results/experiments/full_semantic_pooling_ablation_results"))
MODELS = [m.strip() for m in os.getenv("POOLING_MODELS", "bert-base-uncased,roberta-base,gpt2").split(",") if m.strip()]
N_BASE = int(os.getenv("POOLING_N_BASE", "150"))
BATCH_SIZE = int(os.getenv("POOLING_BATCH_SIZE", "32"))
SEED = int(os.getenv("POOLING_SEED", "42"))


def reset_full_dataset_seed():
    full.N_BASE = N_BASE
    full.random.seed(SEED)
    full.np.random.seed(SEED)
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)


def safe_name(model_name):
    return model_name.replace("/", "__")


def last_token_pool(hidden, attention_mask):
    lengths = attention_mask.sum(dim=1).clamp(min=1) - 1
    batch_idx = torch.arange(hidden.shape[0], device=hidden.device)
    return hidden[batch_idx, lengths]


def get_pooled_vectors(model_name, prompts):
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"

    model = AutoModel.from_pretrained(model_name, output_hidden_states=True)
    model.eval()

    pooled = {"mean": [], "cls": [], "last_token": []}

```

Code listing: run_full_semantic_pooling_ablation.py (2)

```
    for start in range(0, len(prompts), BATCH_SIZE):
        batch = prompts[start:start + BATCH_SIZE]
        inputs = tokenizer(
            batch,
            padding=True,
            truncation=True,
            max_length=180,
            return_tensors="pt",
        )

        with torch.no_grad():
            out = model(**inputs)

        hidden = out.hidden_states[-1]
        mask = inputs["attention_mask"]

        pooled["mean"].append(full.mean_pool(hidden, mask).cpu().numpy())
        pooled["cls"].append(hidden[:, 0, :].cpu().numpy())
        pooled["last_token"].append(last_token_pool(hidden, mask).cpu().numpy())

        done = start + len(batch)
        print(f"  {model_name}: {done}/{len(prompts)}", flush=True)

    del model
    gc.collect()

    return {k: np.vstack(v) for k, v in pooled.items()}


def build_features(vectors, pair_df, representation):
    src = vectors[pair_df["source_idx"].to_numpy()]
    tgt = vectors[pair_df["target_idx"].to_numpy()]

    if representation == "x_only":
        return src
    if representation == "y_only":
        return tgt
    if representation == "delta":
        return tgt - src
    if representation == "concat":
        return np.hstack([src, tgt])
    raise ValueError(representation)


def run_one(model_name, pooling, representation, vectors, pair_df):
    X = build_features(vectors, pair_df, representation)
    y = pair_df["class"].astype(str).to_numpy()
    split = pair_df["split"].astype(str).to_numpy()

    train = split == "train"
    test = split == "test"

    classifiers = {
        "logreg": make_pipeline(
```

Code listing: run_full_semantic_pooling_ablation.py (3)

```
            StandardScaler(),
            LogisticRegression(max_iter=5000, class_weight="balanced"),
        ),
        "linear_svc": make_pipeline(
            StandardScaler(),
            LinearSVC(class_weight="balanced", max_iter=20000),
        ),
    }

    rows = []
    for clf_name, clf in classifiers.items():
        clf.fit(X[train], y[train])
        pred = clf.predict(X[test])
        rows.append({
            "model": model_name,
            "pooling": pooling,
            "representation": representation,
            "classifier": clf_name,
            "accuracy": float(accuracy_score(y[test], pred)),
            "n_base": N_BASE,
            "n_train": int(train.sum()),
            "n_test": int(test.sum()),
        })
    return rows


def main():
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    reset_full_dataset_seed()

    pair_df = full.build_dataset()
    prompts, pair_df = full.index_prompts(pair_df)
    pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
    pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)

    rows = []
    for model_name in MODELS:
        print(f"\n=== MODEL: {model_name} ===", flush=True)
        pooled_vectors = get_pooled_vectors(model_name, prompts)

        for pooling, vectors in pooled_vectors.items():
            for representation in ["x_only", "y_only", "concat", "delta"]:
                rows.extend(run_one(model_name, pooling, representation, vectors, pair_df))

    result = pd.DataFrame(rows)
    result.to_csv(OUT_DIR / "full_semantic_pooling_ablation.csv", index=False)
    pivot = result.pivot_table(
        index=["model", "pooling", "classifier"],
        columns="representation",
        values="accuracy",
    ).reset_index()
    pivot.to_csv(OUT_DIR / "full_semantic_pooling_ablation_pivot.csv", index=False)
    print(pivot)

```

Code listing: run_full_semantic_pooling_ablation.py (4)

```

if __name__ == "__main__":
    main()
```

Code listing: run_upat_procrustes_nulls.py (1)

```
import argparse
import os
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.linalg import svd
from sklearn.metrics import accuracy_score, f1_score
from sklearn.preprocessing import normalize

from run_upat_large import UPATAudit


OUT_DIR = Path("results/experiments/upat_large_results")
CSV_DIR = OUT_DIR / "csv"
FIG_DIR = OUT_DIR / "figures"
CSV_DIR.mkdir(parents=True, exist_ok=True)
FIG_DIR.mkdir(parents=True, exist_ok=True)

N_NULL = int(os.getenv("UPAT_PROCRUSTES_NULL_REPEATS", "30"))
SEED = int(os.getenv("UPAT_PROCRUSTES_NULL_SEED", "1729"))


def parse_args():
    parser = argparse.ArgumentParser(
        description="Run UPAT cross-model Procrustes null controls."
    )
    parser.add_argument(
        "--n-null",
        type=int,
        default=N_NULL,
        help="Number of null repeats per source-target direction and null type.",
    )
    parser.add_argument(
        "--seed",
        type=int,
        default=SEED,
        help="Base random seed for null controls.",
    )
    return parser.parse_args()


def make_direction_context(audit, train_model, test_model):
    source_sp = audit.spaces[train_model]
    target_sp = audit.spaces[test_model]

    common_texts = sorted(set(source_sp["texts"]) & set(target_sp["texts"]))
    source_idx = np.array([source_sp["idx"][text] for text in common_texts])
    target_idx = np.array([target_sp["idx"][text] for text in common_texts])

    min_dim = min(source_sp["raw"].shape[1], target_sp["raw"].shape[1])
    source_anchor = source_sp["raw"][source_idx, :min_dim]
```

Code listing: run_upat_procrustes_nulls.py (2)

```
    target_anchor = target_sp["raw"][target_idx, :min_dim]
    source_all = source_sp["raw"][:, :min_dim]
    target_all = target_sp["raw"][:, :min_dim]

    source_mean = source_anchor.mean(axis=0, keepdims=True)
    target_mean = target_anchor.mean(axis=0, keepdims=True)

    source_anchor_c = source_anchor - source_mean
    target_anchor_c = target_anchor - target_mean
    source_all_c = source_all - source_mean
    target_all_c = target_all - target_mean

    return {
        "source_anchor_c": source_anchor_c,
        "target_anchor_c": target_anchor_c,
        "source_all_c": source_all_c,
        "target_all_c": target_all_c,
    }


def apply_fast_procrustes(context, target_order=None):
    target_anchor_c = context["target_anchor_c"]
    if target_order is not None:
        target_anchor_c = target_anchor_c[target_order]

    u, _, vt = svd(context["source_anchor_c"].T @ target_anchor_c, full_matrices=False, check_finite=False)
    r = u @ vt
    return context["source_all_c"] @ r, context["target_all_c"]


def apply_fast_random_orthogonal(context, seed):
    rng = np.random.default_rng(seed)
    dim = context["source_all_c"].shape[1]
    q, _ = np.linalg.qr(rng.normal(size=(dim, dim)))
    return context["source_all_c"] @ q, context["target_all_c"]


def evaluate_transfer(audit, train_model, test_model, context, *, pairing="matched", shuffle_labels=False, random_orthogonal=F
    rng = np.random.default_rng(seed)

    if pairing == "random_pairing":
        target_order = rng.permutation(len(context["target_anchor_c"]))
    elif pairing != "matched":
        raise ValueError(f"Unknown pairing mode: {pairing}")
    else:
        target_order = None

    if random_orthogonal:
        source_aligned, target_aligned = apply_fast_random_orthogonal(context, seed=seed)
    else:
        source_aligned, target_aligned = apply_fast_procrustes(context, target_order=target_order)

    source_deltas = normalize(audit.make_raw_deltas(train_model, source_aligned))
    target_deltas = normalize(audit.make_raw_deltas(test_model, target_aligned))
```

Code listing: run_upat_procrustes_nulls.py (3)

```

    y_train = audit.y[audit.train_mask].copy()
    if shuffle_labels:
        y_train = rng.permutation(y_train)

    pred = audit.fit_predict(
        source_deltas[audit.train_mask],
        y_train,
        target_deltas[audit.test_mask],
        seed=seed,
    )

    return {
        "acc": accuracy_score(audit.y[audit.test_mask], pred),
        "f1": f1_score(audit.y[audit.test_mask], pred, average="macro"),
    }


def run_nulls(audit):
    observed = pd.read_csv(CSV_DIR / "cross_model_procrustes.csv")
    rows = []
    summary_rows = []

    for obs in observed.itertuples(index=False):
        train_model = obs.train_model
        test_model = obs.test_model
        if train_model == test_model:
            continue

        print(f"\nNulls: {train_model} -> {test_model}", flush=True)
        context = make_direction_context(audit, train_model, test_model)

        for null_type in ["random_pairing", "random_labels", "random_orthogonal"]:
            f1_values = []
            acc_values = []

            for repeat in range(N_NULL):
                seed = SEED + repeat
                if null_type == "random_pairing":
                    metrics = evaluate_transfer(
                        audit,
                        train_model,
                        test_model,
                        context,
                        pairing="random_pairing",
                        shuffle_labels=False,
                        seed=seed,
                    )
                else:
                    metrics = evaluate_transfer(
                        audit,
                        train_model,
                        test_model,
                        context,
```

Code listing: run_upat_procrustes_nulls.py (4)

```
                    pairing="matched",
                    shuffle_labels=null_type == "random_labels",
                    random_orthogonal=null_type == "random_orthogonal",
                    seed=seed,
                )

            f1_values.append(metrics["f1"])
            acc_values.append(metrics["acc"])
            rows.append({
                "train_model": train_model,
                "test_model": test_model,
                "null_type": null_type,
                "repeat": repeat,
                "null_acc": metrics["acc"],
                "null_f1": metrics["f1"],
                "observed_aligned_f1": obs.aligned_f1,
                "observed_raw_f1": obs.raw_f1,
                "observed_gain_f1": obs.gain_f1,
            })

            if (repeat + 1) % 100 == 0 or repeat + 1 == N_NULL:
                print(f"  {null_type}: {repeat + 1}/{N_NULL}", flush=True)

        f1_arr = np.array(f1_values)
        acc_arr = np.array(acc_values)
        summary_rows.append({
            "train_model": train_model,
            "test_model": test_model,
            "null_type": null_type,
            "n_null": N_NULL,
            "observed_raw_f1": obs.raw_f1,
            "observed_aligned_f1": obs.aligned_f1,
            "observed_gain_f1": obs.gain_f1,
            "null_mean_f1": f1_arr.mean(),
            "null_std_f1": f1_arr.std(ddof=1) if len(f1_arr) > 1 else 0.0,
            "null_max_f1": f1_arr.max(),
            "null_mean_acc": acc_arr.mean(),
            "p_empirical_null_ge_observed": ((f1_arr >= obs.aligned_f1).sum() + 1) / (N_NULL + 1),
            "observed_minus_null_mean_f1": obs.aligned_f1 - f1_arr.mean(),
        })

    pd.DataFrame(rows).to_csv(CSV_DIR / "procrustes_null_raw.partial.csv", index=False)
    pd.DataFrame(summary_rows).to_csv(CSV_DIR / "procrustes_null_summary.partial.csv", index=False)

raw_df = pd.DataFrame(rows)
summary_df = pd.DataFrame(summary_rows)

raw_df.to_csv(CSV_DIR / "procrustes_null_raw.csv", index=False)
summary_df.to_csv(CSV_DIR / "procrustes_null_summary.csv", index=False)
(CSV_DIR / "procrustes_null_raw.partial.csv").unlink(missing_ok=True)
(CSV_DIR / "procrustes_null_summary.partial.csv").unlink(missing_ok=True)
return raw_df, summary_df
```

Code listing: run_upat_procrustes_nulls.py (5)

```
def plot_summary(summary_df):
    if summary_df.empty:
        return

    for null_type, df in summary_df.groupby("null_type"):
        labels = [
            f"{row.train_model.split('/')[-1]}\n->{row.test_model.split('/')[-1]}"
            for row in df.itertuples(index=False)
        ]
        x = np.arange(len(df))

        plt.figure(figsize=(max(12, len(df) * 0.45), 6))
        plt.bar(x, df["observed_aligned_f1"], label="observed aligned F1")
        plt.bar(x, df["null_mean_f1"], alpha=0.65, label=f"{null_type} mean F1")
        plt.xticks(x, labels, rotation=70, ha="right", fontsize=7)
        plt.ylabel("Macro F1")
        plt.title(f"UPAT Procrustes null baseline: {null_type}")
        plt.legend()
        plt.tight_layout()
        plt.savefig(FIG_DIR / f"10_procrustes_null_{null_type}.png", dpi=220, bbox_inches="tight")
        plt.close()


def main():
    global N_NULL, SEED
    args = parse_args()
    N_NULL = args.n_null
    SEED = args.seed
    print(f"Procrustes null repeats: {N_NULL}; seed: {SEED}", flush=True)

    audit = UPATAudit()
    audit.build_dataset()
    audit.extract_all_spaces()

    cross_path = CSV_DIR / "cross_model_procrustes.csv"
    if not cross_path.exists():
        audit.run_cross_model_procrustes()

    _, summary_df = run_nulls(audit)
    plot_summary(summary_df)
    print(f"\nSaved: {CSV_DIR / 'procrustes_null_raw.csv'}")
    print(f"Saved: {CSV_DIR / 'procrustes_null_summary.csv'}")


if __name__ == "__main__":
    sys.exit(main())
```

Code listing: run_upat_alignment_size_heldout.py (1)

```
import argparse
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.linalg import svd
from sklearn.metrics import accuracy_score, f1_score
from sklearn.preprocessing import normalize

from run_upat_large import UPATAudit
from upat_dataset import UPATDataset, UPATDatasetConfig


OUT_DIR = Path("results/experiments/upat_large_results")
CSV_DIR = OUT_DIR / "csv"
FIG_DIR = OUT_DIR / "figures"
CSV_DIR.mkdir(parents=True, exist_ok=True)
FIG_DIR.mkdir(parents=True, exist_ok=True)


def parse_args():
    parser = argparse.ArgumentParser(
        description="Run held-out anchor alignment-size curves for UPAT cross-model Procrustes transfer."
    )
    parser.add_argument(
        "--sizes",
        default="25,50,100,250,500,1000",
        help="Comma-separated held-out alignment anchor sizes.",
    )
    parser.add_argument(
        "--repeats",
        type=int,
        default=10,
        help="Repeats per source-target direction and alignment size.",
    )
    parser.add_argument(
        "--anchor-target-count",
        type=int,
        default=1200,
        help="Number of unique held-out anchor texts to generate before subsampling.",
    )
    parser.add_argument(
        "--seed",
        type=int,
        default=31415,
        help="Base random seed.",
    )
    return parser.parse_args()


def collect_texts(df):
    return set(df["source"]) | set(df["target"])
```

Code listing: run_upat_alignment_size_heldout.py (2)

```python


def build_heldout_anchor_texts(main_texts, target_count, seed):
    anchor_texts = []
    seen = set(main_texts)

    for offset in range(1000):
        cfg = UPATDatasetConfig(
            train_templates=200,
            test_templates=200,
            seed_train=seed + 2 * offset,
            seed_test=seed + 2 * offset + 1,
        )
        df = UPATDataset(cfg).build()

        for text in sorted(collect_texts(df)):
            if text in seen:
                continue
            seen.add(text)
            anchor_texts.append(text)
            if len(anchor_texts) >= target_count:
                return anchor_texts

    raise RuntimeError(
        f"Only generated {len(anchor_texts)} held-out anchor texts; requested {target_count}."
    )


def fit_procrustes(source_anchor, target_anchor, source_all, target_all):
    source_mean = source_anchor.mean(axis=0, keepdims=True)
    target_mean = target_anchor.mean(axis=0, keepdims=True)

    source_anchor_c = source_anchor - source_mean
    target_anchor_c = target_anchor - target_mean
    source_all_c = source_all - source_mean
    target_all_c = target_all - target_mean

    min_dim = min(
        source_anchor_c.shape[1],
        target_anchor_c.shape[1],
        source_all_c.shape[1],
        target_all_c.shape[1],
    )
    source_anchor_c = source_anchor_c[:, :min_dim]
    target_anchor_c = target_anchor_c[:, :min_dim]
    source_all_c = source_all_c[:, :min_dim]
    target_all_c = target_all_c[:, :min_dim]

    u, _, vt = svd(source_anchor_c.T @ target_anchor_c, full_matrices=False, check_finite=False)
    r = u @ vt
    return source_all_c @ r, target_all_c


def evaluate_direction(audit, train_model, test_model, source_anchor, target_anchor, seed):
```

Code listing: run_upat_alignment_size_heldout.py (3)

```
    source_sp = audit.spaces[train_model]
    target_sp = audit.spaces[test_model]

    source_aligned, target_aligned = fit_procrustes(
        source_anchor=source_anchor,
        target_anchor=target_anchor,
        source_all=source_sp["raw"],
        target_all=target_sp["raw"],
    )

    source_deltas = normalize(audit.make_raw_deltas(train_model, source_aligned))
    target_deltas = normalize(audit.make_raw_deltas(test_model, target_aligned))

    pred = audit.fit_predict(
        source_deltas[audit.train_mask],
        audit.y[audit.train_mask],
        target_deltas[audit.test_mask],
        seed=seed,
    )

    return {
        "acc": accuracy_score(audit.y[audit.test_mask], pred),
        "f1": f1_score(audit.y[audit.test_mask], pred, average="macro"),
    }


def run_curve(audit, anchor_spaces, anchor_texts, sizes, repeats, seed):
    observed = pd.read_csv(CSV_DIR / "cross_model_procrustes.csv")
    observed = observed[observed["train_model"] != observed["test_model"]]
    rng = np.random.default_rng(seed)
    rows = []

    for obs in observed.itertuples(index=False):
        train_model = obs.train_model
        test_model = obs.test_model
        print(f"\nHeld-out alignment: {train_model} -> {test_model}", flush=True)

        source_anchor_all = anchor_spaces[train_model]
        target_anchor_all = anchor_spaces[test_model]

        for size in sizes:
            real_size = min(size, len(anchor_texts))
            for repeat in range(repeats):
                sample = rng.choice(len(anchor_texts), size=real_size, replace=False)
                metrics = evaluate_direction(
                    audit=audit,
                    train_model=train_model,
                    test_model=test_model,
                    source_anchor=source_anchor_all[sample],
                    target_anchor=target_anchor_all[sample],
                    seed=seed + 1000 * repeat + real_size,
                )
                rows.append({
                    "train_model": train_model,
```

Code listing: run_upat_alignment_size_heldout.py (4)

```
                    "test_model": test_model,
                    "alignment_size": real_size,
                    "repeat": repeat,
                    "heldout_anchor_pool_size": len(anchor_texts),
                    "observed_full_anchor_f1": obs.aligned_f1,
                    "observed_raw_f1": obs.raw_f1,
                    "heldout_acc": metrics["acc"],
                    "heldout_f1": metrics["f1"],
                    "heldout_minus_raw_f1": metrics["f1"] - obs.raw_f1,
                    "heldout_minus_full_anchor_f1": metrics["f1"] - obs.aligned_f1,
                })

            print(f"  size {real_size}: {repeats}/{repeats}", flush=True)

        pd.DataFrame(rows).to_csv(CSV_DIR / "heldout_alignment_curve_raw.partial.csv", index=False)

    raw_df = pd.DataFrame(rows)
    summary_df = (
        raw_df
        .groupby("alignment_size")
        .agg(
            mean_f1=("heldout_f1", "mean"),
            std_f1=("heldout_f1", "std"),
            mean_acc=("heldout_acc", "mean"),
            mean_full_anchor_f1=("observed_full_anchor_f1", "mean"),
            mean_raw_f1=("observed_raw_f1", "mean"),
            mean_gap_to_full_anchor=("heldout_minus_full_anchor_f1", "mean"),
            mean_gain_over_raw=("heldout_minus_raw_f1", "mean"),
            n=("heldout_f1", "count"),
        )
        .reset_index()
    )
    direction_summary_df = (
        raw_df
        .groupby(["train_model", "test_model", "alignment_size"])
        .agg(
            mean_f1=("heldout_f1", "mean"),
            std_f1=("heldout_f1", "std"),
            mean_acc=("heldout_acc", "mean"),
            observed_full_anchor_f1=("observed_full_anchor_f1", "first"),
            observed_raw_f1=("observed_raw_f1", "first"),
            mean_gap_to_full_anchor=("heldout_minus_full_anchor_f1", "mean"),
            mean_gain_over_raw=("heldout_minus_raw_f1", "mean"),
            n=("heldout_f1", "count"),
        )
        .reset_index()
    )

    raw_df.to_csv(CSV_DIR / "heldout_alignment_curve_raw.csv", index=False)
    summary_df.to_csv(CSV_DIR / "heldout_alignment_curve_summary.csv", index=False)
    direction_summary_df.to_csv(CSV_DIR / "heldout_alignment_curve_by_direction.csv", index=False)
    (CSV_DIR / "heldout_alignment_curve_raw.partial.csv").unlink(missing_ok=True)
    return raw_df, summary_df, direction_summary_df

```

Code listing: run_upat_alignment_size_heldout.py (5)

```

def plot_curve(summary_df, direction_summary_df):
    plt.figure(figsize=(8, 5))
    ordered = summary_df.sort_values("alignment_size")
    plt.errorbar(
        ordered["alignment_size"],
        ordered["mean_f1"],
        yerr=ordered["std_f1"],
        marker="o",
        capsize=4,
        label="held-out anchor Procrustes",
    )
    plt.plot(
        ordered["alignment_size"],
        ordered["mean_raw_f1"],
        linestyle="--",
        label="raw cross-model mean",
    )
    plt.plot(
        ordered["alignment_size"],
        ordered["mean_full_anchor_f1"],
        linestyle=":",
        label="full-anchor Procrustes mean",
    )
    plt.xscale("log")
    plt.xlabel("Held-out alignment anchor texts")
    plt.ylabel("Macro F1")
    plt.title("UPAT held-out alignment-size curve")
    plt.grid(True, which="both", alpha=0.35)
    plt.legend()
    plt.tight_layout()
    plt.savefig(FIG_DIR / "11_heldout_alignment_size_curve.png", dpi=220, bbox_inches="tight")
    plt.close()

    largest_size = direction_summary_df["alignment_size"].max()
    largest = direction_summary_df[direction_summary_df["alignment_size"] == largest_size].copy()
    labels = [
        f"{row.train_model.split('/')[-1]}\n->{row.test_model.split('/')[-1]}"
        for row in largest.itertuples(index=False)
    ]
    x = np.arange(len(largest))
    plt.figure(figsize=(max(12, len(largest) * 0.45), 6))
    plt.bar(x, largest["observed_raw_f1"], label="raw F1")
    plt.bar(x, largest["mean_f1"], alpha=0.75, label=f"held-out alignment F1, n={largest_size}")
    plt.bar(x, largest["observed_full_anchor_f1"], alpha=0.45, label="full-anchor alignment F1")
    plt.xticks(x, labels, rotation=70, ha="right", fontsize=7)
    plt.ylabel("Macro F1")
    plt.title("UPAT held-out alignment by direction")
    plt.legend()
    plt.tight_layout()
    plt.savefig(FIG_DIR / "11_heldout_alignment_by_direction.png", dpi=220, bbox_inches="tight")
    plt.close()


```

Code listing: run_upat_alignment_size_heldout.py (6)

```
def main():
    args = parse_args()
    sizes = [int(x.strip()) for x in args.sizes.split(",") if x.strip()]
    if not sizes:
        raise ValueError("At least one alignment size is required.")

    audit = UPATAudit()
    audit.build_dataset()

    main_texts = collect_texts(audit.df)
    anchor_target_count = max(args.anchor_target_count, max(sizes))
    anchor_texts = build_heldout_anchor_texts(
        main_texts=main_texts,
        target_count=anchor_target_count,
        seed=args.seed,
    )
    pd.DataFrame({"text": anchor_texts}).to_csv(CSV_DIR / "heldout_alignment_anchor_texts.csv", index=False)
    print(f"Held-out anchor texts: {len(anchor_texts)}", flush=True)

    audit.extract_all_spaces()

    cross_path = CSV_DIR / "cross_model_procrustes.csv"
    if not cross_path.exists():
        audit.run_cross_model_procrustes()

    anchor_spaces = {}
    for model_name in audit.procrustes_models:
        print(f"\nEXTRACTING HELD-OUT ANCHORS {model_name}", flush=True)
        anchor_spaces[model_name] = audit.get_embeddings(model_name, anchor_texts)

    _, summary_df, direction_summary_df = run_curve(
        audit=audit,
        anchor_spaces=anchor_spaces,
        anchor_texts=anchor_texts,
        sizes=sizes,
        repeats=args.repeats,
        seed=args.seed,
    )
    plot_curve(summary_df, direction_summary_df)
    print(f"\nSaved: {CSV_DIR / 'heldout_alignment_curve_raw.csv'}")
    print(f"Saved: {CSV_DIR / 'heldout_alignment_curve_summary.csv'}")
    print(f"Saved: {CSV_DIR / 'heldout_alignment_curve_by_direction.csv'}")


if __name__ == "__main__":
    sys.exit(main())
```

Code listing: run_upat_rise_aware_comparison.py (1)

```
import argparse
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.linalg import svd
from sklearn.metrics import accuracy_score, f1_score
from sklearn.preprocessing import normalize

from run_upat_large import UPATAudit


OUT_DIR = Path("results/experiments/upat_large_results")
CSV_DIR = OUT_DIR / "csv"
FIG_DIR = OUT_DIR / "figures"
CSV_DIR.mkdir(parents=True, exist_ok=True)
FIG_DIR.mkdir(parents=True, exist_ok=True)


def parse_args():
    parser = argparse.ArgumentParser(
        description="Compare delta, MDV, and RISE-style prototype baselines on UPAT."
    )
    parser.add_argument("--seed", type=int, default=2718)
    return parser.parse_args()


def unit(x):
    return normalize(x)


def cosine_rows(a, b):
    return np.sum(unit(a) * unit(b), axis=1)


def log_map_sphere(x, y, eps=1e-7):
    dot = np.clip(np.sum(x * y, axis=1, keepdims=True), -1 + eps, 1 - eps)
    theta = np.arccos(dot)
    tangent = y - dot * x
    sin_theta = np.sin(theta)
    return tangent * (theta / np.maximum(sin_theta, eps))


def exp_map_sphere(x, tangent, eps=1e-7):
    norm = np.linalg.norm(tangent, axis=1, keepdims=True)
    direction = tangent / np.maximum(norm, eps)
    out = np.cos(norm) * x + np.sin(norm) * direction
    small = norm[:, 0] < eps
    if np.any(small):
        out[small] = x[small]
    return unit(out)

```

Code listing: run_upat_rise_aware_comparison.py (2)

```

def householder_to_e1_apply(x, z, eps=1e-7):
    e1 = np.zeros_like(x)
    e1[:, 0] = 1.0
    u = x - e1
    denom = np.sum(u * u, axis=1, keepdims=True)
    out = z.copy()

    regular = denom[:, 0] > eps
    if np.any(regular):
        u_reg = u[regular]
        z_reg = z[regular]
        denom_reg = denom[regular]
        out[regular] = z_reg - 2.0 * u_reg * (np.sum(u_reg * z_reg, axis=1, keepdims=True) / denom_reg)

    antipodal = (~regular) & (x[:, 0] < 0)
    if np.any(antipodal):
        out[antipodal, 0] *= -1.0

    return out


def fit_procrustes(source_anchor, target_anchor, source_all, target_all):
    source_mean = source_anchor.mean(axis=0, keepdims=True)
    target_mean = target_anchor.mean(axis=0, keepdims=True)

    source_anchor_c = source_anchor - source_mean
    target_anchor_c = target_anchor - target_mean
    source_all_c = source_all - source_mean
    target_all_c = target_all - target_mean

    min_dim = min(source_anchor_c.shape[1], target_anchor_c.shape[1], source_all_c.shape[1], target_all_c.shape[1])
    source_anchor_c = source_anchor_c[:, :min_dim]
    target_anchor_c = target_anchor_c[:, :min_dim]
    source_all_c = source_all_c[:, :min_dim]
    target_all_c = target_all_c[:, :min_dim]

    u, _, vt = svd(source_anchor_c.T @ target_anchor_c, full_matrices=False, check_finite=False)
    r = u @ vt
    return source_all_c @ r, target_all_c


def make_full_anchor_alignment(audit, train_model, test_model):
    source_sp = audit.spaces[train_model]
    target_sp = audit.spaces[test_model]
    common_texts = sorted(set(source_sp["texts"]) & set(target_sp["texts"]))
    source_idx = np.array([source_sp["idx"][text] for text in common_texts])
    target_idx = np.array([target_sp["idx"][text] for text in common_texts])
    return fit_procrustes(
        source_anchor=source_sp["raw"][source_idx],
        target_anchor=target_sp["raw"][target_idx],
        source_all=source_sp["raw"],
        target_all=target_sp["raw"],
    )
```

Code listing: run_upat_rise_aware_comparison.py (3)

```


def rows_from_vectors(audit, model_name, vectors):
    sp = audit.spaces[model_name]
    idx = sp["idx"]
    x = np.array([vectors[idx[row.source]] for row in audit.df.itertuples(index=False)])
    y = np.array([vectors[idx[row.target]] for row in audit.df.itertuples(index=False)])
    return x, y


def learn_mdv_raw(x_train, y_train, labels_train, class_ids):
    deltas = y_train - x_train
    return {
        cls: deltas[labels_train == cls].mean(axis=0)
        for cls in class_ids
    }


def predict_mdv_raw(x_test, prototypes, labels_test):
    return np.vstack([x + prototypes[label] for x, label in zip(x_test, labels_test)])


def learn_mdv_unit(x_train, y_train, labels_train, class_ids):
    x_u = unit(x_train)
    y_u = unit(y_train)
    deltas = y_u - x_u
    return {
        cls: deltas[labels_train == cls].mean(axis=0)
        for cls in class_ids
    }


def predict_mdv_unit(x_test, prototypes, labels_test):
    x_u = unit(x_test)
    return unit(np.vstack([x + prototypes[label] for x, label in zip(x_u, labels_test)]))


def learn_rise_style(x_train, y_train, labels_train, class_ids):
    x_u = unit(x_train)
    y_u = unit(y_train)
    tangent = log_map_sphere(x_u, y_u)
    canonical = householder_to_e1_apply(x_u, tangent)
    prototypes = {}
    for cls in class_ids:
        proto = canonical[labels_train == cls].mean(axis=0)
        proto[0] = 0.0
        prototypes[cls] = proto
    return prototypes


def predict_rise_style(x_test, prototypes, labels_test):
    x_u = unit(x_test)
    proto = np.vstack([prototypes[label] for label in labels_test])
    tangent = householder_to_e1_apply(x_u, proto)
```

Code listing: run_upat_rise_aware_comparison.py (4)

```
    return exp_map_sphere(x_u, tangent)


def nearest_target_metrics(y_pred, y_true, labels_true):
    y_pred_u = unit(y_pred)
    y_true_u = unit(y_true)
    sim = y_pred_u @ y_true_u.T
    nearest = sim.argmax(axis=1)
    pred_labels = labels_true[nearest]
    return {
        "retrieval_top1_acc": float(np.mean(nearest == np.arange(len(nearest)))),
        "retrieval_label_acc": accuracy_score(labels_true, pred_labels),
        "retrieval_label_f1": f1_score(labels_true, pred_labels, average="macro"),
    }


def classifier_transfer_metrics(audit, x_train, y_train, x_test, y_test, seed):
    train_delta = normalize(y_train - x_train)
    test_delta = normalize(y_test - x_test)
    pred = audit.fit_predict(
        train_delta[audit.train_mask],
        audit.y[audit.train_mask],
        test_delta[audit.test_mask],
        seed=seed,
    )
    return {
        "delta_classifier_acc": accuracy_score(audit.y[audit.test_mask], pred),
        "delta_classifier_f1": f1_score(audit.y[audit.test_mask], pred, average="macro"),
    }


def evaluate_prototype_method(audit, method, train_model, test_model, train_x, train_y, test_x, test_y, seed):
    train_idx = np.where(audit.train_mask)[0]
    test_idx = np.where(audit.test_mask)[0]
    class_ids = np.unique(audit.y)

    if method == "mdv_raw":
        prototypes = learn_mdv_raw(train_x[train_idx], train_y[train_idx], audit.y[train_idx], class_ids)
        pred_y = predict_mdv_raw(test_x[test_idx], prototypes, audit.y[test_idx])
    elif method == "mdv_unit":
        prototypes = learn_mdv_unit(train_x[train_idx], train_y[train_idx], audit.y[train_idx], class_ids)
        pred_y = predict_mdv_unit(test_x[test_idx], prototypes, audit.y[test_idx])
    elif method == "rise_style":
        prototypes = learn_rise_style(train_x[train_idx], train_y[train_idx], audit.y[train_idx], class_ids)
        pred_y = predict_rise_style(test_x[test_idx], prototypes, audit.y[test_idx])
    else:
        raise ValueError(f"Unknown method: {method}")

    y_true = test_y[test_idx]
    cos = cosine_rows(pred_y, y_true)
    retrieval = nearest_target_metrics(pred_y, y_true, audit.y[test_idx])

    return {
        "train_model": train_model,
```

Code listing: run_upat_rise_aware_comparison.py (5)

```
        "test_model": test_model,
        "method": method,
        "mean_target_cosine": cos.mean(),
        "std_target_cosine": cos.std(ddof=1),
        "median_target_cosine": np.median(cos),
        **retrieval,
    }


def run_within_model(audit, seed):
    rows = []
    for model_name in audit.procrustes_models:
        print(f"\nWithin-model prototype comparison: {model_name}", flush=True)
        sp = audit.spaces[model_name]
        train_x = sp["x_raw"]
        train_y = sp["y_raw"]
        test_x = sp["x_raw"]
        test_y = sp["y_raw"]
        clf = classifier_transfer_metrics(audit, train_x, train_y, test_x, test_y, seed=seed)
        for method in ["mdv_raw", "mdv_unit", "rise_style"]:
            row = evaluate_prototype_method(
                audit, method, model_name, model_name, train_x, train_y, test_x, test_y, seed
            )
            row.update({"comparison": "within_model", **clf})
            rows.append(row)
    return rows


def run_cross_model(audit, seed):
    observed = pd.read_csv(CSV_DIR / "cross_model_procrustes.csv")
    rows = []

    for obs in observed.itertuples(index=False):
        train_model = obs.train_model
        test_model = obs.test_model
        if train_model == test_model:
            continue

        print(f"\nCross-model prototype comparison: {train_model} -> {test_model}", flush=True)
        source_aligned, target_aligned = make_full_anchor_alignment(audit, train_model, test_model)
        train_x, train_y = rows_from_vectors(audit, train_model, source_aligned)
        test_x, test_y = rows_from_vectors(audit, test_model, target_aligned)
        clf = classifier_transfer_metrics(audit, train_x, train_y, test_x, test_y, seed=seed)

        for method in ["mdv_raw", "mdv_unit", "rise_style"]:
            row = evaluate_prototype_method(
                audit, method, train_model, test_model, train_x, train_y, test_x, test_y, seed
            )
            row.update({
                "comparison": "cross_model_full_anchor",
                "observed_raw_f1": obs.raw_f1,
                "observed_aligned_f1": obs.aligned_f1,
                **clf,
            })
```

Code listing: run_upat_rise_aware_comparison.py (6)

```
            rows.append(row)
    return rows


def summarize(raw_df):
    return (
        raw_df
        .groupby(["comparison", "method"])
        .agg(
            mean_target_cosine=("mean_target_cosine", "mean"),
            std_target_cosine=("mean_target_cosine", "std"),
            mean_retrieval_top1_acc=("retrieval_top1_acc", "mean"),
            mean_retrieval_label_acc=("retrieval_label_acc", "mean"),
            mean_retrieval_label_f1=("retrieval_label_f1", "mean"),
            mean_delta_classifier_f1=("delta_classifier_f1", "mean"),
            n=("mean_target_cosine", "count"),
        )
        .reset_index()
    )


def plot_summary(summary_df):
    for metric, ylabel, filename in [
        ("mean_target_cosine", "Mean target cosine", "12_rise_aware_target_cosine.png"),
        ("mean_retrieval_label_f1", "Nearest-target label macro F1", "12_rise_aware_retrieval_f1.png"),
    ]:
        pivot = summary_df.pivot(index="method", columns="comparison", values=metric)
        ax = pivot.plot(kind="bar", figsize=(8, 5))
        ax.set_ylabel(ylabel)
        ax.set_title("UPAT RISE-aware prototype comparison")
        ax.grid(axis="y", alpha=0.35)
        plt.xticks(rotation=25, ha="right")
        plt.tight_layout()
        plt.savefig(FIG_DIR / filename, dpi=220, bbox_inches="tight")
        plt.close()


def main():
    args = parse_args()
    audit = UPATAudit()
    audit.build_dataset()
    audit.extract_all_spaces()

    cross_path = CSV_DIR / "cross_model_procrustes.csv"
    if not cross_path.exists():
        audit.run_cross_model_procrustes()

    raw_rows = []
    raw_rows.extend(run_within_model(audit, seed=args.seed))
    raw_rows.extend(run_cross_model(audit, seed=args.seed))

    raw_df = pd.DataFrame(raw_rows)
    summary_df = summarize(raw_df)
    raw_df.to_csv(CSV_DIR / "rise_aware_comparison_raw.csv", index=False)
```

Code listing: run_upat_rise_aware_comparison.py (7)

```
    summary_df.to_csv(CSV_DIR / "rise_aware_comparison_summary.csv", index=False)
    plot_summary(summary_df)

    print(f"\nSaved: {CSV_DIR / 'rise_aware_comparison_raw.csv'}")
    print(f"Saved: {CSV_DIR / 'rise_aware_comparison_summary.csv'}")


if __name__ == "__main__":
    sys.exit(main())
```

Code listing: analyze_confusion_negation.py (1)

```
from pathlib import Path

import pandas as pd


ROOT = Path(__file__).resolve().parents[1]
EXP = ROOT / "results" / "experiments"
OUT = ROOT / "results"
OUT.mkdir(exist_ok=True)


def parse_name(path):
    stem = path.stem.replace("confusion_", "")
    if stem.endswith("_linear_svc"):
        return stem[:-11].replace("__", "/"), "linear_svc"
    if stem.endswith("_logreg"):
        return stem[:-7].replace("__", "/"), "logreg"
    raise ValueError(path.name)


def load_confusion(path):
    model, classifier = parse_name(path)
    cm = pd.read_csv(path, index_col=0)
    labels = list(cm.index)

    rows = []
    errors = []

    for label in labels:
        total = float(cm.loc[label].sum())
        correct = float(cm.loc[label, label])
        recall = correct / total if total else 0.0
        rows.append({
            "source": "full_semantic",
            "model": model,
            "classifier": classifier,
            "class": label,
            "support": int(total),
            "correct": int(correct),
            "recall": recall,
            "error_rate": 1.0 - recall,
            "is_negation": label == "negation",
        })

        for pred in labels:
            if pred == label:
                continue
            count = int(cm.loc[label, pred])
            if count:
                errors.append({
                    "source": "full_semantic",
                    "model": model,
                    "classifier": classifier,
                    "true_class": label,
```

Code listing: analyze_confusion_negation.py (2)

```
                    "predicted_class": pred,
                    "count": count,
                    "true_support": int(total),
                    "share_of_true": count / total if total else 0.0,
                    "involves_negation": label == "negation" or pred == "negation",
                })

    return rows, errors


def main():
    paths = sorted((EXP / "lie_llm_full_semantic_holdout_results").glob("confusion_*.csv"))
    all_rows = []
    all_errors = []

    for path in paths:
        rows, errors = load_confusion(path)
        all_rows.extend(rows)
        all_errors.extend(errors)

    class_df = pd.DataFrame(all_rows)
    error_df = pd.DataFrame(all_errors)

    class_df.to_csv(OUT / "confusion_class_recall.csv", index=False)
    error_df.sort_values(["count", "share_of_true"], ascending=False).to_csv(
        OUT / "confusion_top_errors.csv",
        index=False,
    )

    neg_summary = (
        class_df
        .assign(group=lambda d: d["is_negation"].map({True: "negation", False: "non_negation"}))
        .groupby(["source", "model", "classifier", "group"])
        .agg(
            mean_recall=("recall", "mean"),
            mean_error_rate=("error_rate", "mean"),
            n_classes=("class", "count"),
        )
        .reset_index()
    )
    neg_summary.to_csv(OUT / "confusion_negation_summary.csv", index=False)

    rank_df = class_df.copy()
    rank_df["recall_rank_low_is_hard"] = rank_df.groupby(["model", "classifier"])["recall"].rank(method="min")
    rank_df.sort_values(["model", "classifier", "recall"]).to_csv(
        OUT / "confusion_class_recall_ranked.csv",
        index=False,
    )

    print("Saved:")
    for name in [
        "confusion_class_recall.csv",
        "confusion_top_errors.csv",
        "confusion_negation_summary.csv",
```

Code listing: analyze_confusion_negation.py (3)

```
        "confusion_class_recall_ranked.csv",
    ]:
        print(OUT / name)


if __name__ == "__main__":
    main()
```

Code listing: build_signed_permutation_multiple_testing.py (1)

```
from pathlib import Path

import numpy as np
import pandas as pd


ROOT = Path(__file__).resolve().parents[1]
EXP = ROOT / "results" / "experiments"
OUT = EXP / "lie_algebraic_identities_results" / "csv"


def benjamini_hochberg(p_values):
    p = np.asarray(p_values, dtype=float)
    n = len(p)
    order = np.argsort(p)
    ranked = p[order]
    adj = np.empty(n, dtype=float)
    prev = 1.0
    for i in range(n - 1, -1, -1):
        rank = i + 1
        val = min(prev, ranked[i] * n / rank)
        prev = val
        adj[order[i]] = val
    return np.clip(adj, 0, 1)


def bootstrap_p_less_than_one(values, n_boot=20000, seed=42):
    rng = np.random.default_rng(seed)
    values = np.asarray(values, dtype=float)
    idx = rng.integers(0, len(values), size=(n_boot, len(values)))
    means = values[idx].mean(axis=1)
    p = (np.sum(means >= 1.0) + 1) / (n_boot + 1)
    return float(p)


def main():
    files = [
        EXP / "lie_algebraic_identities_results" / "csv" / "jacobi_raw_all_models.csv",
        EXP / "lie_algebraic_identities_decoder_results" / "csv" / "jacobi_raw_all_models.csv",
    ]
    raw = pd.concat([pd.read_csv(path) for path in files if path.exists()], ignore_index=True)

    rows = []
    for (model, triple), group in raw.groupby(["model", "triple"]):
        ratios = group["jacobi_to_null_mean_ratio"].to_numpy()
        mean_ratio = float(ratios.mean())
        p_less = bootstrap_p_less_than_one(ratios)
        rows.append({
            "model": model,
            "triple": triple,
            "n": int(len(group)),
            "mean_ratio_to_null": mean_ratio,
            "bootstrap_p_ratio_less_than_1": p_less,
            "direction": "below_null" if mean_ratio < 1 else "above_null",
```

Code listing: build_signed_permutation_multiple_testing.py (2)

```
        })

    result = pd.DataFrame(rows).sort_values(["bootstrap_p_ratio_less_than_1", "model", "triple"])
    result["bonferroni_p"] = np.minimum(result["bootstrap_p_ratio_less_than_1"] * len(result), 1.0)
    result["bh_fdr_p"] = benjamini_hochberg(result["bootstrap_p_ratio_less_than_1"].to_numpy())
    result["passes_bonferroni_0_05"] = result["bonferroni_p"] < 0.05
    result["passes_bh_fdr_0_05"] = result["bh_fdr_p"] < 0.05

    out_path = OUT / "signed_permutation_multiple_testing.csv"
    result.to_csv(out_path, index=False)
    print(out_path)
    print(result)


if __name__ == "__main__":
    main()
```